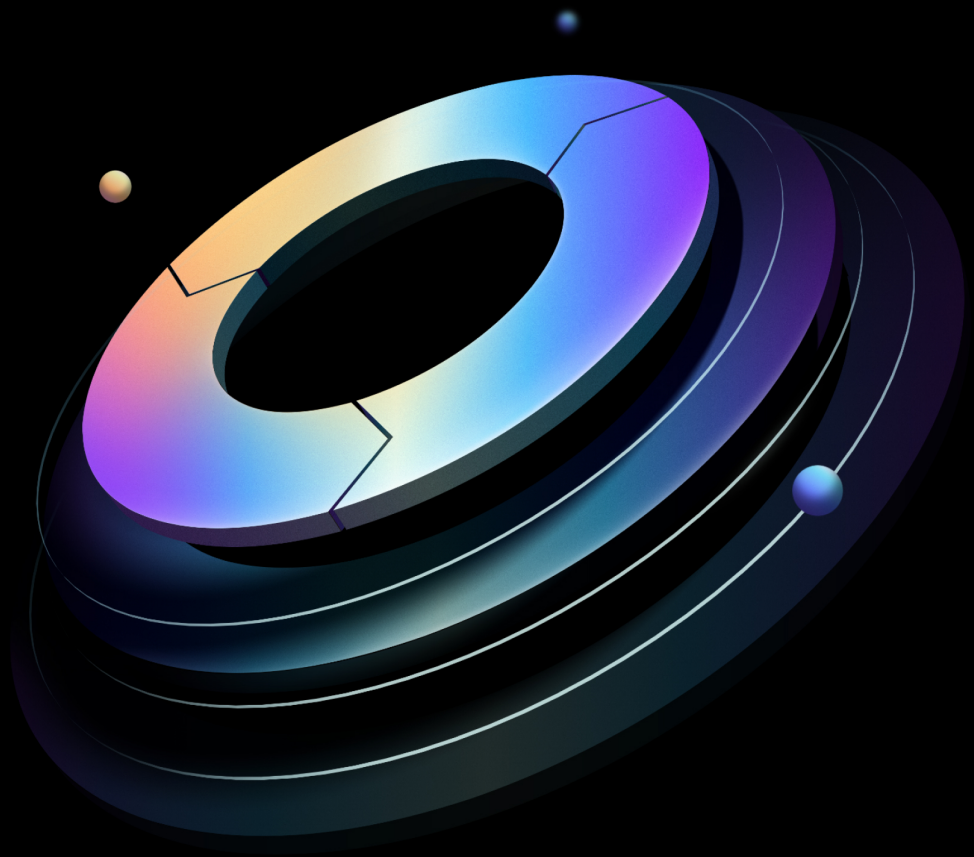
HashiCorp

Terraform Administration Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **8**
1.1 Why use HashiCorp Validated Designs? . . . 8
1.2 Audience . . . 8

2 People and process **9**
2.0.1 Organizing for industrialized cloud adoption . . . 9
2.0.2 Platform team . . . 10
2.0.3 Security team . . . 12
2.0.4 Producers/consumers . . . 13
2.0.5 Organizational big picture . . . 14
2.0.6 Golden IaC workflows . . . 16

3 Consumption models **18**
3.1 Service catalog . . . 20
3.1.1 Vending portal . . . 20
3.1.2 Pre-configured components . . . 21
3.1.3 Accelerated certified consumption path . . . 21
3.1.4 Features and capabilities to support a service catalog model . . . 22
3.2 Infrastructure franchise model . . . 23
3.2.1 Central control team . . . 24
3.2.2 Consumption workflow and upfront governance . . . 24
3.2.3 Controlled vending of specific components . . . 24
3.2.4 Features and capabilities to support an infrastructure franchise model . . . 25

4 Organizing resources **26**
4.1 Foundational organization concepts . . . 26
4.1.1 Access management . . . 27
4.1.2 Delegate access . . . 28
4.1.3 A word on naming conventions . . . 29
4.2 Organizations, projects, workspaces, and Stacks . . . 29
4.2.1 Organizations . . . 29
4.2.2 Projects . . . 33
4.2.3 Workspace . . . 35
4.2.4 Terraform Stacks . . . 37

5 Identity and access management 47
5.1 Post-installation tasks . . . 47
5.2 SSO and teams . . . 47
5.3 Connect to a version control system . . . 48
5.3.1 GitHub app . . . 48
5.3.2 OAuth . . . 49
5.4 Audit . . . 49
5.5 HCP Terraform agents . . . 50
5.5.1 Compatibility . . . 50
5.5.2 Operational considerations . . . 50
5.5.3 Upgrading HCP Terraform agents . . . 51

6 Audit log management 52
6.1 Audit . . . 52
6.2 HCP Terraform audit trail logs . . . 53
6.2.1 HCP Terraform audit trail logs . . . 53

7 Observability and alert management 53
7.1 Observability in HCP Terraform and Terraform Enterprise . . . 54
7.2 Monitoring focuses for Terraform Enterprise . . . 55
7.2.1 Traffic . . . 55
7.2.2 Demand capacity . . . 56
7.2.3 Failure rate . . . 56
7.2.4 Run resource usage . . . 57
7.2.5 Health checks . . . 57
7.3 Configuring data collection . . . 58
7.3.1 Terraform Enterprise metrics and logs . . . 58
7.3.2 HCP Terraform audit trail logs . . . 60
7.3.3 Metrics and logs on HCP Terraform agents . . . 61
7.4 Using observability data . . . 62
7.4.1 Data collection and aggregation . . . 62
7.4.2 Alerting and notification . . . 63

8 Backup and restore operations 64
8.1 Introduction . . . 64
8.2 Definitions . . . 65

8.3 Best practices . . . 66
8.3.1 Maintain the backup and restore process . . . 66
8.3.2 Manage sensitive values . . . 66
8.3.3 Process audit logs . . . 66
8.3.4 Terraform Enterprise backup API . . . 67
8.4 Initial considerations . . . 67
8.5 Application server . . . 67
8.6 Object store . . . 71
8.7 Database . . . 73
8.8 Redis cache . . . 75
8.9 Backup a `disk` deployment . . . 75
8.10 Restore Terraform Enterprise . . . 76
8.11 Definitions and best practices . . . 77
8.11.1 Data management . . . 78
8.11.2 Record event data . . . 80
8.11.3 Relationship between database and object store . . . 80
8.12 Preparation . . . 81
8.13 Application server . . . 81
8.13.1 Application server recovery and restoration . . . 81
8.14 Object store . . . 82
8.14.1 Object store content loss and corruption . . . 83
8.14.2 Recover lost object store content . . . 86
8.14.3 Recover corrupt object store content . . . 88
8.15 Database . . . 92
8.15.1 Database recovery . . . 92
8.15.2 Database restoration . . . 93
8.16 Redis cache . . . 98
8.17 Restore a `disk` mode Terraform Enterprise instance . . . 98
8.17.1 `disk` mode recovery . . . 98
8.17.2 Mounted disk mode restoration . . . 99

9 Disaster recovery operations 100
9.1 Detection of need . . . 101
9.2 Standard operating procedure (SOP) for failover . . . 101
9.2.1 Recommended failover steps . . . 101

9.3 Standard operating procedure (SOP) after failover . . . 102
9.4 Standard operating procedure (SOP) for failback . . . 102
9.5 Additional information . . . 103
9.5.1 Object store . . . 103
9.5.2 Database . . . 104

10 Support readiness **106**
10.1 Capturing command-line trace logs . . . 106
10.1.1 Enable trace logging . . . 106
10.1.2 Narrow log scope . . . 107
10.1.3 Clean up after troubleshooting . . . 107

11 Self-hosted agents **108**
11.1 Specific to Terraform Enterprise . . . 108
11.1.1 **Access resources in restricted networks** . . . 109
11.1.2 **Customizing the tools available and support custom logic** . . . 109
11.2 **Implementation guidance** . . . 109
11.2.1 **Building a custom agent image** . . . 110
11.2.2 **Deployment** . . . 110
11.2.3 **Agent authentication** . . . 115
11.2.4 Observability . . . 116
11.2.5 **Considerations for Terraform Enterprise** . . . 118
11.3 Request Forwarding (HCP Terraform Only - Premium SKU) . . . 118
11.4 Network requirements . . . 121

12 Private registry management **124**
12.1 Roles and responsibilities . . . 125
12.1.1 Registry Administrator . . . 125
12.1.2 Producer . . . 126
12.1.3 **Consumer** . . . 127
12.2 Sharing custom Terraform modules . . . 127
12.2.1 Publishing a module in the private registry . . . 129
12.3 Sharing custom providers . . . 132
12.3.1 Publishing a custom provider to your private registry . . . 133
12.4 Sharing whitelisted public Terraform modules and providers . . . 135
12.4.1 Publishing public providers or modules to the private registry . . . 135

12.4.2 Publishing public providers or modules to an air-gapped private registry . . 136
12.4.3 **Removing a public provider or module from the private registry** 139
12.5 Discovering Terraform Modules and Providers 140
12.6 Consuming Terraform Modules or Providers 141
12.7 Module development and maintenance 142
12.7.1 Module testing 142
12.7.2 Module version deprecation 148

13 Policy as code and Sentinel management 149
13.1 Benefits of using Sentinel 150
13.2 Explanation of the policy enforcement workflow 151
13.3 Key people and their responsibilities 152
13.4 Sentinel policy authoring best practices 153
13.4.1 Repository organization 153
13.5 Writing policies 155
13.5.1 Enforcement level 156
13.5.2 Testing policies 157
13.6 Policy Sets 159
13.6.1 Boundaries 159
13.6.2 Policy Set structure 160
13.7 Sentinel Policy Set management 160
13.8 Sentinel SDLC 161
13.8.1 General points 161
13.9 Summary 163

14 Run tasks management 163
14.1 Benefits of Run Tasks 165
14.2 Overall flow for implementing Run Tasks 165
14.3 Overall flow for Run Tasks within a run 166
14.4 Available Run Tasks 167
14.5 Run Task specific requirements 167
14.5.1 Terraform version 167
14.5.2 Permissions required for managing Run Tasks 168
14.6 Managing Run Tasks in an organization 168
14.6.1 Using HCP Terraform UI 169

14.6.2 Using Run Tasks API . . . 169
14.6.3 Using the TFE Provider . . . 170
14.7 Best practices for Run Tasks . . . 170
14.7.1 Security considerations when using Run Tasks . . . 171
14.7.2 Tips for effective use of Run Tasks in HCP Terraform . . . 171
14.7.3 Select Run Tasks to integrate based on need and benefit . . . 171
14.7.4 Manage the number and impact of Run Tasks . . . 171
14.7.5 Consider the availability of Run Tasks when selecting tools . . . 172

15 Custom Run Task Integrations 172
15.1 Building custom Run Task integrations . . . 172
15.1.1 Requirements for building a custom integration . . . 173
15.2 The Run Tasks integration API . . . 174
15.2.1 Run Task flow of events . . . 174
15.2.2 Best practices for building and testing Custom Run Task integrations . . . 175

16 No-code provisioning management 176
16.1 Implementation guidance . . . 176
16.1.1 Roles and responsibilities . . . 176
16.1.2 Permissions requirements . . . 177
16.1.3 Credentials for provisioning . . . 177
16.1.4 Configuring at scale . . . 177

17 Changelog 179
17.1 2026-08 . . . 179
17.2 2026-01 . . . 179
17.3 2025-11 . . . 179
17.4 2025-10 . . . 179
17.5 2025-09 . . . 179
17.6 2025-07 . . . 179
17.7 2025-05 . . . 179
17.8 2025-04 . . . 180
17.9 2025-03 . . . 180
17.10 2024-12 . . . 180
17.11 2024-04 . . . 180
17.12 2024-01 . . . 180

17.132023-11 180
17.142023-10 180

18 Credits **180**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

The Terraform Administration Guide is for platform teams operating HCP Terraform or Terraform Enterprise as a shared service. It assumes the Terraform platform is already available: HCP Terraform is provided by HashiCorp as SaaS, and Terraform Enterprise self-hosted deployments have completed the Installation Guide.

For practitioner workflows such as infrastructure provisioning, imports, private-registry consumption, policy enforcement during runs, run-task results, continuous validation, workspace notifications, no-code module consumption, and ephemeral workspaces, see the Terraform User Guide.

Note

Terraform Enterprise-only operations, including backup, restore, failover, and self-hosted infrastructure responsibilities, are marked explicitly. HCP Terraform is operated by HashiCorp; HCP-specific guidance is called out inline where relevant.

2 People and process

The API model for consuming infrastructure, championed by the release of AWS S3 in 2006, required a new way to orchestrate the management of infrastructure components, and IaC emerged as the most efficient approach. Terraform Enterprise stands out as a best-of-breed tool in this area, but it is not just a standalone tool: its effective use requires thoughtful allocation of staff resources and processes to achieve a notable return on investment.

Implementing changes to people and processes can be difficult due to cultural and political dynamics. Understanding the underlying concepts and reasons is crucial. Being practical is vital since modifying an organizational structure is gradual, and specific requirements infer adjustments over time.

This section discusses key organizational ideas that our customers have found valuable for deploying Terraform Enterprise effectively and embracing the Cloud Operating Model discussed earlier.

2.0.1 Organizing for industrialized cloud adoption

We see many of our customers adopting an industrialized IT operations framework composed of a number of functions fulfilled by one or more teams. The following diagram illustrates the key functions and how they relate to each other based on our current context of infrastructure provisioning.

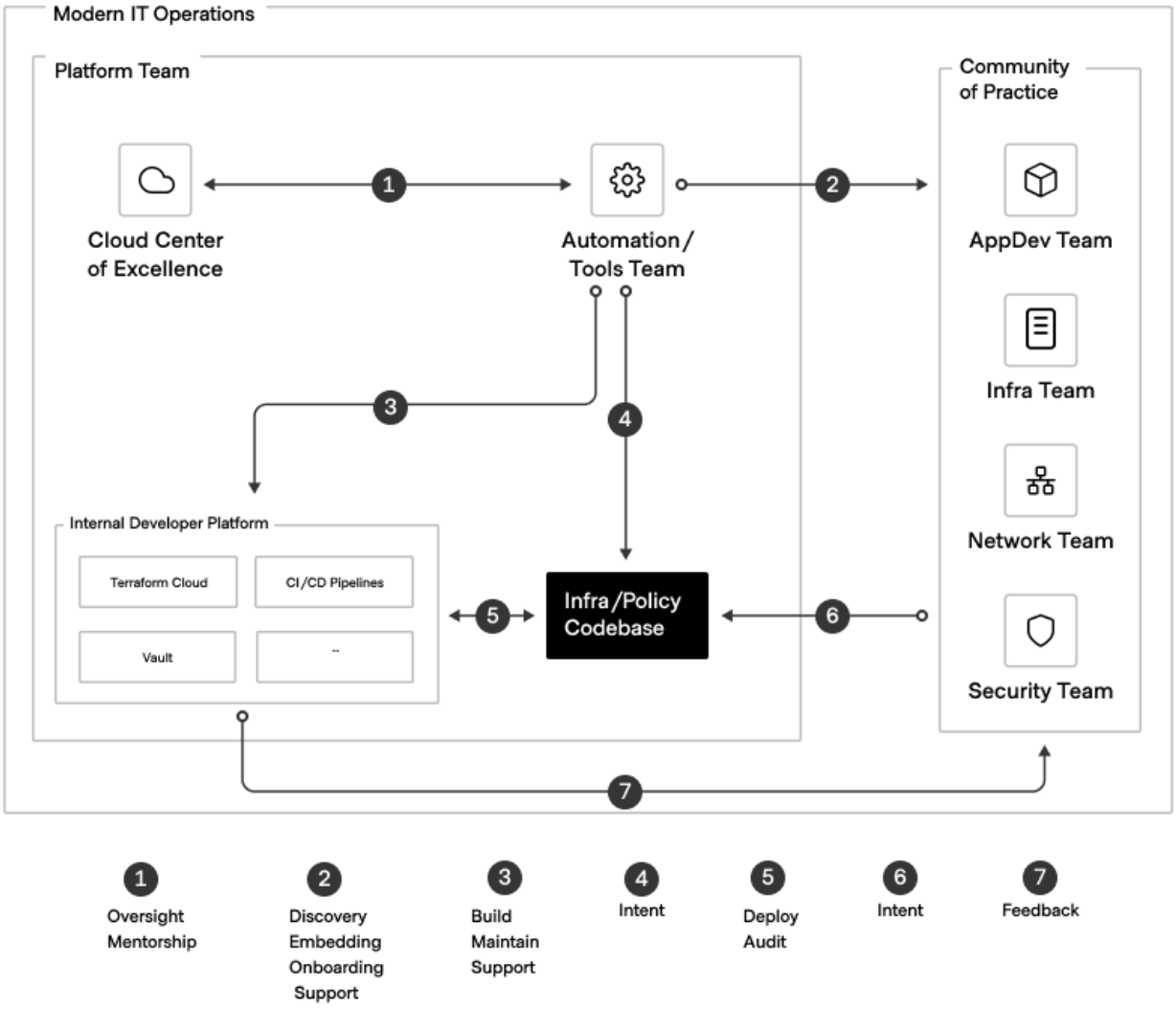


Figure 1: Overview of the platform team and how it relates to the rest of the organization

2.0.2 Platform team

The platform team serves as the central hub within the organization, orchestrating various functions and ensuring streamlined operations across teams. One or more teams with separate areas of responsibility may compose the platform team, whose functions encompass several key roles. These teams collaborate to ensure that application team developers have access to these shared

services. Areas of remit include the following.

- **Cloud Center of Excellence (CCoE):** This strategic component of the platform team drives cloud adoption and aligns cloud strategies with business objectives. The CCoE establishes governance frameworks, fosters knowledge sharing, and optimizes cloud resources to maximize value and ensure compliance.
- **Automation/Tools:** As a tactical component of the platform team, this function manages essential tools and automation to support efficient system operations. It implements golden workflows and reusable modules, collaborating with stakeholders to ensure that services meet consumer needs and align with strategic directives. This may or may not be a separate subteam.
- **IaC/PaC:** The platform team sets the intent for infrastructure-as-code and policy-as-code, collaborating with security to ensure adherence to standards and best practices. This promotes standardization, scalability, and compliance across projects.
- Internal Developer Platform (IDP): Facilitated by the platform team, this streamlines development cycles and reduces organizational complexity. It uses a product management approach, allowing development teams to consume golden paths in a self-service mode, thereby enhancing productivity and efficiency.

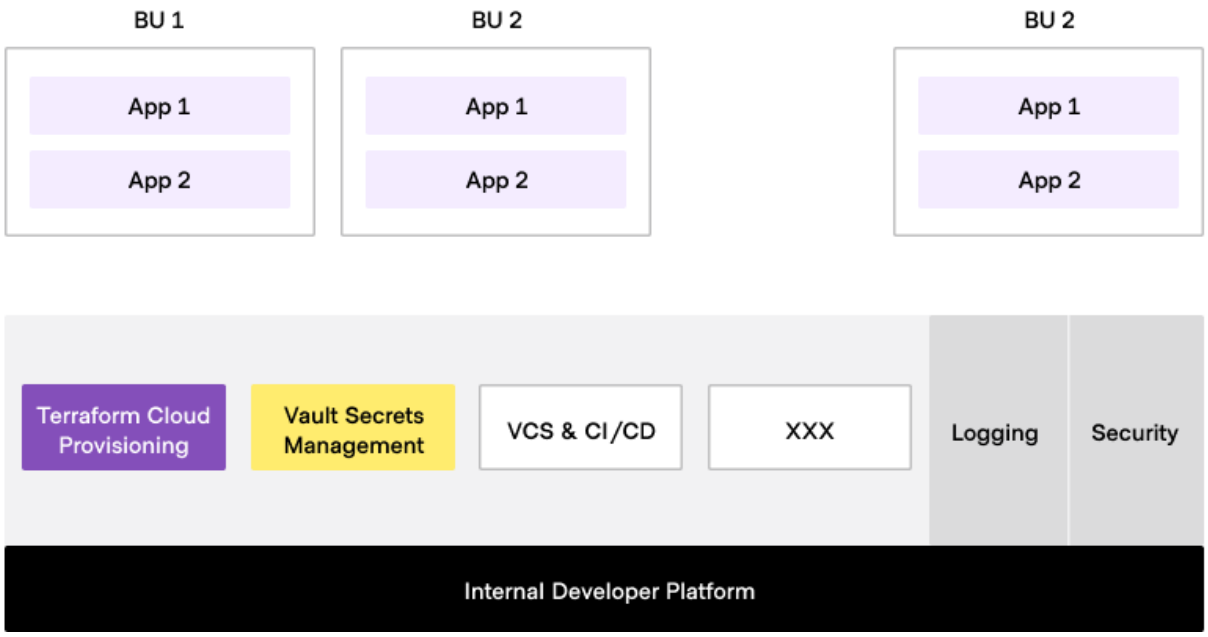


Figure 2: The internal developer platform

2.0.3 Security team

The security team collaborates with the platform team to establish governance policies and deploy monitoring tools for securing cloud-based infrastructure. They implement security controls around network segmentation, access management, and encryption to prevent unauthorized access and data breaches.

Ensuring compliance with regulations and internal policies is a key focus, with the team conducting audits and enforcing security policies. They integrate security controls into CI/CD pipelines, scanning code and images for vulnerabilities.

Prioritize identity and access management (IAM), with the team managing user identities and permissions to minimize the risk of unauthorized access. Data security measures include encryption, data classification, and data loss prevention to protect sensitive data.

In threat detection and incident response, the team sets up monitoring systems and intrusion detection tools to respond to security incidents, minimizing impact and ensuring resolution.

Cloud security is a function of the security team. This function ensures HCP Terraform security aligns with organizational policies by:

- Subscribing to security updates and vulnerability alerts and collaborating with SRE to apply patches as needed.
- Managing TLS certificate validity and renewal for Terraform Enterprise, ensuring trust with integrations.
- Scan Terraform Enterprise instances for vulnerabilities and integrating SIEM tools for audit logging.
- Providing governance inputs to CCoE, including defining and enforcing PaC guardrails and maintaining standards such as CIS benchmarks.

In collaboration with the platform team, they enforce security controls and conduct acceptance reviews to enhance platform security.

2.0.4 Producers/consumers

In a shared service model, producers configure the system to meet consumer needs and consumers utilize shared services provided by the platform team.

Producers' Responsibilities:

- Provide seamless onboarding to the HCP Terraform platform.
- Manage the private registry.
- Oversee Policy-as-Code (PaC) implementation.
- Offer necessary enablement to consumers.

Consumers' Responsibilities:

- Initiate requests for access to the HCP Terraform platform.
- Write Terraform code based on available private registry modules and recommended practices established by the platform team.
- This collaboration ensures that shared services are readily accessible and efficiently utilized, unlocking the benefits of the HCP Terraform platform while meeting specific business needs and adhering to set guidelines and standards.

2.0.5 Organizational big picture

The hierarchy of application teams and platform teams can vary depending on the organization. In some organizations, such as those in banking, telecommunications, and healthcare, application teams may fall under a different hierarchy than the platform team. In other organizations, such as those focused on technology products, the application teams and platform teams may be part of the same hierarchy.

Other organizational functions, including finance and security, provide governance inputs to the platform team and request reporting and feedback to measure effective governance.

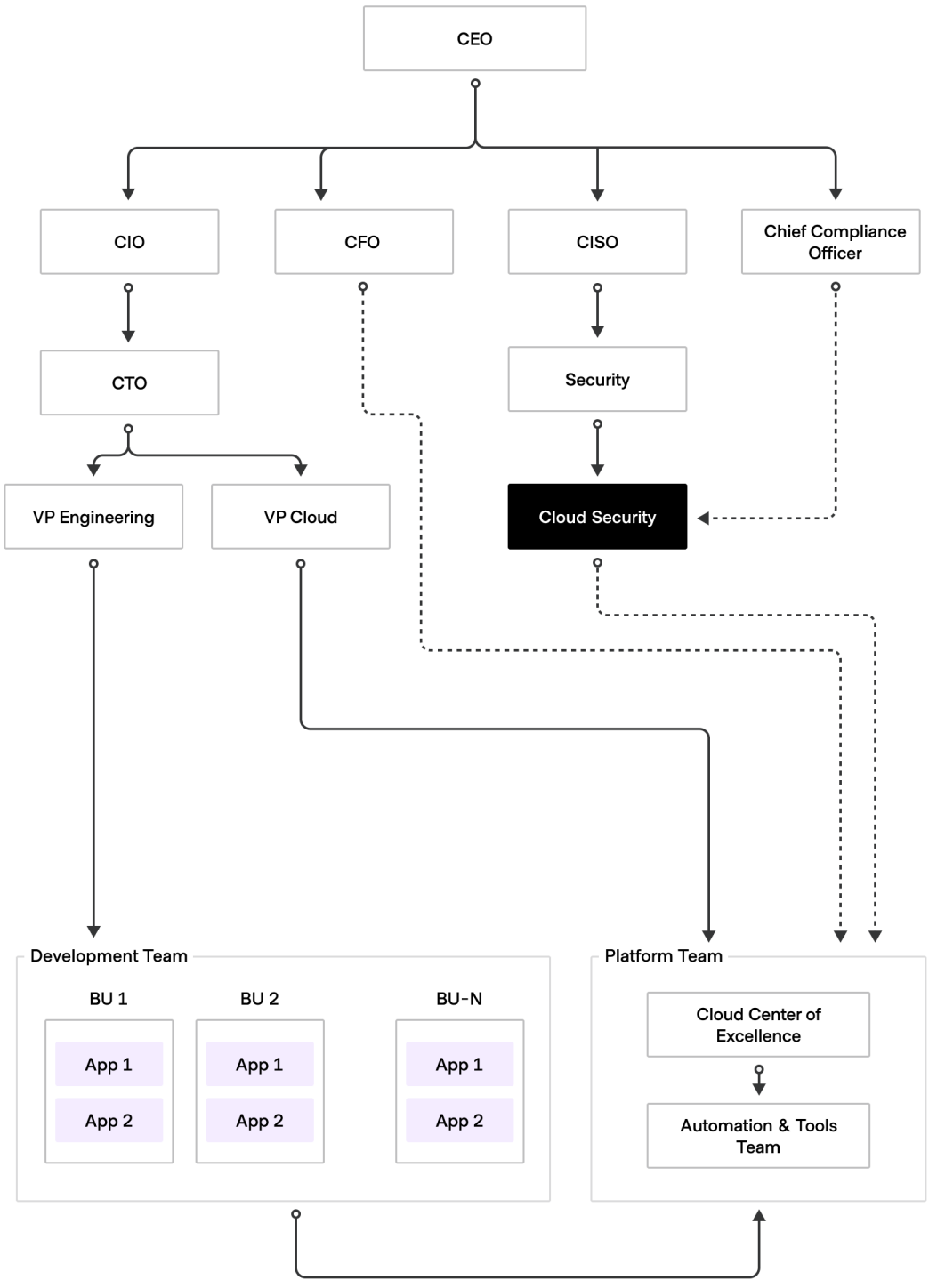


Figure 3: Organizational big picture

By centralizing expertise, fostering collaboration, and implementing best practices, the platform team accelerates time-to-market, enhances security, and drives innovation within the organization.

2.0.6 Golden IaC workflows

A golden workflow is a standardized, repeatable process for completing a specific task or achieving a specific outcome. It is typically well-documented, tested, efficient, and effective. Golden workflows are often used in DevOps and provide the following.

- Facilitate automation
- Improve efficiency, productivity, consistency, quality, communication and collaboration
- Reduce errors, onboarding time, and time-to-market for software components

Using pre-approved Terraform configurations and curated modules, the platform team shifts these golden workflows left, empowering teams to independently provision infrastructure resources while maintaining centralized management and control using best practices. The following diagram illustrates the golden workflow.

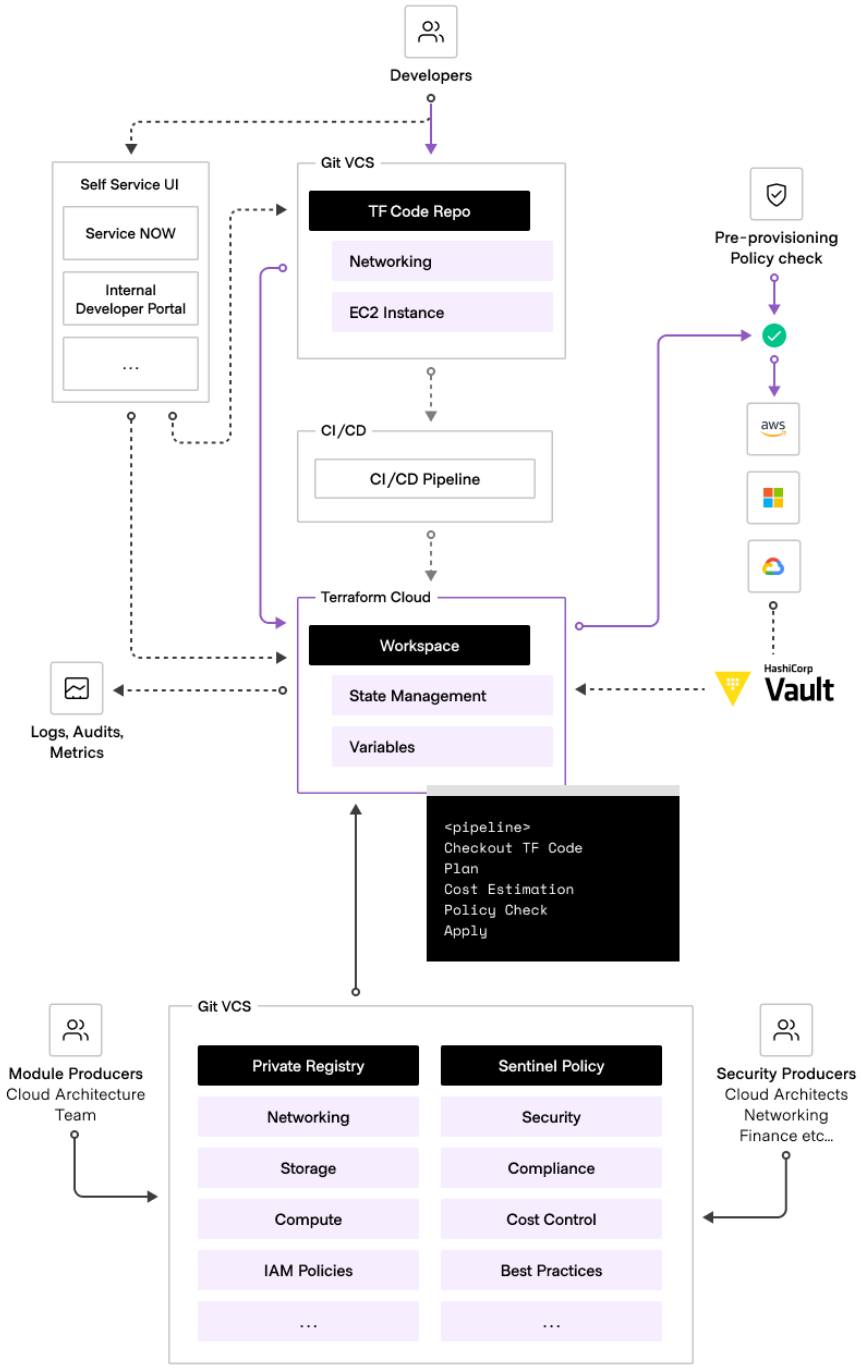


Figure 4: Infrastructure as Code golden workflow

Within these golden workflows, there are typically two aspects.

1. Producer workflows

- Module development - to create Terraform modules and register them with the private registry.
- PaC development - to develop Sentinel/OPA code and registering the policies they constitute in HCP Terraform.

1. Consumer workflows

- Landing zone workflow - to provision cloud accounts/credentials and core Terraform configuration elements from reusable modules. This also deploys relevant VCS repositories and HCP Terraform projects, workspaces and Stacks.
- Developer workflow - to facilitate the creation of IaC relevant to support the application components under remit.

3 Consumption models

The evolution of IT infrastructure systems has led to increased sophistication and productivity gains for organizations. However, it also resulted in functional silos and complex team-to-team handoffs, hindering agility. DevOps, application-focused teams, and cloud computing emerged to reduce handoffs and increase agility, offering sophisticated technologies without communication costs. Early adopters gained a competitive edge. Speed to market is now a crucial competitive advantage. Adopting practices like infrastructure as code (IaC) for provisioning and management is essential.

IaC uses code-based automation frameworks to define and manage data center components, proven effective in the DevOps model. Organizations typically offer two consumption models for Terraform Enterprise and IaC:

This evolution is leading to a situation where speed to market is more of a competitive advantage than ever. But for those organizations that made the jump or those that are considering it, it also raises the question of how to adopt those practices and technologies for infrastructure provisioning and management.

Infrastructure as code (IaC) is a way of managing and defining data center components with code-based automation frameworks. It is an effective pattern for provisioning and managing infrastruc-

ture in the DevOps model. When medium to large organizations begin using HCP Terraform and IaC, they typically offer two models of consumption to their IT customers:

1. Service catalog model: This model involves defining a core set of standard infrastructure patterns from which end customers can choose. It provides an accelerated or certified consumption path for infrastructure provisioning with limited customization options.
2. Infrastructure franchise model: Similar to real-life franchises, this model operates under a strict contract framework. With a robust policy model and supervision, end customers build and run their infrastructure independently, with significant customization options.

Comparing the two consumption models of Infrastructure as Code, it is clear that implementing a digital transformation project of this scale requires careful alignment of resources. Both models can coexist, but you can only begin with one as it requires a lot of resources to create the initial catalog or set up necessary guardrails.

The diagram below illustrates when each model is preferable based on the target user group's technical knowledge level, and the group's need for customization.

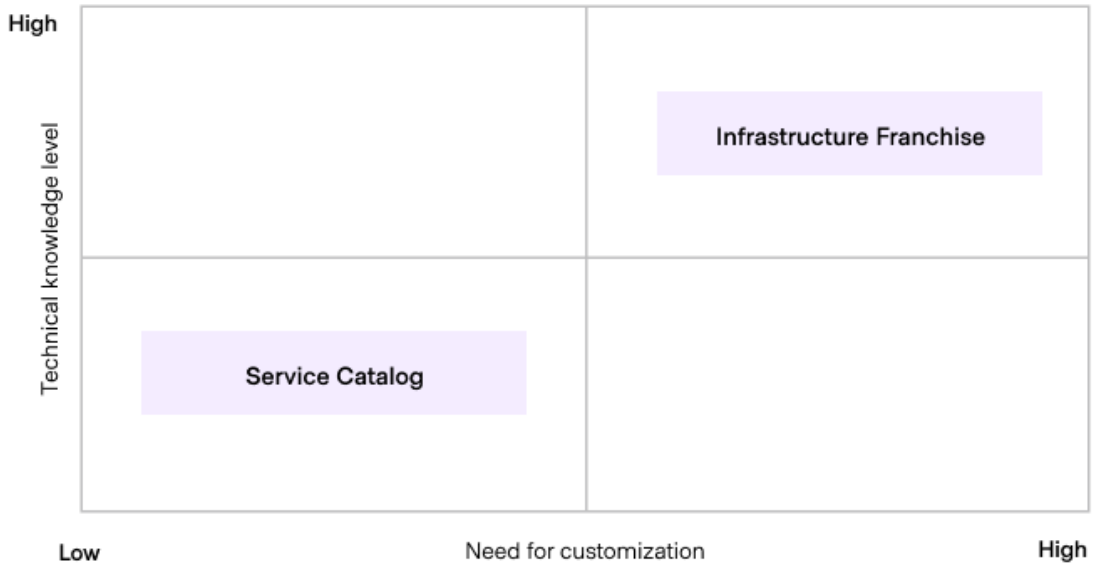


Figure 5: Choosing an IaC consumption model

	Service catalog	Infrastructure franchise
Primary UX	Vending portal (UI, ServiceNow and so on)	Custom workflow (Git, API/command line, CI/CD and so on)
Building	Standard modules	Use IaC within defined guardrails to build anything
Customization	Limited (using module parameters)	Unlimited
Primary security model	Validated modules	Guardrails (policy-as-code and so on)
Persona	Any - including business user/project manager	Development teams, SRE and so on

Figure 1: Highlighting the functions and features consumed in the operating models.

3.1 Service catalog

A service catalog functions as a centralized vending portal offering heavily prescribed and pre-configured infrastructure components. The primary objective of this model is to facilitate an accelerated consumption path for certified infrastructure elements. More details about the key aspects of this model are below.

3.1.1 Vending portal

The catalog is a central platform where users can view, request, and build standardized infrastructure components provided by a core group of power users. The Platform team maintains this portal and its standard components, offering them as a service to end customers. The aim is to give users a self-service infrastructure experience, allowing them to audit requests and enforce chargeback policies.

3.1.2 Pre-configured components

The availability of pre-configured components underpin the service catalog philosophy. These components can range from entire application templates to specific infrastructure aspects. The goal is to guarantee uniformity in infrastructure deployment. Organizations can use two methods to create and certify infrastructure:

1. **Platform team**: A designated central team creates and manages the organization's standardized patterns.
2. **Contribution Pipeline**: Establish an endorsement path and contribution pipeline to allow people outside the platform team to create endorsed patterns.

3.1.3 Accelerated certified consumption path

Successful organizations, like Meta, Netflix, and others, attribute their success to defining consistent infrastructure. When starting projects, business units have two options as follows.

1. Standard infrastructure patterns: Choose from predefined, validated infrastructure patterns to receive accelerated services and faster delivery.
2. Custom architectures: Opt to define their own architecture, leading to a slower delivery process as each component requires building and verification.

Naturally, this approach encourages business units to access existing trusted infrastructure building blocks for their projects:

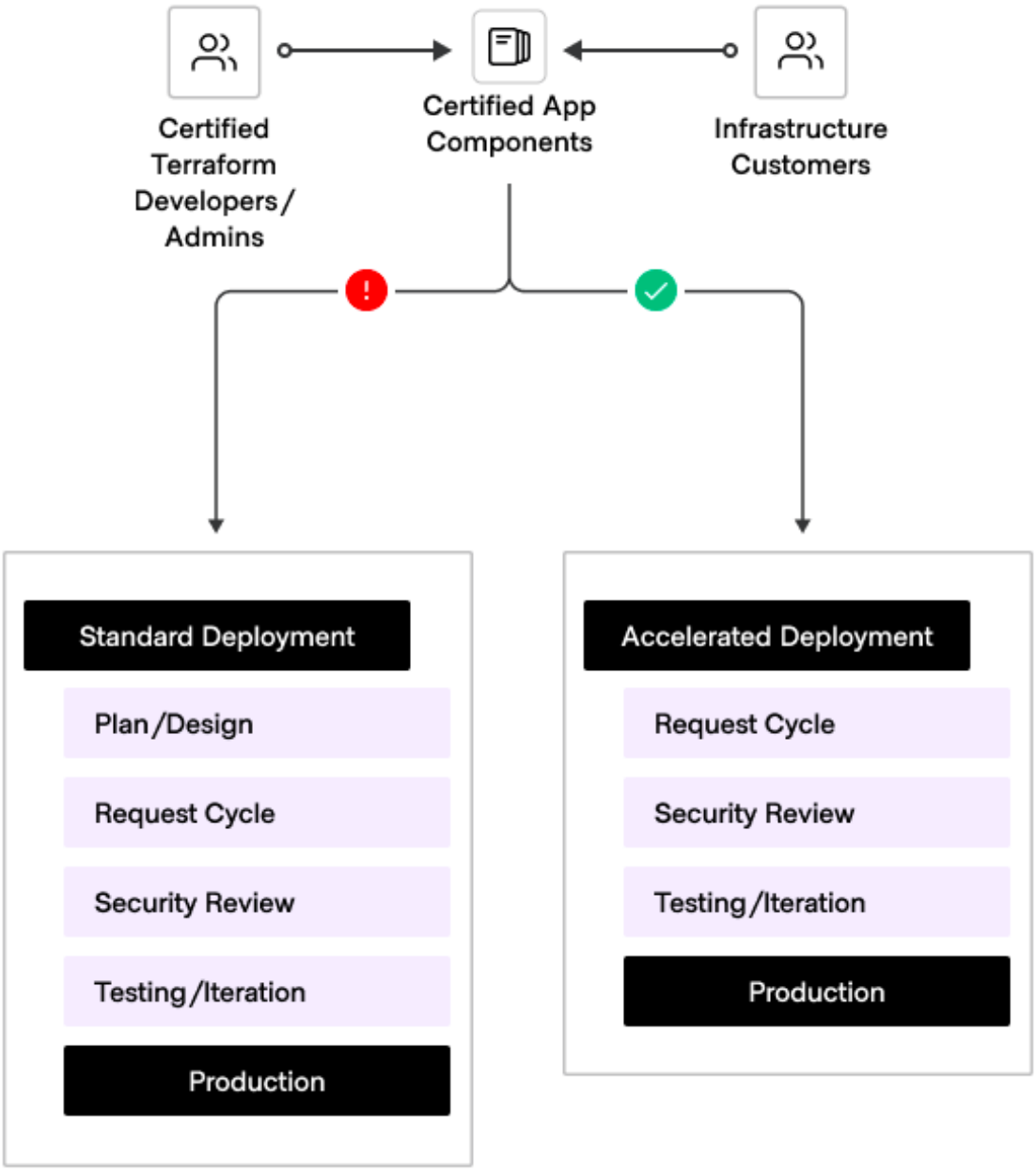


Figure 6: Service catalog workflow

3.1.4 Features and capabilities to support a service catalog model

Feature	Use case
Private registry	The private registry operates similarly to the public Terraform registry, enabling you to share Terraform providers and modules within your organization. It offers support for versioning and provides a searchable list of available providers and modules.
Private providers and modules	Your organization's private registry hosts providers and modules, restricting access to only members of your organization. In Terraform Enterprise, private modules are also accessible to other organizations configured to share modules with your organization. This allows for controlled access and sharing of custom-built modules across organizations within the Terraform Enterprise environment.
Run Tasks	Run Tasks in HCP Terraform enable direct integration with third-party tools and services during specific stages of the run lifecycle. When triggered, a run task sends an API payload to the external service, which includes essential run-related details and a callback URL for the service to respond back to HCP Terraform, indicating whether the task passed or failed. This feature allows seamless interaction with external tools, enhancing the flexibility and extensibility of your infrastructure automation workflows.
Third party certified integrations	There is a full list of certified integrations. You may also build and customize your own integrations based on your specific requirements.

3.2 Infrastructure franchise model

The franchise model is a way of distributing products or services. It involves a platform team that sets up the company's best practices, available services, and general instructions for using the infrastructure service. The main features of this model are:

1. The central control team
2. Consumption workflows and upfront governance
3. Controlled vending of specific components

3.2.1 Central control team

The franchise model has a platform team called the franchiser. This team provides franchisees with the workflow and resources they need. The franchiser is responsible for keeping the system running, setting rules, and adding extra capabilities to the workflow.

3.2.2 Consumption workflow and upfront governance

This model provides:

- **A consumption workflow**: Provide end users and customers with a way to access provisioning resources. This allows them to join the workflow and manage their own resource requirements.
- **Upfront governance**: Governance at the outset ensures that all customers and end users have the ability to provision resources while following legal, regulatory, and enterprise guidelines. Complying with regulations becomes easier and more cost-effective and markedly reduces time-to-market by obviating the need to acquire separate compliance sign-off ahead of go-live.

3.2.3 Controlled vending of specific components

The franchise model allows access to resources, but it also implements proactive controls to meet legal, regulatory, and enterprise standards. Common controls ensure the safety and security of the organization's existing resources. Also, policies control new resources as they go through testing, and validation. See below for an illustration of this model.

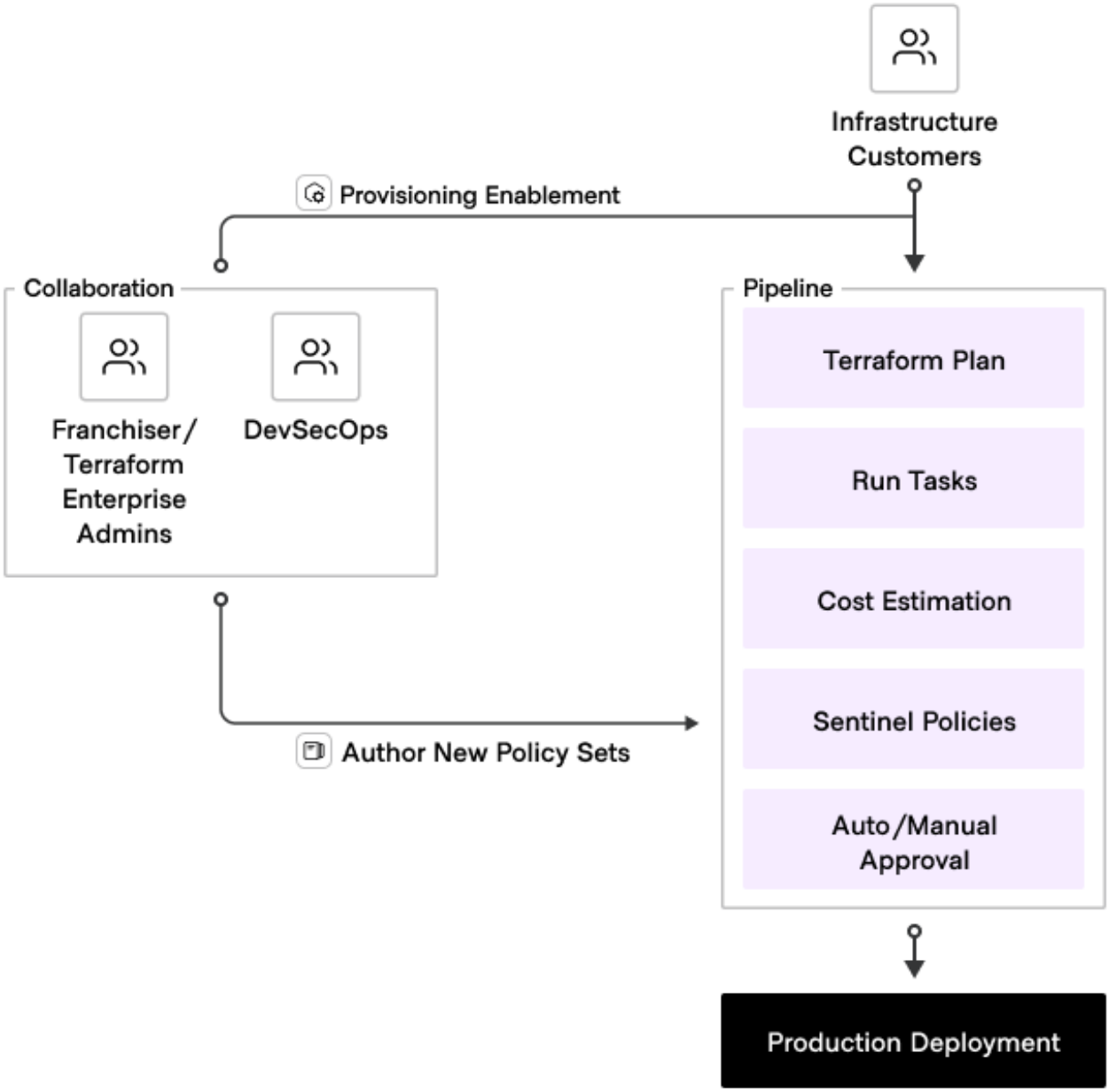


Figure 7: Franchise model workflow

3.2.4 Features and capabilities to support an infrastructure franchise model

An example of how to use each of these features to support services in a franchise model is below.

Feature	Use case
Terraform workspaces	When working with Terraform, you manage collections of infrastructure resources. Typically, organizations handle multiple collections. To facilitate this, Terraform employs a persistent working directory when run locally. This directory contains the configuration, state data, and variables for each collection of infrastructure. By organizing the configurations into separate directories, you can group infrastructure resources in a more meaningful and structured manner, allowing for better organization and management.
Terraform workspace projects	This feature enables the provisioning of self-managed portions of Terraform Enterprise. It allows end customers/users to use Terraform Enterprise while adhering to the same policies and controls as the root organization.
Sentinel policies	Sentinel is a policy-as-code framework embedded within HashiCorp Enterprise products. It empowers organizations to make fine-grained, logic-based policy decisions utilizing information from external sources.

4 Organizing resources

4.1 Foundational organization concepts

HCP Terraform helps you organize the infrastructure resources you want to manage with Terraform. It begins with the *organization* concept, which corresponds to your entire organization. You can have one or more projects under the organization. Each project then manages a collection of workspaces and Stacks as applicable. Workspaces and Stacks are containers managing Terraform configurations and their associated state files.

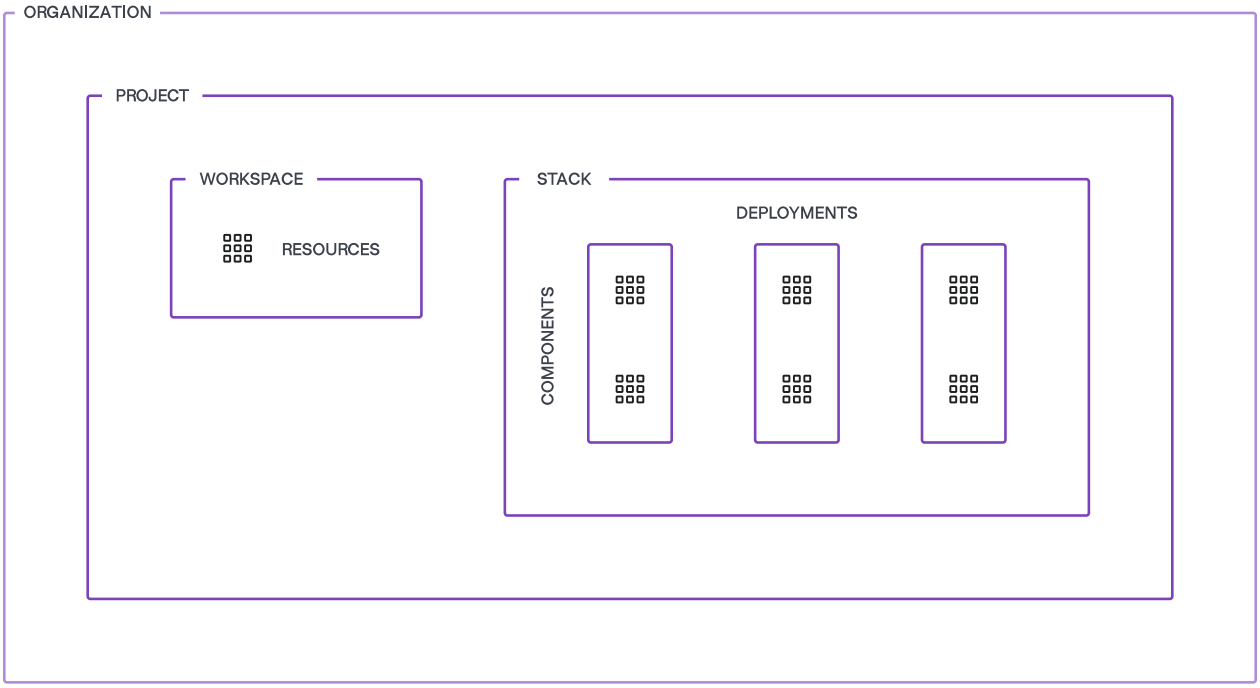


Figure 8: HCP Terraform structure

4.1.1 Access management

HCP Terraform bases access on three components:

1. User accounts
2. Teams
3. Permissions

HCP Terraform uses user accounts to authenticate, authorize, personalize, and track user activities. A team is a container used to group related users together and helps to assign access rights. A user can be a member of one or more teams. Permissions are the specific rights and privileges granted to groups of users to perform certain actions or access particular resources within HCP Terraform.

When managing your HCP Terraform environment, we recommend implementing SAML/SSO for user management, in conjunction with RBAC. SAML/SSO provides a centralized and secure approach to user authentication, while RBAC allows you to define and manage granular access con-

trols based on roles and permissions. By combining SAML/SSO and RBAC, you can streamline user onboarding and offboarding processes, enforce strong authentication measures, and ensure that users have the appropriate level of access to resources within your Terraform environment. This integrated approach enhances security, simplifies user management, and provides a seamless experience for you and your team members.

4.1.2 Delegate access

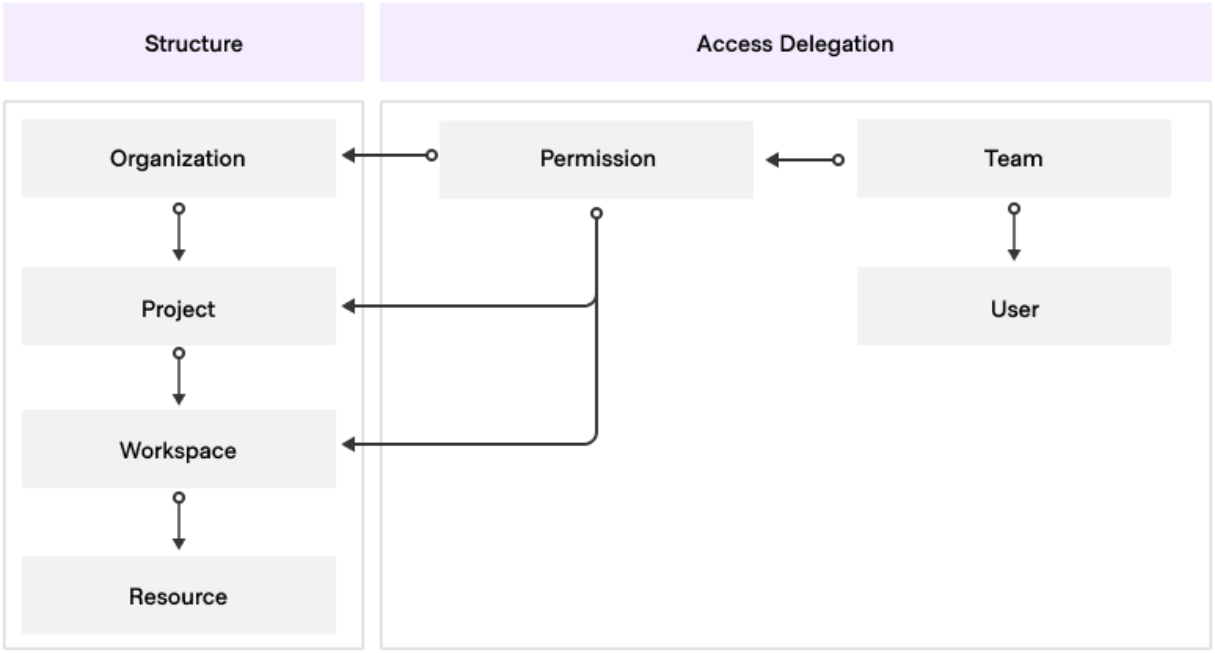


Figure 9: HCP Terraform access delegation

In HCP Terraform it is possible to assign team permissions at the following levels:

1. Organization
2. Project
3. Workspace/Stack

The set permission level defines the scope of this permission. For example, a permission assigned at one of the following.

- Workspace-level applies only to that workspace and not to any other workspace.
- Stack-level applies only to that Stack and not to any other Stack.
- Project-level applies to the project and the workspaces and Stacks it manages.
- Organization-level applies everywhere in that organization.

4.1.3 A word on naming conventions

You create and name a number of entities (teams, projects, workspaces, and so on) as you configure your HCP Terraform instance. We recommend that you create and maintain a naming convention document which includes the naming conventions for each entity, as well as any other conventions you decide to use such as prefixes or suffixes. This document helps ensure that your Terraform experience is well-organized as you scale out. Pass this document to all new internal clients to your HCP Terraform service to standardize their approach to operations.

4.2 Organizations, projects, workspaces, and Stacks

4.2.1 Organizations

The organization in Terraform Enterprise serves as the highest-level construct, encompassing various components and configurations. These include the following:

- Access management
  - Single sign-on
  - Users and teams
  - Organization API tokens
- Projects, workspaces, and Stacks
- SSH keys
- Private registry
- VCS providers
- Variable sets
- Policies
- Agent pools

We recommend centralizing core provisioning work within a single organization in HCP Terraform to ensure efficient provisioning and management.

This approach is compatible with a multi-tenant environment where you have to support multiple business units (BUs) within the same organization. Our robust role-based access control system combined with projects ensures that each BU has its own set of permissions and access levels. Projects enable you to divide workspaces, Stacks, and teams within the organization. We discuss them in more detail below.

Terraform Enterprise uses workload concurrency to maintain a run queue, and a worker RAM allocation. Consider the following points.

- The default concurrency is ten, meaning Terraform Enterprise spawns a maximum of ten worker containers and thus processes 10 workspace or Stack deployment runs simultaneously.
- The system allocates 2048MiB of RAM to each agent container by default. For most workloads, this is sufficient, but the platform team must be aware that for workloads requiring RAM residency larger than this, the container daemon harvests the container if it runs out of memory (OOM) and provides an undesirable error to the user. Proactive resource monitoring and reporting is thus key.
- For VM-based installations, the default RAM requirements for onboard agents is thus 20480MiB. If larger RAM requirements are necessary, consider vertically scaling the compute nodes, but we recommend that customers instead make use of HCP Terraform agents to offload the processing of large workloads to a dedicated agent pool. HCP Terraform agents also work for Terraform Enterprise. In this case, switch the affected workspaces or Stacks from a remote execution type to an agent execution type.
- For active/active configurations, system concurrency is per active node (HashiCorp support up to a maximum of five nodes in active/active setups).
- Similar consideration applies to CPU requirements but RAM considerations are a higher priority due to the need to contain the Terraform graph during operation.
- Note also that in the default configuration, if the instance is processing ten concurrent non-production runs, subsequent submission of production runs have to wait until concurrency becomes available which may delay these runs. This is a key consideration for the platform team to manage.
- We recommend liaising with your HashiCorp Solutions Engineer as necessary.

Naming convention

We recommend the following naming convention for organizations:

- `<customer-name>-prod`: This organization serves as the main organization which processes what the platform team considers as production workloads.
- `<customer-name>-test`: This organization is for integration testing and special proofs of concept conducted by the platform team.

Access management and permissions

This section extends public documentation on the subject of HCP Terraform RBAC.

The owners team

When establishing an organization, HCP Terraform creates an owners team automatically. You cannot remove this fundamental team.

As a member of the owners team, you have comprehensive access to all aspects of the organization, including policies, projects, workspaces, Stacks, VCS providers, teams, API tokens, authentication, SSH keys, and cost estimation. HCP Terraform reserves a number of team-related tasks for the owners team. You cannot delegate these tasks.

- Creating and deleting teams.
- Managing organization-level permissions granted to teams.
- Viewing the full list of teams (including teams marked as secret).

You can generate an organization-level API token to manage teams, team membership, workspaces and Stacks. It is important to note that this API token does *not* have permission to perform plans and applies. Before assigning a workspace or Stack to a team, you can use either a member of the organization owners group or the organization API token to carry out initial setup.

Organization-level permissions

Key organization-level RBAC permissions and their respective purposes are below.

- **Manage policies**: Allows the platform team to create, edit, read, list, and delete the organization's Sentinel policies. This permission also grants you access to read runs on all workspaces for policy enforcement purposes, which is necessary to set enforcement of policy sets.
- **Manage run tasks**: Enables the platform team to create, edit, and delete run tasks within the organization.

- **Manage policy overrides**: Permits the platform team to override soft-mandatory policy checks. This permission also includes access to read runs on all workspaces for policy override purposes.
- **Manage VCS settings**: Allows the platform team to manage the VCS providers and SSH keys available within the organization.
- **Manage private registry**: Grants the platform team the ability to publish and delete providers and modules in the organization's private registry. These permissions are only available to organization owners.
- **Manage membership**: Enables the platform team to invite users to the organization, remove users from the organization, and add or remove users from teams within the organization. This permission allows the platform team to view the list of users in the organization and their organization access. However, it does not provide the ability to create teams, modify team settings, or view secret teams.

As you review the preceding permissions, it becomes clear that many of them are not required for typical users. Therefore, we recommend granting these organization-wide permissions to a select group of your team members, reserving them for the most senior members on your team and restricting access for members of the development team. This approach helps prevent unintended modifications or disruptions to your organization's configuration.

It is crucial to treat the organization token as a secret and limit access to only privileged staff. Also, you have the flexibility to create additional teams within your organization and grant them specific organization-level permissions to suit the needs of the team and maintain proper access control.

Setting up organization-level access

Because of the broad elevated privileges that the owners team has, we recommend limiting membership to a small group of trusted users (that is, the HCP Terraform platform team). Create additional teams as needed to manage areas of the organization as necessary. For example, the security team may need to manage policies. We would generally recommend automating everything.

You may need to make adjustments to this model based on the breakdown of roles and responsibilities in your organization. If you do, update this matrix based on your context and make sure you update your HCP Terraform deployment documentation managed by the CCoE.

4.2.2 Projects

HCP Terraform projects are containers for one or more workspaces and Stacks. You can attach a number of configuration elements at the project level, which are then automatically shared with all workspaces and Stacks managed by this project:

1. Team permissions
2. Variable set(s)
3. Policy set(s)

Because of the inheritance mechanism, projects are the main organization tool to delegate the configuration and management responsibilities in a multi-tenant setup. The two standard project-level permissions used to achieve this delegation are:

1. Admin - provides full control over the project (including deleting the project).
2. Maintain - provides full control of everything in the project, except the project itself.

Variations on the proposed delegation model are possible by creating your own custom permission sets and assigning those to the appropriate teams in your organization. The benefits of this are:

- **Increased agility with workspace organization**:
  - Add related workspaces or Stacks to projects to simplify and organize a team's view of the organization.
  - Teams can create and manage infrastructure in their designated project without requesting administrator access at the organization level.
- **Reduced risk with centralized control**:
  - Project permissions allow teams to have administrative access to a subset of workspaces and Stacks.
  - This helps application teams manage their workspaces and Stacks without interfering with other teams' infrastructure and allow organization owners to maintain the principle of least privilege.
- **Best efficiency with self-service**:
  - Projects integrate no-code provisioning, so teams with project-level administrator permissions can provision no-code modules directly into their project without requiring organization-wide workspace management privileges. No-code module deployment is

into new Terraform Enterprise workspaces, which is a consideration for platform teams managing license consumption.

Project design

When deploying HCP Terraform at scale, a key consideration is the effective organization of your projects to ensure maximum efficiency. Projects are typically allocated to an application team. Variable sets are either assigned at the project level (for example for cost-codes, owners, and support contacts) and at the workspace level depending on requirements.

Automated onboarding is critical for the success of your project. Use either the HCP Terraform and Terraform Enterprise provider or the HCP Terraform API to automate the creation of the application team project and initial workspaces or Stacks and assign the necessary permissions to the teams (we recommend the former). This ensures that project allocation is consistent at scale. Consider this approach as a landing zone concept. Avoid any manual tasks required to onboard new teams as this results in platform team toil at scale.

We recommend assigning separate projects for each application within a business unit. The suggested naming convention for projects is: <`business-unit-name`>-<`application-name`>.

Access management and permissions

Create teams without requiring any of the aforementioned organization-level permissions. Instead, permissions can be explicitly granted at the project and workspace levels. We recommend setting permissions at the project level as it offers better scalability and allows teams to have the flexibility to create their own workspaces and Stacks when necessary. By granting permissions at the project level, you can effectively manage access control while providing teams with the necessary privileges. Below are the permissions you can grant and their respective functionalities:

- **Read**: Users with this access level can view any information on the workspace, including state versions, runs, configuration versions, and variables. However, they cannot perform any actions that modify the state of these elements.
- **Plan**: Users with this access level have all the capabilities of the `read` access level, and in addition, they can create runs.
- **Write**: Users with this access level have all the capabilities of the `plan` access level. They can also execute functions that modify the state of the workspace, approve runs, edit variables on the workspace, and lock/unlock the workspace.

- **Admin**: Users with this access level have all the capabilities of the `write` access level. Also, they can delete the workspace, add and remove teams from the workspace at any access level, and have read and write access to workspace settings such as VCS configuration.

Variations on the standard roles are possible by creating your own custom permission sets and assigning those to the appropriate teams in your project. We recommend using workspace-level permissions to demonstrate segregation of duty for regulatory compliance, such as to limit access to production workspaces to production services teams, and the converse for application development teams.

4.2.3 Workspace

A workspace is the most granular container concept in HCP Terraform. They serve as isolated environments where you can independently apply infrastructure code. Each workspace maintains its own state file, which tracks the resources provisioned within that specific environment.

This separation allows for distinct configurations, variables, and provider settings for each workspace. By organizing your workspaces under their respective projects, you establish a clear and hierarchical relationship among different infrastructure components.

Workspace design

When designing workspaces, it is important to consider the following:

- **Workspace isolation**: Isolate each workspace from other workspaces to prevent unintended changes or disruptions. This isolation ensures that changes made to one workspace do not affect other workspaces.
- **Workspace naming**: We recommend following a naming convention for workspaces to maintain consistency and clarity. The recommended format, with a maximum of 90 characters, is as follows: <`business-unit`>-<`app-name`>-<`layer`>-<`env`>. By incorporating the business unit, application name, layer, and environment into the workspace name, you can identify and associate workspaces with specific components of your infrastructure.

Naming convention

We recommend following a naming convention for workspaces to maintain consistency and clarity. The recommended format is: <`business-unit`>-<`app-name`>-<`layer`>-<`env`> where:

- <business-unit>: The business unit or team that owns the workspace.
- <app-name>: The name of the application or service that the workspace manages.
- <layer>: The layer of the infrastructure that the workspace manages (for example network, compute, data, app).
- <env>: The environment that the workspace manages (for example dev, test, prod).

By incorporating the business unit, application name, layer, and environment into the workspace name, you can identify and associate workspaces with specific components of your infrastructure. We recommend a maximum of ninety characters but this is not a hard limit.

Many application teams do not need the layer component in the name as the infrastructure they need for their application is clear. However, to standardize, consider use of the string main or app in place of the layer component as this helps maintain consistency across the organization. This field is also useful for delineation if the team splits into two as your business grows.

Multi-workspace patterns

Most Terraform deployments use more than one workspace to manage application infrastructure, with the baseline being one workspace per environment. Do not aim to manage an application's entire infrastructure in a single workspace. It is not recommended due to considerations such as prod/nonprod separation, blast radius, volatility, team responsibilities, resource scale, graph size, memory requirements and plan times. We explore these concepts in Best Practices - Workspaces - Terraform Enterprise.

Technical restrictions of the Terraform language can also force the use of multiple workspaces. For example, due to *known after apply* restrictions, you cannot plan some configurations in one operation to create a Kubernetes cluster and use it to configure a provider, or to create a Vault cluster then configure secrets engines.

Splitting the configuration over multiple workspaces solves these problems, but at the cost of increased complexity.

- Use orchestration to ensure workspaces run when required based on changes to upstream workspaces, and in the correct sequence. This can be in the form of workspace run triggers, Terraform orchestration code (terraform_workspace_run resource), or CI/CD pipelines.
- Using the attributes of one resource as an input for a dependent resource is more difficult if they are in different workspaces, as it requires data sources to pull the attributes, either from upstream workspace outputs or directly from the infrastructure itself.

- It is more difficult for users to keep track of resources when spread over multiple workspaces

Thus the overall function of a collection of workspaces may depend on multiple Terraform configurations, workspace variables and variable sets, workspace outputs and corresponding data sources, Terraform Enterprise workspace state sharing permissions, workspace run triggers, and CI/CD pipeline definitions.

4.2.4 Terraform Stacks

Instead of coordinating a collection of separate Terraform configurations and workspaces, Terraform Stacks introduce a new configuration layer that better represents the patterns of infrastructure at scale. You can define complex infrastructure as a set of interconnected components, repeat that infrastructure across multiple environments or regions, and manage it all from a single, centralized codebase.

With Stacks, you can access the following capabilities.

- Split infrastructure into smaller components: break up large configurations into reusable parts, with clear dependencies defined through standard Terraform inputs and outputs
- Reuse the same configuration across environments: define a Stack once and deploy it multiple times while managing each deployment independently.
- Codify relationships between components: avoid complex, scattered data sources and workspace wiring by declaring relationships directly in the Stack configuration.

Stacks building blocks

The core building blocks of a Stack are components and deployments.

Components represent the individual layers of your infrastructure, for example, networking, databases, or application services. Each component wraps a standard Terraform module and can declare dependencies on other components by consuming their outputs as inputs. This makes it straight forward to model complex systems with clear relationships between layers, without splitting them into separate workspaces.

Deployments allow you to repeat the component configuration across multiple environments, regions, or cloud accounts. For example, you might deploy the same configuration to dev, staging, and production, each as a separate deployment. Plan and approve each deployment independently, giving you control over the scope of each change.

Everything-as-code

One of the core principles of Stacks is to codify the entire behavior of infrastructure within version-controlled configuration files. This reduces reliance on external settings, avoids hidden behavior, and ensures deployments are predictable.

Specifically, Stacks follow a GitOps-style model for configuration management: Define non-secret inputs explicitly in the deployment configuration and stored in version control. There is no equivalent to workspace variables, and no hidden platform settings that affect how a Stack behaves. The goal is that the full intent and structure of a Stack is clear from the code alone.

Workload identity

Consistent with the everything-as-code principle, the preferred form of authentication in Stacks is workload identity. Define identity directly in the Stack configuration using `identity_token` blocks in the deployment configuration, and explicit provider configuration in each component.

This approach manages declarative provider authentication, avoiding the need for long-lived secrets or platform-managed environment variables.

Managing secrets

Stacks do not use workspace variables. Instead, use variable sets to manage secrets such as tokens, passwords, and credentials for providers that do not support workload identity.

Pull these values from variable sets as defined in the deployment configuration. Mark short-lived credentials provided using a variable set as ephemeral to ensure secrets are not persisted in state. Persist longer-lived secrets, such as license keys, in state using the stable keyword. Version all non-secret configuration in Stack configuration files.

Other differences from workspaces

Stacks introduces several capabilities that are not available in traditional workspace-based Terraform workflows. These features enable more flexible composition of resources and more dynamic configuration patterns, relevant to complex, multi-environment or multi-region setups.

Deferred changes

Stacks support deferred changes, a mechanism that allows Terraform to satisfy all resource dependencies using iterative plan and apply steps. This enables configurations that would otherwise fail due to `known_after_apply` attributes (such as creating a Kubernetes cluster and then deploying a Helm chart into it), to coexist within a single deployment. Terraform plans and applies what it can, then re-evaluates the configuration based on updated state, repeating this process until the deployment converges.

Explicit provider definition and iteration

In Stacks, define all providers and configured explicitly. HCP Terraform passes each provider as a named argument to the components that require it, which makes provider usage transparent and avoids assumptions based on default configurations or implicit inheritance from the root module.

This explicit model also enables provider iteration: a Stack can dynamically configure multiple provider instances using `for_each` to support scenarios like deploying infrastructure across multiple regions, accounts, or subscriptions. Each instance is explicitly defined, passed to the relevant components, and orchestrated within a single deployment, without duplicating component logic.

What are the best use cases for Stacks?

Stacks simplifies and scales Terraform workflows that would otherwise become complex or repetitive using traditional workspaces and modules. They are well suited to:

Complex, modular infrastructure

Infrastructure that is too large to manage as a single Terraform configuration or that you cannot plan in a single operation benefits from the component-based structure and partial plan behavior of Stacks.

Infrastructure repeated across contexts

Stacks allows repeatable and consistent configurations across multiple environments, regions, or provider instances, with the flexibility to manage and approve each instance independently.

Common patterns and use cases

Multi-environment applications

Deploy the same infrastructure pattern across dev, staging, and production with environment-specific parameters. Specify auto-approval of plans based on the environment and the risk of the proposed changes.

Multi-tenant infrastructure

Manage infrastructure for multiple customers or teams using the same Stack definition, with tenant-specific deployments.

Cross-account or cross-subscription infrastructure

Use dynamic provider iteration or separate deployments to manage resources across multiple cloud accounts or subscriptions from a single Stack.

Landing zones

Implement foundational infrastructure (for example networking, identity, policy) as a reusable Stack with consistent configuration across environments or business units.

Multi-region AWS deployments

Use multiple deployments or `for_each` over regions within the same deployment to manage infrastructure in multiple AWS regions within the same Stack.

> **Note**
>
> AWS provider v6.0 now supports multi-region configurations natively within a single provider instance, giving a third option for multi-region applications. A Stack can contain any combination of these techniques.

What are the best use cases for workspaces?

While Stacks offers more structure and flexibility for complex or large-scale deployments, workspaces remain a suitable choice for smaller projects or scenarios that benefit from strict separation between environments.

Small, self-contained projects

Workspaces are a good fit when managing the entire infrastructure using a single Terraform configuration, and planned/applied in one operation without orchestration.

Hard separation between environments

Each workspace maintains its own isolated state file, making workspaces well suited to scenarios requiring strict separation between environments, such as compliance boundaries, production safeguards, or minimal blast radius.

Branch-per-environment workflows

If your team uses a branching strategy where each environment maps to a separate Git branch (for example dev, staging, prod), workspaces align well with this model. Control promotion across environments using pull requests between branches, with HCP Terraform or CI/CD pipelines running plans in the appropriate workspace.

CI/CD-driven environment promotion

Workspaces are useful when CI/CD pipelines are mandatory to control promotion across environments or contexts. For example, when gating promotion with approvals or automated test outcomes, CI/CD systems can manage execution in different workspaces based on pipeline stage or branch.

> Note
>
> Terraform Stacks does not offer full feature parity with workspaces. Some platform features, most notably policy-as-code enforcement, are not yet supported for Stacks. This may influence the decision for teams with compliance or governance requirements. In such cases, continuing to use workspaces for policy-enforced infrastructure may be appropriate until full support is available. See the feature support matrix for more information.

Stack configuration structure

The diagram below illustrates the division of Stack configuration into two key sets of files: `*.tfcomponent.hcl` and `*.tfdeploy.hcl`. Together, they define the structure, behavior, and lifecycle of a Stack.

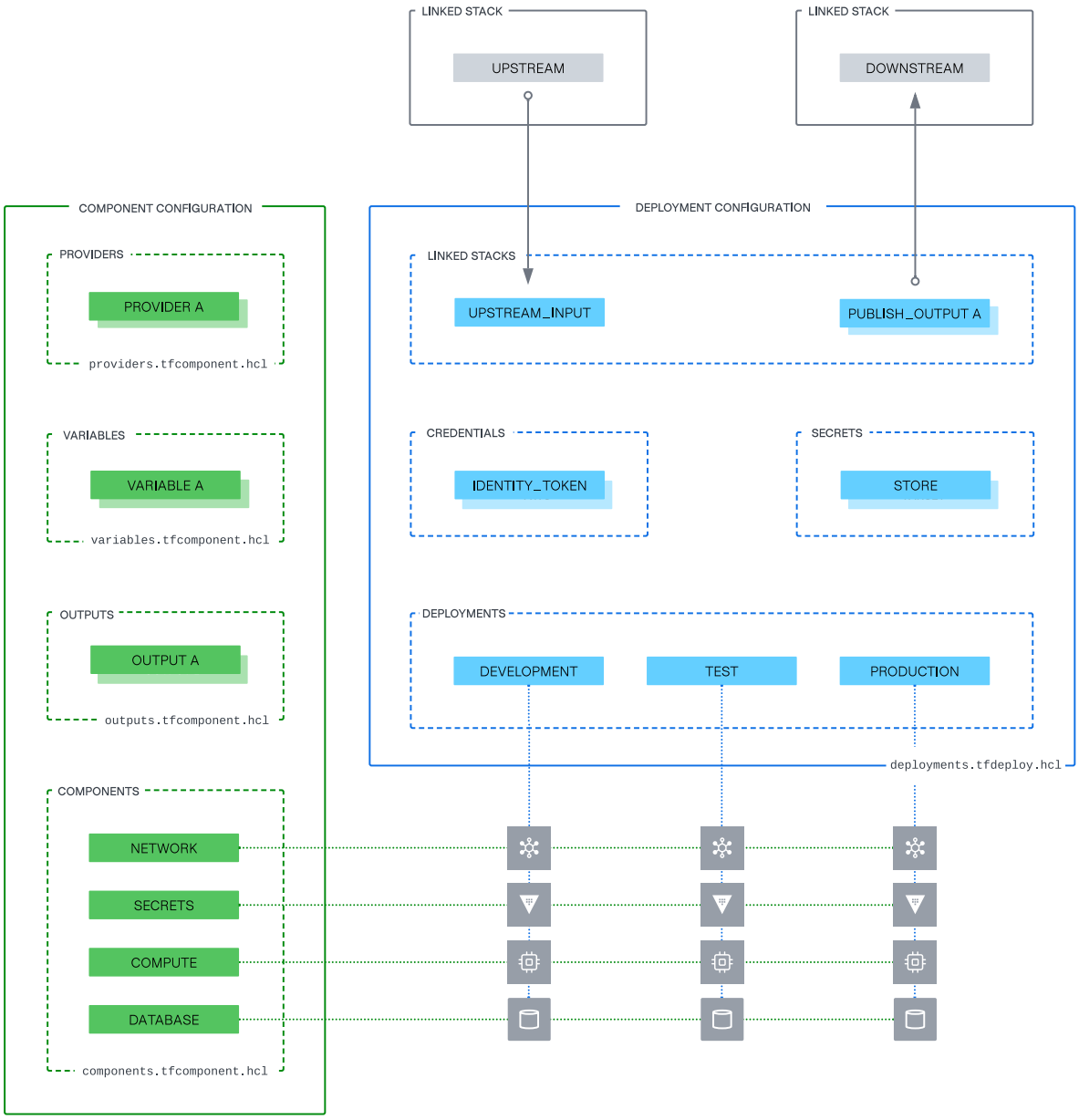


Figure 10: Stacks HVD diagrams

Component configuration (`*.tfcomponent.hcl`)

The component configuration defines the vertical structure of the Stack: the components, the inputs and outputs, and all provider configurations. These form the reusable building blocks of the Stack. All deployments reference this same component layout. They are replicas of the same structure, each instantiated in a specific environment or context.

The component configuration feels familiar to Terraform practitioners, as it follows the module composition pattern. in other words, a Terraform root module that calls other Terraform modules. Both call modules that can be in the registry, a git repository, or a subdirectory. Both define and configure providers, and both define input variables and outputs. The difference is that Stacks formalizes each module call as a separate component.

Unlike a Terraform root module, the Stacks component configuration cannot include raw `.tf` files. This means you must place any supporting logic typically placed at the root level of a workspace, such as data sources, in a dedicated supporting component. This encourages modularity and ensures that all logic, whether functional or structural, exists within a consistent pattern. Use of a submodule is a good solution for this.

Deployment configuration (`*.tfdeploy.hcl`)

The deployment configuration defines one or more deployments: apply individual instances of the component configuration to specific targets such as environments, regions, tenants, or subscriptions. Each deployment represents a complete independent unit of the Stack, using the same component layout but parameterized for a specific context.

Configure deployments with input values corresponding to the variables defined in the component configuration. In addition the deployment configuration defines several other aspects of how the Stack behaves:

- Authentication, typically using `identity_token` blocks.
- Secrets injection, using variable sets.
- Deployment groups and orchestration logic, defining the conditions for auto-approval of plans.
- Upstream and downstream connections, using `upstream_input` and `publish_output` blocks, enabling Stacks to exchange data and form larger compositions.

The diagram shows multiple deployments (for example dev, staging, production) sharing the same component definitions but managed independently.

Designing and using deployments effectively

Each deployment represents a separate instance of the Stack's component configuration and maps to a single state file. During execution of a Stack, each deployment runs in its own agent.

To structure the Stack effectively, it is important to understand when to use separate deployments, how provider iteration fits in, and how this affects concurrency.

When to use deployments

Use deployments when any of the following apply.

- The entire component structure must be repeatable across distinct contexts (for example dev, staging, prod, or multiple regions/accounts).
- There is a defined roll-out sequence for promoting changes across the contexts.
- You need independent approval policies, or credentials per instance.
- There are strong isolation requirements (for example tenancy).

For stronger isolation needs, consider breaking infrastructure into separate Stacks entirely. See Linked Stacks for guidance.

When to use `for_each` for provider configuration

Use `for_each` on provider blocks in the component configuration when any of the following apply.

- You want to repeat selected components across multiple regions or accounts.
- Multiple provider instances share a common lifecycle and identity/credential model.
- You want a single deployment to manage infrastructure across those scopes.

Avoid this pattern if any of the following apply.

- You need different authentication methods, approval flows, or orchestration for each instance.
- You require separate state, identity, or lifecycle controls.

Isolation and execution boundaries

Use deployments to achieve state, credential, and orchestration separation. If isolation requirements go beyond that (for instance ownership boundaries or audit separation), use multiple Stacks and define upstream/downstream relationships using linked Stack outputs and inputs.

Parallelism versus concurrency

Within a deployment, components run in parallel within a single agent.

As each deployment runs in its own agent, take agent concurrency into account in planning. In HCP Terraform, concurrency is subject to an organization-level limit on simultaneous agent runs.

Defining multiple providers

It is possible to declare multiple providers explicitly in the component configuration using `for_each`. This supports use cases like deploying to multiple regions or managing resources across accounts.

- Define version constraints once as HCP Terraform shares these across all instances.
- Provider configurations must be fully specified and keyed (for example by region or tenant).
- HCP Terraform passes each provider explicitly to the components that require it.

Example: A deployment passes a list of regions. The component configuration uses `for_each` to create an AWS provider instance per region.

Linked Stacks

Link Stacks together to form larger systems through explicit input/output relationships. A Stack can consume outputs from another Stack and publish its own outputs for downstream Stacks to use. Declare these relationships in the deployment configuration using `upstream_input` and `publish_output` blocks.

This enables teams to build infrastructure that spans multiple domains or layers with different scopes (for example platform and application). Linked Stacks provide clear boundaries while preserving composition.

When to split infrastructure across multiple Stacks

In general, a single Stack represents a coherent unit of infrastructure. Manage, deploy, and version this as one. A 12-factor approach would suggest keeping infrastructure for all environments in the same Stack, using multiple deployments to target different contexts.

However, splitting infrastructure across multiple Stacks is appropriate in some cases:

Different lifecycle or ownership

- If different teams manage components, for example network infrastructure managed by a platform team, compute infrastructure managed by application teams.
- If different schedules apply to deployment of components, for example a shared Kubernetes cluster updated quarterly, apps deployed every day.

Shared platform infrastructure

- If deployment of multiple services onto shared resources is relevant, for example a shared EKS or AKS cluster used by several apps.
- You want to isolate app-specific infrastructure from shared foundations.

Stronger environment separation

- Your organization requires branch-per-environment workflows with different controls for prod versus non-prod.
- You want to enforce VCS policies or approvals per environment tier.
- Examples:
  - One Stack for dev, and another for test + prod.
  - One Stack for dev + test, and a separate Stack for prod.

Use linked Stack outputs to pass values (for example network IDs, URLs) between Stacks while maintaining ownership boundaries and separation of concerns.

Stack structure: summary of recommendations

Use case	`for_each` components	Deployments	Linked Stacks
Manage everything as **one unit**	Yes	Yes	No
Repeat **selected components** across regions/accounts	Yes	No	No
Repeat the **entire Stack** across environments or contexts	No	Yes	No
Repeat across **regions**	Yes	Yes	Only if hard isolation needed
Repeat across **environments** (dev, staging, prod)	No	Yes	Maybe (for strict separation)
Use different **credentials or roles** per scope	No	Yes	Maybe (for org boundaries)

Use case	`for_each` components	Deployments	Linked Stacks
Require different **orchestration or approval flows**	No	Yes	Yes
Infrastructure managed by **different teams**	No	No	Yes
Differently scoped infrastructure (shared base infra and per-app infra)	No	No	Yes
Different **lifecycle or release schedule**	No	No	Yes
Require **separate VCS branch or workflow**	No	No	Yes

5 Identity and access management

This section shows you how to configure HCP Terraform or Terraform Enterprise to meet your requirements for delivering IaC services to your internal customers.

5.1 Post-installation tasks

An optional step at the end of automated installation of Terraform Enterprise is the creation of the Initial Admin Creation Token (IACT), but further initialization tasks complete the setup. There are two viable options for post-provision configuration in Terraform Enterprise: API scripts or using the Terraform Provider. The best practice is to use the Terraform Enterprise provider to derive state for the configuration.

5.2 SSO and teams

The setup of SSO for HCP Terraform is not covered in detail in this document, as configurations can vary depending on the SAML 2.0-compliant provider used.

- You can find specific instructions for popular identity providers here.

- For applicable SAML 2.0 documentation, here.

We recommend automating the creation of teams alongside projects, Stacks, and workspaces using the Terraform Enterprise provider. After creating the initial teams required, automate the addition of users within your strategic IdP platform. By adding a team attribute name (default: MemberOf) attribute within your IdP, you can automatically assign users to the created groups in the SAML configuration. You can find further details here.

Note: For user accounts used by pipelines, use `IsServiceAccount` set to true to flag these in SAML. See `isserviceaccount`. Create teams in Terraform Enterprise with the *exact* name of the group in the IdP. Do not create users manually - Terraform Enterprise creates these automatically after it processes the SAML assertion at login.

5.3 Connect to a version control system

We recommend connecting HCP Terraform/Terraform Enterprise to your VCS provider to enable workflows for managing modules in the private registry, managing Sentinel or OPA policy-sets, and connecting VCS-backed Stacks and workspaces to VCS repositories.

5.3.1 GitHub app

If using GitHub, the preferred method of VCS connection is a GitHub App, for a number of reasons. A GitHub App is not tied to a specific user, does not require a personal access token, and is safe to set up as a static VCS connection with no need to rotate tokens. A GitHub App connects Terraform to GitHub, but confers no access permissions itself. Instead, the following elements govern access to repositories.

- The GitHub organization(s) housing the GitHub App.
- Individual GitHub user permissions (to use the GitHub App, Terraform users must authorize it with their GitHub credentials).

See GitHub's documentation for more detail.

Terraform Enterprise supports a single site-wide GitHub App, shared across all Terraform Enterprise organizations. To use it, create and configure it using a site administrator account.

Choose the appropriate level of visibility for your GitHub App from the following list.

- If you only have one GitHub organization, configure your GitHub App as private.
- If you have multiple organizations under the same GitHub Enterprise, create the App under your enterprise account (at the time of writing, this feature is in public preview).
- If you need access to repositories across multiple GitHub organizations, configure your GitHub App as public but do not list it on GitHub's marketplace.

HCP Terraform has a pre-configured public GitHub app which you can install in any GitHub organization(s) as required.

5.3.2 OAuth

For providers other than GitHub, authorize VCS connections using OAuth within each Terraform Enterprise organization. Do this using a manual UI workflow using an ID and secret from the VCS, or automated using the HCP Terraform and Terraform Enterprise provider using an access token from the respective VCS account. As with all static tokens, it is good practice to rotate this periodically.

OAuth authorizes Terraform Enterprise to act as a specific user or account on the VCS. The permissions of that account govern the access Terraform has to VCS organizations and repositories. For simplicity, we recommend setting up a single OAuth connection per VCS provider in each Terraform Enterprise organization. Use a service account with access to all IaC and policy-as-code repositories, to avoid any dependency on individual employee accounts.

If you require a more granular or least-privilege permissions model, you can create multiple OAuth VCS connections, each mapping a subset of VCS repositories to a subset of Terraform projects. In this case it would be a function of a landing-zone module to create these OAuth connections and project mappings on demand whenever a new project onboards.

5.4 Audit

In HCP Terraform only, the Audit Trails API provides access to audit events. We recommend Terraform Enterprise customers forward logs for monitoring and auditing purposes. See the section on observability for more details. Include audit, logging, and monitoring in the target architecture for the delivery of your project. **Do not wait until after go-live to implement observability.**

5.5 HCP Terraform agents

Pools of HCP Terraform agents allow HCP Terraform and Terraform Enterprise to communicate with isolated APIs in discrete network segments such as those on premise. We recommend defining agent pools and assigning Stacks and workspaces to them using the Terraform Enterprise provider as appropriate. This improves the availability of your on-premise APIs and allows for configuration of network hardware, internal VCS, CI/CD systems and so on, using IaC. This improves the return on investment of HCP Terraform and Terraform Enterprise.

You may choose to run multiple agents within your network, up to the organization's purchased agent limit (which only affects HCP Terraform customers; there is no such limit for Terraform Enterprise customers). Each agent process runs a single Terraform run at a time. Multiple agent processes can be concurrently run on a single instance, license limit permitting.

We strongly recommend reading the HCP Terraform agent documentation as part of your implementation planning.

5.5.1 Compatibility

Agents support Terraform versions 0.12 and later only. Workspaces configured to use Terraform versions below 0.12 cannot select the agent-based execution mode.

When running Terraform Enterprise, consult the release notes for agent version restrictions for your target Terraform Enterprise version and also the HashiCorp considerations for HCP Terraform agents on Terraform Enterprise.

5.5.2 Operational considerations

The agent runs in the foreground as a long-running process that continuously polls for workloads from HCP Terraform. An agent process may stop due to a variety of reasons. We strongly recommend pairing the agent with a process supervisor to ensure that it automatically restarts in case of an error. On a Linux host, do this using a `systemd` manifest such as the one below.

```
[Unit]
Description="HashiCorp HCP Terraform Agent"
Documentation="https://www.terraform.io/cloud-docs/agents"
Requires=network-online.target
After=network-online.target

[Service]
Type=Simple
User=tfcagent
Group=tfcagent
ExecStart=/usr/local/bin/tfc-agent
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
EnvironmentFile=-/etc/sysconfig/tfc-agent

[Install]
WantedBy=multi-user.target
```

If using HCP Terraform agents in containers, use single-execution mode to ensure your agent only runs a single workload and guarantees a clean working environment for every run.

5.5.3 Upgrading HCP Terraform agents

When HashiCorp releases a new version of Terraform Enterprise, we test it against the most current HCP Terraform agent version. Should you need to upgrade your HCP Terraform agent version, we recommend testing the new version in a separate agent pool before rolling it out to your main agent pool(s). Use the platform team's dedicated HCP Terraform organization or Terraform Enterprise instance for this.

When considering use of HCP Terraform agents, do not skip testing, and we suggest that the extent of the testing effort must match the extent of the change. Patches and minor changes require little testing, while major changes require more extensive tests.

The diagram below illustrates our recommended roll-out approach for a new HCP Terraform agent version:

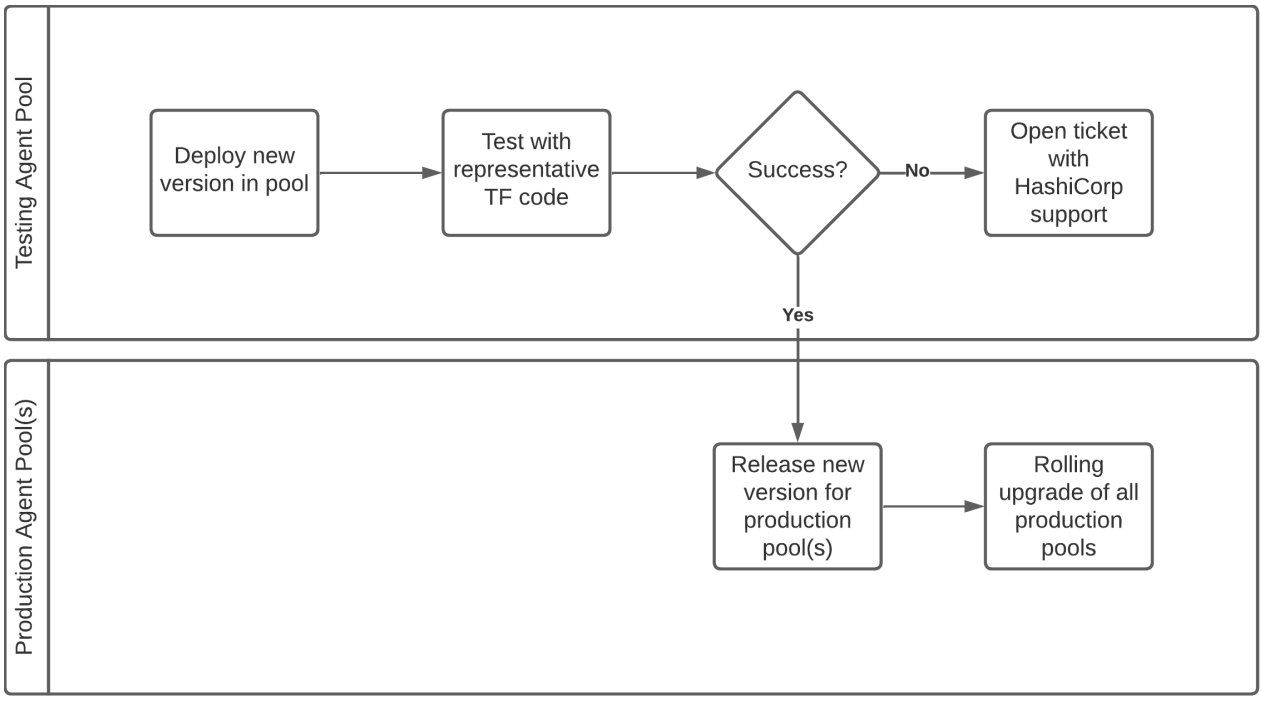


Figure 11: Rolling out a new version

6 Audit log management

This page covers recurring audit-log management. Initial log and event sink setup should be completed during installation or service onboarding.

6.1 Audit

In HCP Terraform only, the Audit Trails API provides access to audit events. We recommend Terraform Enterprise customers forward logs for monitoring and auditing purposes. See the section on observability for more details. Include audit, logging, and monitoring in the target architecture for the delivery of your project. **Do not wait until after go-live to implement observability**.

6.2 HCP Terraform audit trail logs

6.2.1 HCP Terraform audit trail logs

HCP Terraform features an Audit Log API endpoint that you must use to collect the audit events and store them in the appropriate system. To implement this solution, you need the following.

- A method to schedule and automate the audit events collection,
- A secure storage solution to store the audit events, and
- A data lifecycle solution to correctly dispose of the audit events once they are no longer required.

> Warning
>
> HCP Terraform keeps audit trail records for 14 days and then discards it. Design and configure your audit trail collection solution for HCP Terraform with this in mind.

If you use a Security Information and Event Management system (SIEM), this must be the destination for those audit events. Suppose you are not using a SIEM but instead are using a centralized log management solution (Datadog, New Relic, Elastic, and so on). In that case, you must send those audit events to your centralized log management system. If neither of these solutions is available, still collect those audit events and store them securely using an object storage solution, such as AWS S3.

7 Observability and alert management

Observability is about being able to look at the live outputs of a system running in production and answer questions about what is going on. It is often associated to monitoring, but both concepts are distinct:

- **Observability** is about setting up a system so that we have access to real-time data about what the system is doing
- **Monitoring** is about capturing this data and analyzing it to make decisions. Example of monitoring goals are: Detecting failures or performance degradation, capacity planning, intruder detection.

7.1 Observability in HCP Terraform and Terraform Enterprise

Observability features play a critical role in HCP Terraform and Terraform Enterprise for maintaining a secure and compliant infrastructure-as-code workflow. These features provide detailed visibility into user actions, changes to infrastructure, and system events, creating a comprehensive record of all activities within the platform.

By capturing this information, organizations can effectively monitor and analyze activities, identify potential security threats, track configuration changes, and troubleshoot issues as soon as possible. Audit and logging help meet regulatory requirements, bolster accountability, and support incident response efforts.

HCP Terraform and Terraform Enterprise offer several features to support observability:

Observability feature	HCP Terraform	Terraform Enterprise
Operational logs	No	Yes
Audit trail	Yes	Yes
Metrics	Yes	Yes
HCP Terraform agent logs	Yes	Yes

Operational logs track the performance and behavior of the system. They provide information about the system's functioning, such as error messages, warnings, and other events that can help troubleshoot issues and identify performance bottlenecks. SRE teams typically use operational logs to monitor and maintain the service.

On the other hand, *audit trail* logs focus on tracking security-related events and activities within a system. They capture information about login attempts, access control changes, suspicious activities, and other security events. Audit trail logs are crucial for detecting and investigating security incidents, as they record who did what in a system. Security analysts and incident response teams often use these logs to identify and respond to potential threats.

Metrics measure application component performance and usage and are the raw data used to detect service quality issues or inform a capacity planning exercise.

HCP Terraform and Terraform Enterprise differ regarding operational logs, which stems from the shared responsibility model associated with HCP Terraform. With HCP Terraform, HashiCorp's SRE

team tracks the operational logs as part of the service's operation. With Terraform Enterprise, your SRE team in charge of running the internal infrastructure-as-code service tracks those operation logs.

HCP Terraform and Terraform Enterprise also emit different metics. For information on the metrics you collect from HCP Terraform, refer to Collect organization metrics. For information on Terraform Enterprise metrics, refer to Enable metrics collection.

If you are using HCP Terraform agents, including those logs in the list of logs you collect and analyze is important. This helps ensure that you have a complete picture of all activities and can identify any issues or errors that may arise. By analyzing your logs together, you are better equipped to make informed decisions about optimizing and improving your HCP Terraform deployment. Remember to always stay vigilant and monitor your logs to ensure your systems run securely.

7.2 Monitoring focuses for Terraform Enterprise

You can configure Prometheus to gather metrics from Terraform Enterprise and its underlying components. Terraform Enterprise generates operational metrics that Prometheus can collect. This setup allows you to monitor system metrics, including CPU, memory usage, and request latency. For more information on constructing Prometheus queries, refer to this documentation.

Also, you can use Grafana to visualize the metrics collected by Prometheus. You can create dashboards that provide real-time insights into the health of your Terraform Enterprise infrastructure, such as resource utilization trends and request rates. You can access the Terraform Enterprise Grafana dashboard here.

By utilizing these tools to monitor your instances, it allows your business to account for the health, availability, and scalability of your system. Recommended monitoring guidelines for your Terraform Enterprise instances are as follows.

7.2.1 Traffic

To effectively schedule downtime for maintenance and determine peak hours of your Terraform Enterprise instance, measure the traffic flow by analyzing throughput of completed jobs. Do this by taking the rate of all the jobs entering a terminal status. An example of the query is as follows.

```
rate(sum by(status)(tfe_run_current_count{status=~"applied|planned_and_finished|
    errored|discarded|canceled"})[30s:]) * 60
```

7.2.2 Demand capacity

As your business continues to grow, so do your operational capacity needs. When there are an increased number of runs that exceed system capacity configurations, it causes the starting jobs to hang in a `pending` state. To determine if Terraform Enterprise has been consistently exceeding configured capacity limits, the following query can take a look at the system as a whole and allow engineers to identify spikes in usage.

```
sum (tfe_run_current_count{status="pending"})
```

You may also want to narrow down this search by aggregating by workspace or organization. Do this with the following example.

```
sum by (organization_name) (tfe_run_current_count{status="pending"})
```

By monitoring the demand capacity of your system, it allows your practitioners to determine the best time to scale capacity resources. Also note that it is important to set alerts in a for metrics exceeding thresholds in any capacity, such as CPU or memory usage passing through a set percentage threshold determined by your business to signal potential overload. Alerts for the performance of PostgreSQL, Redis, and Vault help identify issues affecting Terraform Enterprise stability or performance.

7.2.3 Failure rate

An increase in run failures can suggest a widespread issue within your Terraform Enterprise. In order to effectively troubleshoot, practitioners must monitor run counts for any particular run status properly. A `status` labels the `tfe_run_current_count` metric, allowing you to investigate run counts filtered by the associated status. An example of querying a status labeled `errored` is as follows.

```
sum (tfe_run_current_count{status="errored"})
```

Should you wish to view the trends of failures, consider examining the rate with the following query.

```
rate(sum (tfe_run_current_count{status="errored"})[1m:])
```

Other container and global metrics are in our public documentation.

7.2.4 Run resource usage

Proactively scaling your instances based on CPU or memory usage can help avoid performance degradation during peak loads. Ensure a consistent and reliable user experience ultimately resulting in proactive scaling. Use the following query to determine which organization is consuming which percentage of the instance's CPU.

```
sum by (organization_name)((rate(tfe_container_cpu_usage_kernel_ns{run_type!=""}[1m])
     + rate(tfe_container_cpu_usage_user_ns{run_type!=""}[1m])) / 1e7)
```

Use the same query for memory usage, excluding `rate` which is out of scope for RAM usage.

```
sum by(organization_name)(tfe_container_memory_used_bytes{run_type!=""})
```

You can further aggregate per workspace and filter down by organization:

```
sum by(workspace_name)(tfe_container_memory_used_bytes{run_type!="",organization_name
    ="my-org"})
```

7.2.5 Health checks

Health checks are crucial for scaling Terraform Enterprise because they ensure the system's reliability and performance by proactively detecting and addressing issues before they impact availability or efficiency during scaling operations. Learn more on how to monitor the health of application through external health checks in our public documentation.

In situations where external health checks might add undesirable network load, such as in high-traffic environments, it is important to consider the impact. Frequent health checks can exacerbate network congestion.

This is how you can access health check external endpoint using curl:

```
curl http://$(docker inspect ptfe_health_check|jg -r .[].NetworkSettings.Networks[].
    IPAddress):23005/_health_check
```

This returns the following.

```
{"passed": true, "checks": [
   {"name": "Archivist Health Check", "passed": true},
   {"name": "Terraform Enterprise Health Check", "passed" : true},
   {"name" : "Terraform Enterprise Vault Health Check", "passed" : true},
   {"name": "Fluent Bit Health Check", "passed": false, "skipped" : true},
   {"name": "RabbitMQ Health Check", "passed": true},
   {"name": "Vault Server Health Check", "passed": true}
]}
```

7.3 Configuring data collection

In this section we'll cover how to set the data collection for:

1. Collecting metrics and logs (including audit trail logs) on Terraform Enterprise
2. Audit trail logs on HCP Terraform
3. Collecting metrics and logs on HCP Terraform agents (applicable for HCP Terraform and Terraform Enterprise)

7.3.1 Terraform Enterprise metrics and logs

Configuring metrics collection on Terraform Enterprise

Explicitly enable Terraform Enterprise metrics collection as it is not enabled by default. Do this by setting the `TFE_METRICS_ENABLE` parameter to **`true`**. Next, configure your monitoring tool of choice to periodically query the Terraform Enterprise metrics endpoint to collect and store this information.

Configuration parameter	Description
`TFE_METRICS_HTTP_PORT`	The HTTP port which exposes metrics.
`TFE_METRICS_HTTPS_PORT`	The HTTPS port which exposes metrics.

Configuration parameter	Default Value
`TFE_METRICS_HTTP_PORT`	`9090`

Configuration parameter	Default Value
`TFE_METRICS_HTTPS_PORT`	`9091`

Capture metrics in two formats using the metrics endpoint URL. The table below provides the available options as well as the URL to use. We recommend capturing metrics over an encrypted connection.

Metrics Endpoint URL	Metrics format
`https://<tfe_instance>:9091/metrics`	JSON
`https://<tfe_instance>:9091/metrics?format=prometheus`	Prometheus

Terraform Enterprise computes an aggregate metric value from a 5-second sample. Terraform Enterprise keeps this value in memory for 15 seconds before flushing it. This means that if the monitoring tool pooling frequency is greater than 15 seconds (for example every 60 seconds), you may be missing information necessary to detect short-lived issues.

If you are running multiple Terraform Enterprise instances, you must collect the metrics from each deployed Terraform Enterprise instance and aggregate the information to have a global view of the Terraform Enterprise service.

> Note
>
> If you are using the `terraform-aws-tfe` module to deploy Terraform Enterprise, note that by default, it enables metrics collection (see the `enable_metrics_collection` input variable).

Configuring log collection on Terraform Enterprise

Terraform Enterprise emits logs to standard output and standard error. We recommend collecting Terraform Enterprise logs in a central location, preferably using a specialized tool that provides searching and alerting capabilities, although we support sending logs to object storage for example.

There is a limited number of supported log destinations.

Category	Supported log destinations
AWS	AWS S3, AWS CloudWatch
Microsoft Azure	Azure Blob Storage, Azure Log Analytics
Google Cloud Platform	Google Cloud Platform Cloud Logging
Specialized SaaS	Datadog, Splunk Enterprise HTTP Event Collector (HEC)
Other	Syslog, Fluent Bit or Fluentd instance

Note

The Terraform Enterprise HVD Module does not enable log forwarding (see the `log_forwarding_enabled` input variable) by default.The module has support for:1) Forwarding logs to AWS CloudWatch (using a built-in configuration template). 2) Forwarding logs to AWS S3 (using a built-in configuration template). 3) Forwarding logs to a custom destination.

Implementing audit trail on Terraform Enterprise

Terraform Enterprise generates audit logs alongside its service logs. Use the official Terraform Enterprise logs documentation as the source of truth for supported destinations, forwarding options, event types, and the current audit log format.

When forwarding Terraform Enterprise logs to a Security Information and Event Management solution (SIEM) or another centralized logging platform, parse the documented audit fields. The current documentation identifies the audit log component as `terraform-enterprise.audit` and documents event types such as authentication success, authentication failure, and CSRF violations.

7.3.2 HCP Terraform audit trail logs

HCP Terraform features an Audit Log API endpoint that you must use to collect the audit events and store them in the appropriate system. To implement this solution, you need the following.

- A method to schedule and automate the audit events collection,
- A secure storage solution to store the audit events, and

- A data lifecycle solution to correctly dispose of the audit events once they are no longer required.

Warning

HCP Terraform keeps audit trail records for 14 days and then discards it. Design and configure your audit trail collection solution for HCP Terraform with this in mind.

If you use a Security Information and Event Management system (SIEM), this must be the destination for those audit events. Suppose you are not using a SIEM but instead are using a centralized log management solution (Datadog, New Relic, Elastic, and so on). In that case, you must send those audit events to your centralized log management system. If neither of these solutions is available, still collect those audit events and store them securely using an object storage solution, such as AWS S3.

7.3.3 Metrics and logs on HCP Terraform agents

Configuring HCP Terraform agent metrics collection

The HCP Terraform agent binary exposes telemetry data using the OpenTelemetry protocol. This behavior allows the user to use a standard OpenTelemetry collector to push the metrics to a monitoring solution that supports the protocol, such as Prometheus or Datadog.

Because of that, to collect telemetry data from the agent, you need to have:

- A way to deploy and operate OpenTelemetry collector(s)
- A monitoring system that can integrate with OpenTelemetry collectors

Details about the selection of such a monitoring system or the operations of OpenTelemetry collectors are beyond the scope of this document. However, we provide some guidelines regarding integrating OpenTelemetry collectors with HCP Terraform agents.

Tip

If you are a Datadog customer, instead of deploying OpenTelemetry collectors, we recommend that you configure the Datadog agent to accept gRPC OpenTelemetry Protocol (OTLP) connections and then configure your HCP Terraform agent to use the Datadog agent as a metrics destination.

Because OpenTelemetry is a push system, you must start the collector before the HCP Terraform agent. Conversely, shut down the collector after you have stopped all HCP Terraform agents using that collector. We recommend having a one to one ratio of HCP Terraform agent instance and OpenTelemetry collector for long running HCP Terraform agents as this simplifies management.

The OpenTelemetry integration tags metrics with a number of useful fields, including the agent pool ID (`agent_pool_id`) and the agent name (`agent_name`). If you do not already have a naming convention for your HCP Terraform agents, then we recommend building one, as it helps you organize your dashboard with the metrics collected from HCP Terraform agents. You can then set the agent's name at startup time using the `TFC_AGENT_NAME` environment variable or the `-name` command line option.

Configuring HCP Terraform agent log collection

If you are using HCP Terraform agents in your Terraform enter deployment then configure log collection for agents.

7.4 Using observability data

7.4.1 Data collection and aggregation

As you collect and aggregate metrics and logs, you must consider the following.

- Volume (including granularity and frequency)
- Data retention
- Security and privacy

As you collect metrics, you must consider the *volume of information* relative to the value of individual metrics. This impacts the range of metrics collected and the collection frequency, and decisions in this area have clear tradeoffs.

Collecting a wide range of metrics at a high frequency yields the most data and potentially a more accurate view of the system's health and load. But it comes at a higher cost because you must collect, store, analyze, alert, and report on all this information.

One approach to control costs without reducing the range of metrics collected is to have clear *data retention rules* and leverage metrics roll-up.

We recommend keeping highly detailed metrics for shorter periods (we suggest two weeks). Discard metrics after two weeks, or roll them up (aggregating 60-second resolution data to 5 minutes, hourly, or per day). If your monitoring platform supports rolling up data, you may have the choice of implementing staged roll-up as follows.

- Full resolution for the last two weeks.
- 5 minutes roll-up for metrics older than two weeks, up to a month.
- Hourly roll-up for metrics over a month.
- Discard metrics older than three months.

For logs, we recommend keeping application logs for a short period (we suggest two weeks) and discarding them after that. For audit trail logs, retention is typically longer, and you need to align with the policy of your security organization.

When it comes to logs, be mindful that they may contain sensitive information. We recommend putting proper security measures in place to protect the collected metrics and logs. This includes implementing access controls, encryption (at rest and in transit), and monitoring mechanisms to safeguard sensitive information.

Warning

Mark variables as sensitive in HCP Terraform when they contain value that must be confidential (such as credentials). This causes the Terraform binary to redact this information in logs and in general take additional measures to prevent accidental data leaks.

7.4.2 Alerting and notification

Logging and audit capabilities in Terraform and Terraform Enterprise are essential for capturing and monitoring all events within the infrastructure management solution. Terraform Open Source mainly focuses on individual resource communication and API responses, while Terraform Enterprise provides comprehensive logs, including interactions with the solution, security events, and various inter-communication logs.

Terraform Enterprise offers application logs and audit logs. Application logs provide information about the services that make up Terraform Enterprise, while audit logs focus on documented security-related events such as successful logins and logouts through the admin console, failed authentication attempts, invalid or expired sessions, and CSRF violations.

For alerting and notification design, start with the official Terraform Enterprise logs documentation. It documents:

1. The current audit event types.
2. The standardized audit log fields, including `event_type`, `resource`, `source_ip`, `status_code`, `request_id`, and `actor_id`.
3. Log forwarding and retention considerations for external observability platforms.

This gives security and platform teams a stable reference for building log parsing, routing, dashboards, and alerts without depending on older internal log formats.

8 Backup and restore operations

> **Note**
>
> This page applies to Terraform Enterprise only. HCP Terraform backup and restore operations are operated by HashiCorp.

8.1 Introduction

Many business verticals require business continuity management (BCM) for production services. A reliable backup of your Terraform Enterprise deployment is crucial to ensuring business continuity. Your backup must include data held and processed by Terraform Enterprise's components so that operators can restore it within the organization's Recovery Time Objective (RTO) and to their Recovery Point Objective (RPO).

Below are the best practices, options, and considerations for backing up Terraform Enterprise. It also recommends standard operating procedures for redundant, self-healing configurations using public and private cloud infrastructure, which add resilience to your deployment and reduce the chances of requiring restoration.

This guide makes reference to the official Terraform Enterprise operating modes and is only relevant to Terraform Enterprise and not HCP Terraform. Also, most of this guide is only relevant to `external` mode and `active-active` deployments except where otherwise stated. Refer to Backup a `Disk` Deployment section below for specific details if you are running a `disk` deployment. If you have a `disk` deployment in production, we strongly recommend you liaise with your HashiCorp account

team regarding urgent migration to one of the other operational modes because, while supported, it is not recommended for production use.

For region redundancy, read this guide in concert with the *Terraform Installation Guide* Multiple Regions page.

For recovery and restoration of Terraform Enterprise, refer to the next page of this guide.

8.2 Definitions

Business continuity (BC) is a corporate capability. This capability exists whenever organizations can continue to deliver their products and services at acceptable, predefined levels whenever disruptive incidents occur. When we refer to *environments*, these map to engineering/pre-development, development, various test (system integration testing, user acceptance testing and so on) and production. Reserve dedicated engineering instances for the platform team only, for replication, backup/restore and upgrade testing.

> **Note**
>
> The ISO 22301 document uses business continuity rather than disaster recovery (DR). As a result, this tutorial refers to business continuity instead of disaster recovery.

Two factors heavily determine your organization's ability to achieve BC:

1. Recovery Time Objective (RTO) is the target time set for the resumption of product, service, or activity delivery after an incident. For example, if an organization has an RTO of one hour, they aim to have their services running within one hour of the service disruption.
2. Recovery Point Objective (RPO) is the maximum tolerable period of data loss during an incident. For example, if an organization has an RPO of one hour, they can tolerate the loss of a maximum of one hour's data before the service disruption.

Based on these definitions, we recommend establishing the valid RTO/RPO for your Terraform Enterprise instance and approach BC accordingly. These factors determine your backup frequency and other considerations discussed below.

In this guide:

- Reference to a public cloud *availability zone* (AZ) is equivalent to a single VMware-based *data center*.

- Reference to a public cloud *multi-availability zone* is equivalent to a *multi-data center* **VMware** deployment.
- The main AZ is the *primary*. Any other AZs in the same region are the *secondary*. The secondary *region* is a business continuity/failover region only and is not an active secondary location. Consider all availability zones equal for the purposes of illustration.

8.3 Best practices

8.3.1 Maintain the backup and restore process

When you deploy Terraform Enterprise:

- Test the backup and restoration process and measure the recovery time to ensure it satisfies your organization's RTO/RPO.
- Document the backup and restoration process.
- Arrange for staff who did *not* write this document to run a test restore using it. This measure increases confidence in the backup and restore process.
- Use a regular schedule to test the backup and restoration process to ensure the documentation is reliable. This increases familiarity with the backup and restoration process and covers situations where staff leave.

8.3.2 Manage sensitive values

For automated deployments, you must manage several sensitive values. On Kubernetes, refer to this support page for base information. The HVD deployment modules also provide further information specific to the deployment method and target platform - see below for further details. The methods below do not back up these data. We recommend HashiCorp Vault for storage of all pipeline secrets. Do not store any of these sensitive values in version control or allow them to leak into shell histories. Do not store any of these sensitive values in version control or allow them to leak into shell histories.

We recommend HashiCorp Vault for storage of all pipeline secrets.

8.3.3 Process audit logs

Audit log processing helps you identify the root cause during a data recovery incident.

Follow the guidance on this Terraform Enterprise logs resource page to aggregate and index logs from the Terraform Enterprise node(s) using a central logging platform such as Splunk, ELK, or a cloud-native solution. Use these as a diagnostic tool in the event of outage, scanning them case insensitively for `ERROR` and `FATAL` messages as part of root cause analysis.

8.3.4 Terraform Enterprise backup API

Terraform Enterprise has a Backup API but this primarily facilitates migrations from one operational mode to another. Only use the backup API for such migrations. Use cloud-native tooling instead for day-to-day backup and recovery on public cloud, and standard approaches for on-premise deployments as detailed below.

8.4 Initial considerations

The following recommendations improve your security posture, reduce the effort required to maintain an optimal Terraform Enterprise instance, and speed up deployment time during a restoration. Apply all relevant points in the list below for your target platform (VM-based or Kubernetes-based deployments).

- Harden server images using CIS benchmarking.
- Secure Terraform Enterprise deployments on both single-tenant and shared Kubernetes clusters. If using Kubernetes clusters, lock down all APIs properly using automated configuration capabilities such as Ansible.
- If using VM-based deployments of Terraform Enterprise, use single-tenant, immutable instances using automation by repaving instances with patched images rather than patching them in place. This process requires you to maintain the setup configuration in the code used to deploy the system and is out of the scope of this document.
- Remove all unnecessary packages from the operating system.
- Ensure to store deployment configuration in a version control system and use git best practices on the repositories used to version the code.

8.5 Application server

We recommend you automatically replace application server nodes when a node or availability zone fails. Replacing the node provides redundancy at the server and availability zone level. Public

clouds and VMware have specific services for this.

Click on the tab(s) below relevant to your cloud deployment for additional cloud-specific recommendations. The recommendation for all Kubernetes-based deployments is to deploy a cluster in line with the *Terraform Installation Guide* recommendations.

AWS

Select your deployment method below for more details. If you are deploying to EKS, your operational mode is `active-active`. You can deploy VM Docker-based instances in `external` mode or `active-active` mode. ::: {data-environment=tcolorbox data-environment-keyword="colframe=HexColor656A76,blanker,left=2mm,top=2mm,bottom=2mm,borderline west={0.5mm}{-1mm}{HexColor656A76}"}

external

- For `external` deployments, set the `min_size` and `max_size` of the ASG to 1. When a node or availability zone fails, the ASG automatically replaces the node. The time it takes for the ASG to replace the node depends on the time it takes the node to be ready. For example, the system comes up only after the node downloads the installation media from the network.

active-active

- For VM-based `active-active` deployments, set the `min_size` and `max_size` parameters to the desired number of nodes. These two values must be the same. If a node fails, the service remains up while the ASG replaces it.
- For Kubernetes-based `active-active` deployments using the HVD Module, in the Helm overrides file, set the `replicaCount` parameter to the desired number of nodes to influence how many server pods run on cluster. This defaults to 3. The module's `eks_nodegroup_scaling_config` variable sets the number of Kubernetes nodes in the node group. This also defaults to 3.

- For both `external` and VM-based `active-active` deployments, populate the ASG `vpc_zone_identifier` list with at least two subnets. If the region supports additional subnets, we recommend a minimum of three subnets since it provides `n-2` AZ redundancy.

Azure

Select your deployment method below for more details. If you are deploying to AKS, your operational mode is `active-active`. You can deploy VM Docker-based instances in `external` mode or `active-active` mode. Use a zone-balanced Linux virtual machine scale set (VMSS) to automatically replace nodes on Azure. Select your operational mode below for more details.For AKS-based deployments, ensure your region supports availability zones prior to deployment and confirm the number in the target zone prior to continuing. ::: {data-environment=tcolorbox data-environment-keyword="colframe=HexColor656A76,blanker,left=2mm,top=2mm,bottom=2mm,borderline west={0.5mm}{-1mm}{HexColor656A76}"}

external

- For `external` deployments, set the zone-balanced Linux VMSS `instances` parameter to 1. When a node or availability zone fails, the VMSS automatically replaces the node in the same region. The time it takes for the ASG to replace the node depends on the time it takes the node to be ready. For example, the system comes up only after the node downloads the installation media from the network.

active-active

- For VM-based `active-active` deployments, set the zone-balanced Linux VMSS `instances` parameter to the desired number of nodes. Two or more instances meet the Azure 99.95% service-level agreement for VM availability. If a node fails, the service remains up while the VMSS replaces it. `active-active` deployments require a fully automated deployment.
- For Kubernetes-based `active-active` deployments using the HVD Module, in the Helm overrides file, set the `replicaCount` parameter to the desired number of nodes to influence how many server pods run on cluster. The module's `aks_tfe_node_pool_node_count` variable sets the number of Kubernetes nodes in the node group. This defaults to 2 because of the number of regions which have fewer than three zones. We recommend using a minimum of three AZs and recommend testing deployment to ensure the node group size fits with your requirements.

- For both `external` and VM-based `active-active` deployments, if using Terraform to deploy, use the `azurerm_linux_virtual_machine_scale_set` resource. The HVD Module does this. Set the `zones` to a minimum of two (preferably three) zones in the region, and set `zone_balance` to true, which provides zone redundancy. :::

GCP

Use a regional managed instance group (MIG) to automatically replace nodes on GCP. Select your operational mode below for more details. ::: {data-environment=tcolorbox data-environment-keyword="colframe=HexColor656A76,blanker,left=2mm,top=2mm,bottom=2mm,borderline west={0.5mm}{-1mm}{HexColor656A76}"}

external

- For `external` deployments, set the `target_size` parameter of the MIG to 1. When a node or availability zone fails, the MIG automatically replaces the node in the same region. Enable the Google compute health check and configure the auto-healing policy of the instance group manager. The time it takes for the MIG to replace the node depends on the time it takes the node to be ready. For example, the system comes up only after the node downloads the installation media from the network.

active-active

- For `active-active` deployments, set the `target_size` parameter of the MIG to the desired number of nodes. If a node fails, the service remains up while the MIG replaces it. `active-active` deployments require a fully automated deployment.

- For both `external` and `active-active` deployments, if using Terraform to deploy, use the `google_compute_region_instance_group_manager` resource as this deploys a regional MIG, and thus ensures that the application server layer can automatically recover from a zone failure. The HVD Module does this. :::

VMware

In an `external` mode scenario, the application server is running as a stateless node.* If you deployed the instance from code (where the preceding sensitive data listed is available to the repaving of the node), you do not need to back up the instance. Redeploying the application server is acceptable. * If you deployed the instance through a vRA/vRO experience and need to manually configure it after it comes up, we recommend backing up the instance. Available VMware operational practices are likely to already include a strategic backup solution, which we would recommend using in this case. There are many VMware backup solutions, and the relative merits are outside the scope of this guide. HashiCorp customers back up Terraform Enterprise virtual machines using Dell EMC RP4VM and Veritas NetBackup, however, solutions vary widely.

:::

8.6 Object store

We recommend the following to support the object store's business continuity:

- Choose fast storage optimized for use that scales well and automatically replicates to another zone in the same region. Each public cloud has a well-known option in this space. For private cloud `external` mode deployments, you must use S3-compatible storage.
- Configure accidental/MFA deletion protection to prevent accidental deletion.
- Be aware that automated backup facilities provided by cloud service providers provide eventual consistency within a time window of around 10-15 minutes depending on network congestion. This means that in a failover situation, not all object changes may have succeeded - see below. It also means, however, that object corruption is also automatically replicated.

Click on the tab(s) below relevant to your cloud deployment for additional cloud-specific recommendations.

AWS

The most likely problem with the object store is service inaccessibility or corruption through human error rather than loss of durability, due to AWS's claim of eleven nines of durability.As a result, S3 Same-Region Replication is not explicitly required for the Terraform Enterprise object store because it does not add sufficient value. Corruption on the primary S3 bucket replicates to the secondary automatically.We recommend the following to ensure you back up your application data appropriately.* Refer to AWS's S3 FAQs for information about S3's durability. * Implement the security best practices for Amazon S3. * Enable versioning on the bucket used as the object store.

Azure

The most likely problem with the object store is service inaccessibility or corruption through human error rather than loss of durability, due to Azure's claim of eleven nines of durability.We recommend the following to ensure you back up your application data appropriately.* Use a zone-redundant storage account blob (ZRS) for a single region deployment. If you require a multi-region solution, then use Geo-zone-redundant storage (GZRS) and refer to the Terraform Installation Guide for more information. * Implement the security recommendations for Azure blob storage. * Enable point-in-time restores. - In order to do this, you need to also enable its prerequisites on the storage account: - Enable soft delete for blobs and set retention to the maximum 365 days. - Enable the change feed - Enable blob versioning - While Microsoft recommends maintaining fewer than one thousand versions per blob, Terraform Enterprise does not overwrite existing objects, so additional blob versions only exist as a result of workspace or accidental deletion operations. - Note the Azure limitations on storage container deletion. * Use a hot access tier, General-Purpose V2 Azure blob storage account. * The bucket must be in the same region as the worker node for maximum efficiency. * Use the `Microsoft.Storage` endpoint.

GCP

The most likely problem with the object store is service inaccessibility or corruption through human error rather than loss of durability, due to GCP's claim of eleven nines of durability. We recommend the following to ensure you back up your application data appropriately.* Refer to Google's Storage documentation for information about Google Cloud Storage's (GCS) durability. * Implement the security recommendations for GCS. * Enable object versioning. * Use a regional `Standard Storage` cloud storage bucket as an object store. If managing a multi-region platform, use a replicated bucket and refer to the *Terraform Installation Guide* Multiple Regions page for more information. * For maximum efficiency, the bucket must be in the same region as the managed instance group which uses it.

VMware

For on-premise `external` deployments, as the architectural requirements include an S3-compatible storage facility, such as minIO or Dell ECS:* If using minIO, use an active/pas-

8.7 Database

Configure the database to be in line with Terraform Enterprise's PostgreSQL requirements.

For high availability in a single public cloud region, we recommend deploying the database in a multi-availability zone configuration to add resilience against *recoverable* outages. For coverage against *non-recoverable* issues (such as data corruption), take regular snapshots of the database.

Click on the tab(s) below relevant to your cloud deployment for additional cloud-specific recommendations.

AWS

In addition to the preceding general recommendations, consider the following AWS-specific recommendations.* Implement the security best practices for Amazon RDS. * Use a multi-AZ deployment. AWS creates the primary DB instance in the *primary* AZ and synchronously replicates the contents to the standby instance in the *secondary* AZ. Refer to AWS's high availability for RDS documentation for more information. * Configure AWS Backup for RDS, using continuous backup and point-in-time-recovery (PITR). * If using Aurora as the Terraform Enterprise RDS database, you automatically benefit from PITR, continuous backup to Amazon S3, and replication across three availability zones. How many days of retention you require is a business decision, but HashiCorp recommends the maximum 35-day retention for maximum flexibility (the HVD Module uses this as a default), but we recommend performing a cost calculation relating to data storage. * Configure database snapshots. - AWS creates the default standby instance DB backup once a day. To achieve an RPO of less than a day, use snapshots taken at shorter time frames, and facilitate region-redundancy by using region-replicated buckets. They also persist beyond the 35-day PITR window. - Trigger DB snapshots automatically at required points during the day. The organization's needs and RPO determines when to trigger DB snapshots. - Continuously monitor the length of time it takes to do the backup and compare this to the RPO to avoid overlapping backups. * In both cases, ensure that backups and snapshots are secure and restrict access to only required staff. * Keep up-to-date with the AWS RDS documentation. * Actively and continuously monitor operational health, and configure automatic event notifications. * Store your snapshots in Amazon S3 in the same region as the platform to reduce recovery time. * The HVD Module used to deploy Terraform Enterprise to both VM- and EKS-based targets uses a variable `rds_preferred_backup_window` set to a default of `04:00-04:30` local time. - Override this in the terraform.tfvars file as needed. Check the value of `rds_preferred_maintenance_window` to ensure no overlap with `rds_preferred_backup_window`. - Set a minimum of 30 minutes. * The HVD Module also allows for the setting of `is_secondary_region` to deploy to two regions. Refer to the Multiple regions section of the Terraform Enterprise Installation Guide for more information on multi-region Terraform Enterprise deployments. * If your Terraform Enterprise uses Aurora, in the `aws_rds_cluster` resource: - Set `availability_zones` to a list of at least three EC2 availability zones. AWS requires a minimum of three availability zones, which is also what HashiCorp recommends to maximize the database layer's recovery capability. The HVD Modules allow for multiple AZs. * Test that the backups work by documenting the restore experience in your run book making edits as necessary to the backup procedure.

Azure

In addition to the preceding general recommendations, consider the following Azure-specific recommendations.* Follow the performance best practices for Azure Database for PostgreSQL. * Enable high availability on the Azure Database for PostgreSQL service (default configuration). - This provides automatic single-region redundancy for recover-

8.8 Redis cache

Because the Redis instance serves as an active cache for Terraform Enterprise, you do not need to maintain backups of the caching layer. The core recommendation is to maintain intra-region redundancy in line with other recommendations in the HVDs. Assuming you are running identical service resources in the secondary region, failing over the cache involves starting use of the cache cluster in the secondary region which commences unpopulated.

8.9 Backup a `disk` deployment

The backup approach for a `disk` operational mode deployment is simpler than for `active-active` mode because it involves a single machine and possibly its business continuity instance. Also, a `disk` deployment backup ensures the integrity of the machine and its attached data disk.

> Tip
>
> Read the Introduction, Definitions and Best Practices sections of this guide before continuing this section.

We only recommend using `disk` mode when provisioning on private cloud where the added complexity of managing an on-premise database, S3-compatible storage and a Redis instance are not already supported in your environment.

We do **not** recommend using `disk` deployments on public cloud since `external` and `active-active` modes provides better scalability. For Twelve Factor compliance, use the same operational mode for both production and non-production.

- Follow the advice in the preceding VMware tabs for information which is applicable to `disk` mode deployments as well.
- Ensure to quiesce the database on `disk` instances before taking a database snapshot. Your backup software may or may not do this automatically.
- `disk` mode uses a separate mountable volume (data disk) that can come in many flavors. To ensure data integrity, ensure the mountable volume has the following capabilities (in this order):
  - Continuous volume replication
  - Use of the same volume mounted on the original instance
  - A backup restored to another volume

- Make copies available in multiple data centers to confer DC-redundancy.

Isolated data disk

- If your primary `disk` node and a backup machine have their isolated data disks and maintain a mirroring strategy such as `lsyncd`, corruption on the primary volume replicates to the disk attached to the passive node. Maintain regular additional snapshots/backups of the data disk.

Shared data disk

- If your primary `disk` node and a backup machine share a SAN-provided data disk (or equivalent network device), use on-premise replication technologies to ensure that the SAN volume has inherent cross-data center resilience. Use additional regular backups to address issues of replicated corruption. Enable versioning if the resource that serves the mounted data disk has versioning.

Note

Do not start more than one `disk` mode instance against the same database simultaneously. If you are using a load balancer and a warm server with the data disk visible in the other data center, ensure Terraform Enterprise is not running on it while the primary is.

8.10 Restore Terraform Enterprise

All business verticals require business continuity management (BCM) for production services, and the rise in cyber crime has intensified these requirements. To ensure business continuity, you must implement and test a recovery and restoration plan for your Terraform Enterprise deployment prior to go-live. This plan must include all aspects of data held and processed by Terraform Enterprise's components so that operators can restore it within the organization's recovery time objective (RTO) and to their recovery point objective (RPO).

This guide discusses the best practices, options, and considerations to recover and restore Terraform Enterprise. It also recommends redundant, self-healing configurations using public and private cloud infrastructure, which add resilience to your deployment and reduce the chances of requiring backups.

This guide assumes you are familiar with Terraform Enterprise backup processes and procedures. If not, first refer to the Terraform Enterprise Backup page to create backups for your Terraform Enterprise instance.

Most of this guide is only relevant to *single-region*, multi-availability zone, `external` or `active-active` operational mode deployments except where otherwise stated. Having a multi-region deployment does not preclude your potential need to restore data in one of the regions, because all automated data replication services you might use for active/passive architectural models automatically replicate corruption events.

We recommend you automate the recommendations listed below to reduce the recovery time.

> Note
>
> If you are experiencing an outage or need to recover your Terraform Enterprise instance, contact HashiCorp Support for direct assistance.

8.11 Definitions and best practices

Business continuity (BC) is a corporate capability. This capability exists whenever organizations can continue to deliver their products and services at acceptable, predefined levels whenever disruptive incidents occur.

> Note
>
> The ISO 22301 document uses business continuity rather than disaster recovery (DR). As a result, this guide refers to *business continuity* instead of *disaster recovery*.

Two factors heavily determine your organization's ability to achieve BC:

1. Recovery Time Objective (RTO) is the target time set for the resumption of product, service, or activity delivery after an incident. For example, if an organization has an RTO of one hour, they aim to have their services running within one hour of the service disruption.
2. Recovery Point Objective (RPO) is the maximum tolerable period of data loss after an incident. For example, if an organization has an RPO of one hour, they can tolerate the loss of a maximum of one hour's data before the service disruption.

Based on these definitions, we recommend establishing the valid RTO/RPO for your Terraform Enterprise instance and approach BC accordingly. These factors determine your backup frequency

and other considerations discussed below.

In this guide:

- See here for a definition of point-in-time-recovery (PITR).
- We express outage time as `T0`. The number after `T` represents the time relative to the outage time in minutes. For example, `T-240` is equivalent to four hours before the established outage event start time.
- *Source* refers to the Terraform Enterprise instance that you need to recover. *Destination* refers to its replacement.
- A public cloud *availability zone* (AZ) is equivalent to a single VMware-based *data center*.
- A public cloud *multi-availability zone* is equivalent to a *multi-data center* VMware deployment.
- The main AZ is the *primary*. Any other AZs in the same region are the *secondary*.

Note

Depending on the chosen Terraform Enterprise architecture and site-specific configuration aspects, planning, deployment, and confirmation must be done as part of the overall platform management process. While this can be a time-consuming process, we cannot understate the importance of reliable understanding of what to do in an outage situation.

8.11.1 Data management

Select the tab below for *high level* best practices for your scenario.

Public clouds

Use the Terraform Installation Guide when designing your Terraform Enterprise deployment. This allows you to integrate CSP native tooling, where recommended, to support automated single-region recovery capabilities.For single-region, multi-availability zone deployments:* Your public cloud database deployment automatically replicates the database to at least one other availability zone. * Your object storage service remains online during an availability zone failure.

Private clouds

Select the tab below for high level best practices for your deployment, either `external` or `disk` operational modes. ::: {data-environment=tcolorbox data-environment-keyword="colframe=HexColor656A76,blanker,left=2mm,top=2mm,bottom=2mm,borderline west={0.5mm}{-1mm}{HexColor656A76}"}

external

If you have an `external` mode multi-data center deployment, your PostgreSQL database and S3-compatible object storage automatically replicate to another data center.1) Confirm the replication configuration to ensure these components are actually replicating. 2) Ensure the integrity of the database and object storage copies in the *secondary* data center by suspending the replication to mimic the loss of the *primary* and then deploying a Terraform Enterprise application server configured to connect to the external services in the *secondary* and ensure the platform comes up. 3) Flip the DNS record for the *primary* instance so it resolves to the *secondary* instance to ensure the process works, fully making use of a maintenance window if applicable.

disk

If you have a `disk` mode multi-data center deployment, your application server has PostgreSQL database and object storage mounts.1) Ensure the VM replication process to the *secondary* is working. 2) Confirm that service starts using the replicated data layer in the *secondary*. Take the *primary* instance down before bringing up the *secondary*. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFFF9E8,colframe=HexColorFBEABF,coltitle=HexColor9E4B00,title=Note"} Since Terraform Enterprise encrypts the data in both the object storage and database, you must ensure you can recover by deploying the Terraform Enterprise instance in the *secondary* data center. Unless your work results in a working Terraform Enterprise instance in your *secondary* data center, you cannot know if the restoration process you are working towards works when you need it.

:::

:::

8.11.2 Record event data

In an outage situation, for all scenarios, we recommend that you record event data. This enables you to:

- Perform root cause analysis.
- Work with HashiCorp Support to reduce losses incurred.
- Identify tasks to prevent similar outages from occurring in the future.

For incident management, record the following information as soon as possible.

- The date and time when the change that led to the outage occurred.
- The date and time of the most recent, available runs and state files.
- The date and time of the most recent, available PostgreSQL database snapshot or backup.
- Whether the Terraform Enterprise application configuration values are safe.

8.11.3 Relationship between database and object store

Consider the database and object storage as one conceptual data layer for Terraform Enterprise even though they are technically separate. The database stores links to objects in the object storage and the application uses these links to manipulate the objects stored. For example, this means that restoring the database to an earlier version would then mean lost links to objects created since, even though those objects would still be present in the object store.

It is important to establish *when* the incident began. For example, if a workspace run had created a VM in a workspace at 11AM, if the state file is then corrupted, rewinding to a 10AM state version would mean that the state file would *not* reflect the running VM.

The steps to remediate any one issue depend on the issue details, so this document provides the main concepts and recommendations. If you have specific questions which are not addressed in this document, contact HashiCorp Support if you are troubleshooting an event.

8.12 Preparation

The steps to recover a down service depends on what the specific problem is, and which cloud scenario.

Before you attempt to recover the service, use the node-drain command on the application servers one at a time. This command attempts to stop processing of background jobs on the Terraform Enterprise node in a graceful manner.

8.13 Application server

As part of BCM, expect, and plan, for any outages. Well-known reasons for Terraform Enterprise application server failing include: underlying network or physical circuitry failures, operating system data issues, or kernel panics. Human error could also cause the application server to fail.

8.13.1 Application server recovery and restoration

For both `external` and `active-active` operational mode deployments, design your application servers to automatically replace failed worker nodes and to handle availability zone failures. This deploys redundancy at both the server and availability zone level.

> **Warning**
>
> Do not run two Terraform Enterprise `external` mode application servers which interact with the same external services at the same time. This can cause database or other corruption.

The configuration *must* be the same in both the node and its replacement. Terraform Enterprise uses the encryption password (`TFE_ENCRYPTION_PASSWORD`) to decrypt the internal Vault unseal key and root token. If you do not have the encryption password, **you cannot recover your application data**.

Refer to the tab(s) below for specific recommendations relevant to your implementation.

Public Cloud

- If you encounter a VM scaling group issue, redeploy using an automated deployment capability.
- You do not need to *restore* failed application servers - replace them using automated means.
- For both `external` and `active-active` operational mode deployments, when the VM automatically restarts, expect to find `502` and `503` errors from the browser while the service also restarts.

Private Cloud

For both `external` mode and `disk` mode deployments on VMware, consider the following:* VMware customers using a stretched cluster across the sites must rely on VMware vSphere High Availability to recover a failed VM. * Customers that are not using stretched clusters are to rely on vSAN replication with software orchestration such as Site Recovery Manager. * vMotion is not considered an implicit backup and recovery service. * If you have an `external` mode deployment on premise, redeploy the application server via vRA/vRO, OpenStack Nova/Ironic, or equivalent so that it reconnects to your on-premise object store and database. * If automated, IaC-based replacement is impossible, restore it from a backup unless manual redeployment is quicker. To ensure that reconnection is automatic, it is *critical* to that your vRA template/Glance template/post-deploy configuration management includes the same connection details to the object store and database as the failed VM.We recommend that you use the option which results in the shortest time-to-resolution.

8.14 Object store

Terraform Enterprise object storage contains a historical set of state files for all of your workspaces, logs of workspace runs, plan records and slugs, and work lists that persist tasks from plans to applies.

Terraform Enterprise deployments compliant with the reference architectures and Backup Recommended Pattern use either public cloud storage with eleven nines of availability, or an equivalent S3-compatible private cloud storage facility. This means that multiple copies of object store data exist in multiple locations. As a result, during *expected* outages, like disk or network failure, single-region object storage recovery is automatic. If you need region redundancy, you use regional object

storage replication. Refer to the Multiple Regions section for more information.

This section covers best practices for object storage restoration from *unexpected* outages, like human error or corruption events.

8.14.1 Object store content loss and corruption

This section describes the effects of each object storage file type's loss and corruption on Terraform Enterprise. Refer to the next section for object storage file recovery recommendations. The recovery method depends on whether there is loss or corruption and also the cloud provider you are using.

Workspace current state loss and corruption

If the object store does not have a current workspace state file, this presents two problems:

1. The missing state file has represented either addition or deletion of running infrastructure.
2. You experience the following error on the next workspace run since this current state file is not readable.

```
Failed to save state: Error uploading state: Precondition Failed

The serial provided in the state file is not greater than the serial
currently known remotely. This means that there may be an update performed
since your version of that state. Update to the latest state to resolve
conflicts and try again

Error: Failed to persist state to backend.

The error shown above has prevented Terraform from writing the updated state
to the configured backend. To allow for recovery, the state has been written
to the file "errored.tfstate" in the current working directory.

Running "terraform apply" again at this point will create a forked state,
making it harder to recover.

To retry writing this state, use the following command:
    terraform state push errored.tfstate

Found errored.tfstate file, dumping contents...
```

If corruption occurs to the *current* state file of a workspace, the next run hangs and eventually fail with the following output.

```
Configuring remote state backend...
Initializing Terraform configuration...

Setup failed: Failed terraform init (exit 1): <nil>

Output:

Initializing the backend...

Successfully configured the backend "remote"! Terraform will automatically
use this backend unless the backend configuration changes.
There was an error connecting to the remote backend. Please do not exit
Terraform to prevent data loss! Trying to restore the connection...

Still trying to restore the connection... (2s elapsed)
Still trying to restore the connection... (5s elapsed)
...
Still trying to restore the connection... (5m21s elapsed)
Still trying to restore the connection... (5m42s elapsed)
Error refreshing state: Error downloading state: 500 Internal Server Error
```

Workspace non-current state loss and corruption

A workspace may contain non-current states (previous versions of the state file). If the object store does not have a workspace's non-current state, this does not impact workspace runs. However, when you try to access the missing state version in the UI, the following error results.



Figure 12: Failed State View

If corruption occurs in a workspace's non-current state, this does not impact workspace runs. However, when you try to access the corrupted state version in the UI, the following error results.



Figure 13: Failed State View

Logs loss and corruption

Find Terraform Enterprise logs in the following location based on your deployment.

Operational Mode	Log Path
`external`	`/archivistterraform/logs`
`disk`	`/data/aux/archivist/terraform/logs`

If log files for historical plans and applies are missing or corrupted, this does not impact workspace runs. However, when you try to access the missing state version in the UI, the following error occurs since the Terraform Enterprise is unable to access the log files.

```
undefined
```

> Note
>
> Since the Archivist service uses Redis to cache certain data for several hours, the UI may still be able to access run log data until the cache expires.

JSON plans and provider schemas loss and corruption

Terraform Enterprise writes JSON plans and provider schemas during normal operations.

- For Terraform Enterprise version `v202108-1`, only the Sentinel policy evaluation uses these objects.
- From Terraform Enterprise `v202109-1` and later, the UI uses both JSON plan and provider schema objects to render structured plan output if configured in the workspace.

If the JSON plans and provider schemas are missing or corrupted, you are unable to view the structured plan rendering for historical runs.

Slugs loss and corruption

Some slugs contain caches of the most recent workspace run configurations. If the object store does not have those slugs, the following error occurs when you try to start a new plan from an existing workspace.

```
Setup failed: failed unpacking terraform config: failed to uncompress slug: EOF
```

However, since Terraform Enterprise generates new slugs for new workspace runs, new workspace runs are not affected.

The loss of slugs that do not contain workspace run configurations do not affect system operation and workspace runs. There is no way to differentiate slug types from object storage content listings.

Bucket loss and corruption

If you lose the object store, the following message results, similar to lost slugs containing workspace run configurations.

```
Setup failed: failed unpacking terraform config: failed to uncompress slug: EOF
```

If corruption occurs to all of the objects in the object store, the UI can still operate with cached objects. When the cached objects expire, Terraform Enterprise tries to access the data from the object store. When this happens, the same preceding error occurs.

8.14.2 Recover lost object store content

The primary way to recover lost object store content is to enable object store versioning. Using this, you can recover deleted files from the versioned object storage by un-deleting the files. Refer to the tab(s) below for specific recommendations relevant to your implementation.

AWS/S3-compatible

A versioned S3 bucket (or S3-compatible equivalent) creates a delete marker for the removed object. To recover this object, follow the steps below.1) Create a list of workspaces or individual objects impacted by the missing object and its respective severity. 2) For each affected object, remove the delete marker(s) to restore each item. 3) Refresh the UI after you recover all impacted objects to find the recovered objects. 4) Optionally, start a new workspace plan to verify that you recovered the objects.

Azure

With point-in-time restore enabled in Azure Blob storage, you can restore deleted objects by following the steps below.1) Determine the outage time (`T0`). Use this value for the `--time-to-restore` flag in the restore command. 2) Create a list of workspaces or individual objects impacted by the missing object and its respective severity. 3) Use the `az storage blob restore` Azure command line command to recover the missing object. If you need to recover multiple files, use the `--blob-range` flag. 4) Refresh the UI after you recover all impacted objects to find the recovered objects. 5) Optionally, start a new workspace plan to verify that you recovered the objects.

GCP

With object versioning enabled for a bucket, you can restore deleted objects. Use the versioning document which guides you through enabling GCS bucket versioning via the command line. We strongly recommend enabling bucket versioning with the Terraform `google_storage_bucket` resource.Follow the tasks in the GCS restore documentation or the steps below to recover deleted Terraform Enterprise objects.1) Create a list of workspaces or individual objects impacted by the missing object and its respective severity. 2) For each affected object, get the first non-current object version path, replacing `x` with the affected object's path.`shell-session gsutil ls -a gs://my-bucket/archivistterraform/states/xxxxxxxx/sv-xxxxxxxxxxxxxxxx | \ sort -nr | head -2 | tail -1`Then, restore each path to make a new current object version.`shell-session $ gsutil cp gs://my-bucket/archivistterraform/states/xxxxxxxx/sv-xxxxxxxxxxxxxxxx\#xxxxxxxxxxxxxxxx \ gs://my-bucket/archivistterraform/states/xxxxxxxx/sv-xxxxxxxxxxxxxxxx` ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFAFAFA,colframe=HexColorAEAEB2,coltitle=HexColor3B3D45,title=Tip"} GCS timestamps object versions with a generation number, equivalent to the number of microseconds since the epoch.

1) Refresh the UI after you recover all impacted objects to find the recovered objects.
2) Optionally, start a new workspace plan to verify that you recovered the objects.

:::

If you delete a workspace from Terraform Enterprise, you cannot recover it by manipulating the object storage alone. You also need to restore the database since links to the object store also need restoring. If the workspace deletion occurs after the last database backup, you cannot recover it.

In addition, when you delete a workspace, Terraform Enterprise deletes the corresponding objects from the object store, which creates delete markers in the S3 bucket. If you remove all delete markers to recover from an accidental object storage deletion, this also recovers objects from deleted workspaces. If you do not restore the database in this situation, the database would still not have links to those restored objects. This may be an acceptable option as opposed to identifying and undeleting only those deleted objects. Consult HashiCorp for specifics which may impact decision-making.

If you are unable to use these recommendations to recover the lost objects, use point-in-time-recovery (PITR) scripting to restore all the objects in the object store to just before the outage time (`T0`).

> **Tip**
>
> Take Terraform Enterprise down prior to using PITR scripting.

Since using PITR scripting effectively takes Terraform Enterprise back through time, restore the database to the same time. Follow the guidance in the Database section below. Ensure that the object store is more recent (younger) than the database, otherwise, the database may contain broken object storage links which has further adverse impact on the platform.

8.14.3 Recover corrupt object store content

You can recover corrupt files from versioned object storage by going back to the last-known "good" version.

In this section, *current* refers to the latest available state files which represent the existing deployed infrastructure. *Last good* refers to the most recent previous state file that Terraform Enterprise can process without error.

Terraform Enterprise writes a new state file in the object store for every applied workspace run that requires a change in resources. Terraform Enterprise stores each state file as separate objects. If

corruption occurs in a workspace's *current* state file, all subsequent runs fail. Use the API to recover corrupted state data since you know the failed workspace's name.

The following example demonstrates this process. The table below represents an example workspace with three applies, each adding one VM to the cloud. The *current* state file represents three VMs running on the cloud. However, it is corrupt.

```
┌──────────────────────┬────────┬──────────────────────────────────────┐
      ID │                 SERIAL │ CREATED
         ├┼──────────────────────┼────────┼──────────────────────────────────────┤
+VM3 | sv-2K1mF7GUimf12bEd |      2 | 2021-09-08 15:26:09.228 +0000 UTC | <- corrupt
    current
+VM2 | sv-KxRnWxmpFsYqsNzp |      1 | 2021-09-08 15:24:15.584 +0000 UTC | <- last
    good
+VM1 | sv-gdBm2KUqDuQdzSvQ |      0 | 2021-09-08 15:23:00.23 +0000 UTC
    │└──────────────────────┴────────┴──────────────────────────────────────┘
```

In order to recover the corrupted state, you need to:

1. download the state file for the *last good* run (serial 1),
2. change the serial to 3, and
3. upload it as the new *current*.

This process assumes that Terraform Enterprise is running and the API is available.

The Version Remote State with the HCP Terraform API tutorial covers how to:

1. download the state file
2. modify and create the state payload (**only** update the serial to 3)
3. upload the state file

These instructions are similar for Terraform Enterprise. Update the hostname, organization, and workspaces to reflect your workspace you want to recover.

> **Note**
>
> Due to the number of steps involved, we recommend you automate the process where possible. You can automate the preceding API integration steps using the *unofficial* tfx Go binary and this State Push script may also be of use.

Once complete, the following occurs.

```
┌──────────────────────┬──────────┬──────────────────────────────────────┐
      ID │                  SERIAL │ CREATED
         ├┼──────────────────────┼──────────┼──────────────────────────────────────┤
+VM2 | sv-xceRBCuExqFdvLEB |      3 | 2021-09-08 16:01:39.372 +0000 UTC | <- new
    current <--+
+VM3 | sv-2K1mF7GUimf12bEd |      2 | 2021-09-08 15:26:09.228 +0000 UTC | <-
    corrupted      |
+VM2 | sv-KxRnWxmpFsYqsNzp |      1 | 2021-09-08 15:24:15.584 +0000 UTC | <- last
    good -----+
+VM1 | sv-gdBm2KUqDuQdzSvQ |      0 | 2021-09-08 15:23:00.23 +0000 UTC
    │└──────────────────────┴──────────┴──────────────────────────────────────┘
```

Since the latest run ID (serial 3) is the same as the *last good* one, the listed states in the UI shows two entries with the same run ID.

Set the apply method of the workspace to `Manual apply`.

Warning

Do not run `terraform plan` on a workspace in this situation if it has `auto apply` set. If Terraform Enterprise detects missing infrastructure due to changes to the state, it automatically proceeds to deploy duplicate infrastructure. This is undesirable in an outage.

Then, start a plan so Terraform Enterprise can identify which infrastructure to add into the state file so it reflects the current infrastructure. From the preceding table, the most recent added virtual machine is not represented in the latest state.

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Note

For VCS-backed workspaces only, you need to temporarily alter the workflow to use a remote backend by removing the VCS connection in the UI.

In your local working copy of the repository, add, or modify, the `remote` backend in the `terraform` block to update the remote state on Terraform Enterprise. Replace `hostname`, `organization` and `workspaces.name` with your values.

> **Note**
>
> The Terraform remote backend is only supported on `v0.11.12` and later.

```
terraform {
  required_version = "~> 1.14.1"
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }

  backend "remote" {
    hostname = "tfe.example.com"
    organization = "my-org"

    workspaces {
      name = "workspace1"
    }
  }
}
```

Then, configure the provider credentials in your local terminal.

Initialize your configuration when you are ready to import the existing infrastructure.

```
$ terraform init
```

Then, run `terraform import` for each listed object from the plan output to update the recovered state file. Obtain the necessary object ID(s) by referring to the relevant cloud resource. The following example command imports an EC2 instance with an ID of `i-03a474677481b3380`.

```
terraform import aws_instance.web i-03a474677481b3380

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.
```

When you are recovering corrupted state files, remember that infrastructure deletion is also possible. Consider the following example:

- At `T-480` (8 hours ago), you have a workspace that deployed a virtual machine.

- At `T-240` (4 hours ago), you backup your database.
- At `T-120` (2 hours ago), you apply your configuration, which deletes the VM.
- At `T-0`, you experience a corruption event.

To recover the last good state, you restore your database to `T-240` and rewind your object store to the same time. At this point, even though your state contains a VM, your configuration and the cloud API reflects no VM. If you generate a plan, it returns no changes.

Reconcile your state file with your configuration by running `terraform apply -refresh-only`.

```
$ terraform apply -refresh-only
```

You have restored and reconciled a corrupted or missing state file. If the workspace is VCS-backed, when all imports have completed, re-establish your VCS connection to HCP Terraform and re-run a final `terraform plan`. This returns no changes.

8.15 Database

This section discusses how to restore and recover your Terraform Enterprise PostgreSQL database. This assumes you followed the Backup page of this HVD. It also assumes point-in-time recovery on public clouds providing granular roll-back capability.

8.15.1 Database recovery

If there is an outage in multi-availability zone, single-region database instances and multi-DC private cloud, the database automatically switches the service to the *secondary* AZ/DC and the service resumes. If the database is in the process of failing over to a different availability zone when a workspace run triggers, the run may initially hang, resulting in possible `504` or `502` errors.

504 Error

```
Error queueing plan
504 Gateway Time-out
```

502 Error

```
Run auto-refresh canceled
502 Bad Gateway
```

If this happens, restart the failed run when the database reconnects.

8.15.2 Database restoration

You need to restore your database when unexpected database failures or corruption occurs (due to human errors, or hardware/software failures). The speed it takes you to restore your database after an outage depends on your RPO, and the frequency of your database backups and snapshots.

Consider the following:

- If a database worker node goes down, all database connections drop. The database may report open connections or pool usage until it hits its configured timeout.
- Terraform Enterprise uses write locks to protect the database writes. However, if a connection terminates mid-write, database corruption may occur. This is a general concern related to PostgreSQL database management rather than specific to Terraform Enterprise. This is why it is important to ensure the database backup is sound.
- The restoration method depends on the severity of the database corruption. In some cases, you could just re-index the database; in others, you need to do full restore.
- The database stores the Vault unseal token. It is important because it allows Terraform Enterprise to decrypt the data in the object store. Since this is a small entity by comparison to the size of the database and not updated often, there is a small chance of corrupting the unseal token.

If you need to restore the database, the recommended strategy is:

1. Notify your user base of the outage.

2. Determine the outage time (`T0`), so you have an anchor point in time to focus restoration efforts.
3. Take the application down.
4. Create a *new* database instance by restoring from a backup and applying relevant snapshots as applicable.
5. Create a new *destination* object store and copy the objects from the *source* object store attached to the broken instance.
6. Deploy a fresh Terraform Enterprise instance, configured to use the *new*, restored database and *destination* object store.

Reference the tab(s) below for specific recommendations relevant to your implementation. These recommendations focus on single-region restores. Refer to the multi-region section for multi-region considerations.

AWS

Note

The HVD deployment modules deploy running, whole instances. As a result, you must specify an existing object storage and database paths for deploying Terraform Enterprise instances in restoration scenarios which thus requires editing of the module code. We cannot predict what the requirements might be in a given scenario, thus cannot provide code for every eventuality.

- Migrate copies of the *source* S3 bucket objects to the *destination* bucket. The copies of the *source* S3 bucket objects must match as close to the age of the database as possible. If you cannot restore their ages to the same time as the database restore time, then **the state must be more recent (nearer to `T0`) than the database**.
- Script a solution to manage this transfer. The open source tool `s3-pit-restore` is also useful although unofficial. You cannot use `s3-pit-restore` to restore all buckets in a single command, but it allows you to specify a precise time. ::: {data-environment=tcolorbox data-environment-keyword="colframe=HexColor656A76,blanker,left=2mm,top=2mm,bottom=2mm,borderline west={0.5mm}{-1mm}{HexColor656A76}"}

Using AWS Continuous PITR

- If you are using Amazon RDS continuous backup and PITR, set your recovery point to just before `T0`, so the time-to-recover (TTR) is short.
- In `AWS console` > `AWS Backup`, select your desired backup then click the `Restore` button in the upper right. This creates a new database.

Not using AWS Continuous PITR

- If you are not using Amazon RDS continuous backup and PITR, or if the database that you need to restore is more than 35 days old, you must restore the last good database backup.
- In the `AWS console` > `RDS` > `Databases` pane, select the Terraform Enterprise database and then select the `Maintenance & backups` tab to see available snapshots.

Azure

Note

The HVD deployment modules deploy running, whole instances. As a result, you must specify an existing object storage and database paths for deploying Terraform Enterprise instances in restoration scenarios which thus requires editing of the module code. We cannot predict what the requirements might be in a given scenario, thus cannot provide code for every eventuality.

- Migrate copies of the *source* storage blob objects to the *destination* blob which match as close to the age of the database as possible. If you cannot restore their ages to the same time as the database restore time, then **the state must be more recent (nearer to `T0`) than the database**.
- Use the `az storage blob copy` command to asynchronously duplicate the required objects to the *destination* instance.
- Azure Database for PostgreSQL automatically configures a backup retention of seven days so that your data is restorable to a point-in-time.
- Follow these instructions to restore your database to just before the problem started.

GCP

Note

The HVD deployment modules deploy running, whole instances. As a result, you must specify an existing object storage and database paths for deploying Terraform Enterprise instances in restoration scenarios which thus requires editing of the module code. We cannot predict what the requirements might be in a given scenario, thus cannot provide code for every eventuality.

- Migrate copies of the *source* GCS bucket objects to the *destination* bucket. The copies of the *source* GCS bucket objects must match as close to the age of the database as possible. If you cannot restore their ages to the same time as the database restore time, then **the state must be more recent (nearer to `T0`) than the database**. For example, if your `T0` is 22:00, and you backed up your database at 19:40 (`T-140`), the following bash script generates a list of paths that fall before the specified date.“‘bash for i in $(gcloud storage ls --recursive gs://source_bucket | grep '#') do STAMP =(echo $i | cut -d '#' -f 2) GETMETOBEFORE =(printf “%d000000”
$(date -d ‘2025/09/10 19:40:00’ + “%s”))

if [“$STAMP" -lt "{GETMETOBEFORE}”] then echo ${i} fi done “* `Use the`gcloud storage cp‘ command to copy the required objects to the destination instance.

VMware

- Migrate copies of the *source* S3-compatible bucket objects to the *destination* bucket. The copies of the *source* S3-compatible bucket objects must match as close to the age of the database as possible. If you cannot restore their ages to the same time as the database restore time, then **the state must be more recent (nearer to `T0`) than the database**.
- If you have a database backup facility which ships WALs across data center, corruption is likely replicate to the *secondary*.

:::

- Create a new database from the backup using the same settings. Note the path and connection details.

- Copy the deployment configuration file (Kubernetes Helm overrides.yml file or compose.yml for Docker Compose) from the *source* server(s) to the *destination* instance's deployment configuration. When the autoscaling group boots a new server(s), each one must have the correct configuration and be able to connect to the restored *destination* database instance and *destination* object store.
- Create the *destination* Terraform Enterprise instance using the *destination* database and object storage.
- Confirm platform health.
- Amend DNS to point the service address to the new load balancer.
- Since the object store contains a previous state in time, each workspace applied between the database restore point and `T0` requires a `terraform import` of any *running* infrastructure *not* represented in the state. Follow the example in the Recover corrupt object store content section in this document to do this.
- Notify your user base when you are back online.

8.16 Redis cache

This section is only relevant if you are running the `active-active` operational mode. There are no explicit backup requirements because Terraform Enterprise uses Redis as a cache. However, ensure your Redis instance has regional availability to protect against zone failure.

8.17 Restore a `disk` mode Terraform Enterprise instance

This section discusses the recovery and restoration process for `disk` operational mode Terraform Enterprise instances. Refer to the Operational Mode Decision documentation for information on Terraform Enterprise operating modes.

8.17.1 `disk` mode recovery

Because the same mounted volume stores the Terraform Enterprise object store and database, recovery of `disk` mode deployments involve recovery of the machine or associated disk using appropriate recovery technologies, depending on the outage.

If you are using tooling such as Dell EMC RecoverPoint for Virtual Machines or HPE Zerto, you already have systems policy set up to recover the Terraform Enterprise workload automatically in

the case of a failure. Online continuous data protection platforms are strongly recommended for automated recovery when using single-machine Terraform Enterprise deployments, and we recommend using these to recovery the system in an outage situation. The process differs depending on the recovery platform in place. The alternative to this is to expect to restore the platform in the event of an outage.

8.17.2 Mounted disk mode restoration

Restoration of `disk` operational mode instances involve deploying a replacement VM from a backup. As the establishment of business RTO and RPO are the same for `disk` mode and `external` mode deployments, the amount of remediation required after restoration completes depends on how far back you have to go to restore your data. Refer to the Object Store section in this document for recommendations regarding import of deployed infrastructure objects when updating workspace state.

When restoring Terraform Enterprise in `disk` mode, consider the following steps.

1. Notify your user base of the outage.
2. Determine the outage time (`T0`).
3. Based on the Reference Architecture for VMware, you have data intact in one or both centers. You could have done this by replicating the data layer using a technology such as `lsyncd` in the case of isolated data disks, or using a shared device from a SAN or NAS.

> **Isolated data disks**
>
> If you are using isolated data disks, the *secondary* Terraform Enterprise host must be up to facilitate data replication. However, to avoid corruption, shut down the application on the *primary*.

> **Shared data disks**
>
> If you are using shared data disks, the *secondary* host may have been down. This method is actually safer because it is less likely that the application could start and two instances write to the database at the same time, risking corruption.

Bring up Terraform Enterprise on the *secondary* host, and confirm it can read the data disk.

No Data Corruption

If there is no data corruption, the load balancer uses the *secondary* data center. If you are not using a load balancer, amend DNS instead.

Data Corruption

If there is data corruption in the database, stop Terraform Enterprise and Docker. Then, restore the data disk from a backup.

1. When the restore is complete, start Docker. If you establish data integrity Terraform Enterprise starts.
2. Availability governs the data center you decide to rerun the preceding processes in. If you have brought up Terraform Enterprise in the *secondary* data center, there is either a natural business process regarding safe return to the *primary* data center, or, if the data centers are equivalent, no further technical work is necessary.
3. Notify your user base when you are back online.

9 Disaster recovery operations

Note

This page applies to Terraform Enterprise only. HCP Terraform disaster recovery is operated by HashiCorp.

Note

This page is only relevant to customers who operate two `external` or `active-active` Terraform Enterprise instances in different public cloud regions, and actively replicate the object store and database layers from one region to another.

Fail over a Terraform Enterprise instance only during failover testing or by following a combination of the recommendations below and your site-specific run book for the same.

Customers deploy the application in a range of contexts and regulatory environments, so this page provides prescriptive, but generic, guidance in the regard.

For purposes of clarity, references to `primary` are to your main active operating region, and `secondary` are to your passive, failover region that you replicate your data *to* in case the `primary` region goes offline. Once the outage has passed, the inference is that you failback *from* the `secondary` *to* the `primary`.

9.1 Detection of need

We recommend deep observability provision around Terraform Enterprise both in terms of live metrics consumption and processing the diagnostics API. Further, we strongly recommend connecting alerts to these metrics in case `200` responses are not experienced.

These provisions afford the Terraform Enterprise platform owner the ability to keep a close eye on the health of the platform. It is always better to be able to notify your user base of an outage situation than experience them notifying you.

On receiving a system health alert, the platform team can understand the level of outage and notify senior management accordingly, but we suggest that the decision to failover requires human confirmation.

9.2 Standard operating procedure (SOP) for failover

Automate the actual failover process as much as possible to ensure you execute steps in the correct order once you confirm that you require failover.

9.2.1 Recommended failover steps

1. **Declare outage** - Notify end users and stakeholders.
2. **Contact HashiCorp Support** - Even if you have the situation under control, we recommend alerting us to the situation.
3. **Start failover** - Depending on outage scope.
    - If primary compute is accessible, drain Terraform Enterprise workloads and scale down to zero.
    - If compute is inaccessible (for example, regional outage), skip this step.
4. **Failover actions** (can run in parallel).

    - **DNS** - Update global DNS to point to the failover load balancer.
    - **PostgreSQL** - Promote the failover PostgreSQL from a read replica to read-write. **Only do this after you have fully confirmed that the compute layer in the primary region cannot write to the primary region database instance, to avoid split-brain.**

5. **Terraform Enterprise startup** - Launch a single Terraform Enterprise instance in the failover region (Docker, Podman, Nomad, Kubernetes, OpenShift).
6. **Validate health** - Ensure Terraform Enterprise is operational and accessible. If the cluster is unhealthy, refer to the restore page of this HVD for further assistance.
7. **Scale out** - Once stable, add additional Terraform Enterprise instances as needed.

9.3 Standard operating procedure (SOP) after failover

When the primary region recovers from its outage:

- **S3-compatible storage** - Ensure the failover region's bucket is replicating data back to the primary region.
- **PostgreSQL** - If self-hosted, you must reinitialize the primary region's PostgreSQL instance so it receives replicated data from the failover region.
    - This may not be applicable, if leveraging a cloud service.
    - Ensure the database replicas are up-to-date, because the intention is to promote the replica in the primary region to read-write after failing back.

9.4 Standard operating procedure (SOP) for failback

When the primary region is ready to return to normalized operations, follow these steps:

1. **Schedule failback** - Notify end users and stakeholders of a maintenance window for failback.
2. **Drain workloads** - Scale down Terraform Enterprise workloads to zero in the secondary region.
3. **Failback infrastructure**
    - **S3-compatible storage** - Ensure replication is complete from failover to primary, and no additional objects need replication.
    - **PostgreSQL** - Ensure replication is complete from failover to primary. Demote the PostgreSQL instance in the secondary (failover) region to read-only, then promote Post-

greSQL instance in the primary region to read-write. Double check that you only have one read-write database replica.
- **DNS** - Update DNS to point back to the primary region.

4. **Launch Terraform Enterprise in primary region** - Start a single instance and verify health.
5. **Scale out** - Once stable, add additional Terraform Enterprise instances as needed.
6. **Post-failback verification**
    - Ensure that the Terraform Enterprise application is down in the secondary region.
    - If applicable, ensure you reinitialize failover PostgreSQL (read-only) and ready for future replication.
    - Confirm S3-compatible storage replication from primary to failover is operational.

> **Note**
>
> The decision to fail over from a primary to a secondary region follows official notice from your designated internal executive production staff of an outage of sufficient magnitude that application failover is unavoidable. If you automate both outage detection and automatic failover, it is possible that your Terraform Enterprise instance may fail over due to a transient network issue which is highly undesirable to the risk of data loss during failover.

9.5 Additional information

This section provides links and further details useful when following the SOPs in this section.

The sections below cover only public cloud-related services because, for example, confirmation of the status of replication between datacenter instances of an S3-compatible object store depend on the technology employed. For both object store and database implementations supporting your private cloud Terraform Enterprise deployment, consult the operations team owning the storage platform for more details.

9.5.1 Object store

Select the cloud service provider tab below regarding more object store-related failover information. None of the commands in the tabs below use exposed, dedicated "last write" APIs. They all work by listing objects and sorting by the LastModified/timestamp metadata. For large buckets with millions of objects, this can be slow. In scaled cases, consider enabling access logging instead.

See the *Terraform Installation Guide* Multiple Regions page for more information. The commands below aim to widen your toolkit for understanding more about your failover scenario.

AWS

Confirm the last modified object in an S3 bucket (for example the main object store bucket in the secondary region) by using this command:`bash aws s3api list-objects-v2 \ --bucket YOUR_BUCKET_NAME \ --query 'sort_by(Contents, &LastModified) [-1].{Key: Key, LastModified: LastModified}'\ --output json`* Click here for information on confirming the status of object store replication between regions on AWS.

Azure

Confirm the last modified object in an Azure blob store (for example the main object store storage account in the secondary region) by using this command:`bash az storage blob list \ --account-name YOUR_ACCOUNT_NAME \ --container-name YOUR_CONTAINER_NAME \ --query "sort_by([].{Name:name, LastModified: properties.lastModified}, &LastModified)[-1]"\ --output json`* Click here for information on confirming the status of object store replication between regions on Azure.

GCP

Confirm the last modified object in a GCS bucket (for example the main object store bucket in the secondary region) by using this command:`bash gcloud storage ls --recursive --long gs://YOUR_BUCKET_NAME/ \ | grep -v "^TOTAL:" \ | sort -k2 -r \ | head -1`* Click here for information on confirming the status of object store replication between regions on GCP.

9.5.2 Database

Only AWS provides information on the status of cross-region PostgreSQL database replication from logs. Use the following command against the read replica in the secondary region of any cloud to ascertain the last successful WAL write that occurred.

```
psql -U postgres -d tfe -c "SELECT pg_current_wal_lsn();"
```

where `tfe` is the name of your Terraform Enterprise PostgreSQL database.

If the region with the primary Terraform Enterprise implementation fails and is offline long enough to start the failover recovery, follow the steps below.

1. Notify your user base of the outage.
2. Determine the outage time (`T0`).
3. Check to find if there are any pending objects that did not replicate to the *secondary* region. This process depends on your cloud provider, but recommendations regarding this are in the *Terraform Installation Guide* Multiple Regions page.
4. The preceding link includes links to cloud-specific metric information on replication lag. In addition, run the following script to find the replication lag time on the *secondary* PostgreSQL database by connecting directly to the secondary read replica, first updating the `psql*` environment variables to instantiate your database details.

> **Note**
>
> You must execute the following `psql` command inside your private subnet (either from a VM inside the VPC/VNet or use an equivalently secure connection).

```
export psqlHostSecondary="yourSecondaryPostgreSQL.example.com"
export psqlDatabase="tfe_primary"
export psqlUsername="psqladmin"
export psqlPassword="***"

PGPASSWORD=${psqlPassword} \
psql --host    ${psqlHostSecondary} \
    --dbname   ${psqlDatabase}      \
    --username ${psqlUsername}      \
    --command  "SELECT extract(epoch from now() - pg_last_xact_replay_timestamp()) AS
        slave_lag"
```

This script returns the number of seconds since the last update. Monitor this until it resets.

1. Promote the read replica in the *secondary* region to read-write, and wait for it to come up.
2. Turn on the *secondary* VM scaling group by scaling from zero nodes to > `0`.
3. Amend your DNS setup. Not every cloud supports global DNS at this time, so the approach differs between cloud vendors and will be different on private cloud `external` operational mode setups.
4. Notify your user base when you are back online.

Links for promoting public cloud PostgreSQL secondary region read replicas to read-write:

- **AWS**: Promoting a read replica to a DB cluster for Aurora (document cites MySQL but applies to PostgreSQL)
- **Azure**: Promote an Azure Database for PostgreSQL read replica
- **GCP**: Promote replicas for regional migration or disaster recovery
- **VMware**: Confer with your DBA team. VMware promotes use of `pg_auto_failover` between leader/follower nodes. To circumvent failures when you lose an entire vSAN cluster, VMware recommends vSphere Replication

10 Support readiness

To ensure smooth communication and efficient support, follow these guidelines.

- Each team member has an account created through our support portal.
- Configure each team member permitted to create support tickets on behalf of your company as an Authorized Technical Contact in the support portal. Please provide a list of applicable team members to your company's assigned Solutions Engineer so that they can configure this.
- Have your team familiarize themselves with our documentation on how to open a support ticket with HashiCorp Support and generate/upload a support bundle, whenever applicable.
- Be aware of the support plan level your company has purchased to manage expectations of response times. Also, understand the definitions for each severity level when opening a support ticket through our documentation.

10.1 Capturing command-line trace logs

For issues related to Terraform command-line behavior — whether running locally or in a command-line-driven workflow — collect a trace log before opening a support ticket. Providing trace output up front reduces time-to-triage and helps HashiCorp Support identify the root cause without additional back-and-forth. See the HashiCorp Support KB on enabling trace-level logs for additional context.

10.1.1 Enable trace logging

Set `TF_LOG` and, optionally, `TF_LOG_PATH` before running the affected Terraform command:

```
export TF_LOG=TRACE
export TF_LOG_PATH=./terraform.log
terraform plan
```

Setting `TF_LOG_PATH` directs Terraform to append all log output to the specified file. You must also set `TF_LOG` — `TF_LOG_PATH` alone does not activate logging.

To set `TF_LOG` for a single command without modifying your shell session permanently, use an inline variable assignment and pipe all output to a file:

```
TF_LOG=TRACE terraform plan -no-color 2>&1 | tee plan.log
```

Use `-no-color` to keep log output readable and safe for upload to a support ticket.

10.1.2 Narrow log scope

To limit verbosity to a specific subsystem, use `TF_LOG_CORE` or `TF_LOG_PROVIDER` in place of, or alongside, `TF_LOG`. Both accept the same level values (`TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR`):

- Use `TF_LOG_CORE=TRACE` when the issue appears to originate in the Terraform planning or state layer.
- Use `TF_LOG_PROVIDER=TRACE` when the issue appears to originate in provider calls or resource lifecycle operations.

10.1.3 Clean up after troubleshooting

Unset the log variables after each troubleshooting session to avoid accumulating verbose output in subsequent runs:

```
unset TF_LOG TF_LOG_PATH TF_LOG_CORE TF_LOG_PROVIDER
```

For full details on available log levels and configuration, refer to the Terraform debugging documentation and the environment variables reference.

Note

Trace logs capture Terraform command-line behavior and are appropriate for issues seen in local runs and command-line-driven workflows. For Terraform Enterprise *platform* issues — such as worker failures, application errors, or storage layer problems — use the diagnostics and support bundle workflow covered in the support ticket guidance in the preceding section.

11 Self-hosted agents

Note

This page covers Terraform Enterprise self-hosted agent operations. HCP Terraform is operated by HashiCorp; HCP-only request forwarding guidance is marked inline where relevant.

Self-hosted HCP Terraform agents are lightweight agents that execute Terraform runs on behalf of HCP Terraform or Terraform Enterprise and report back the results.

There are three reasons for using self-hosted agents:

1. To increase the number of concurrent Terraform runs.
2. To access resources in restricted networks (where HCP Terraform or Terraform Enterprise do not have direct access).
3. To provide additional tools to the Terraform runtime via hooks.

Tip

For HCP Terraform customers we recommend using self-hosted agents only if they need #2 and #3 in the list above. If those are not requirements then we recommend that HCP Terraform users leverage the native built in workers available within HCP Terraform.

11.1 Specific to Terraform Enterprise

Deploying HCP Terraform agents and registering them with your Terraform Enterprise instance is one strategy to scale the capacity of your installation, with the other strategies being:

- Migrating to an active/active operational mode and increasing the number of Terraform Enterprise nodes.

- Add more resources to the virtual machines hosting the Terraform Enterprise nodes, and increasing the capacity configuration parameters (`TFE_CAPACITY_CONCURRENCY`, `TFE_CAPACITY_CPU`, `TFE_CAPACITY_MEMORY`).

If you are facing capacity issues and have reached the limits of the two alternate scaling strategies, deploying HCP Terraform agents is your next logical step.

Once deployed and running, the HCP Terraform agent instance will register itself with Terraform Enterprise and start processing runs. When the agent is shut down, it will deregister itself from Terraform Enterprise and reduce the capacity of the instance to process Terraform runs.

11.1.1 Access resources in restricted networks

Some resources that you need to configure with Terraform are sometimes located in isolated networks for security reasons, or because they are in private data centers. Deploying an HCP Terraform agent in the isolated network and providing the agent outbound connectivity to HCP Terraform will solve this challenge.

The tutorial Manage Private Environments with HCP Terraform Agents illustrates how this capability works.

11.1.2 Customizing the tools available and support custom logic

In some cases, you may need additional tools available to the Terraform runtime, or you may need to execute custom processes during strategic points of a Terraform run (see the documentation on HCP Terraform agent hooks). If you are in either one of those situations, then adopting HCP Terraform agents is necessary.

> Note
>
> Terraform Enterprise supports defining a custom worker image, which will allow you to get those benefits without having to deploy and manage self-hosted HCP Terraform agents.

11.2 Implementation guidance

There are two primary ways to deploy self-hosted agents. One is to run the agent on VMs managed by you (e.g EC2 instance), alternatively we provide a Kubernetes operator that can automate

deployment of the agents. We recommend that customers who are skilled in Kubernetes choose the K8S operator since it provides additional capabilities such as autoscaling.

For more details please refer to the Agent documentation.

11.2.1 Building a custom agent image

Since one of the benefits of the self-hosted image is the ability to add custompre/post hooks and other dependencies, having a pipeline to automate this is recommended.

New HCP Terraform agent versions are released regularly and you should be keeping up to date with those releases. We recommend automating image building using your CI/CD tool of choice to lower the burden of keeping up with the releases.

To get notified of new releases, you may use the following sources of information:

- Our documentation site (HCP Terraform Agent Changelog)
- The HashiCorp Releases API (Announcing the HashiCorp Releases API)

If you decide to deploy the HCP Terraform agent on a virtual machine, we recommend using Packer to build the images to deploy and the HCP Packer integration with HCP Terraform to deploy the HCP Terraform agent instances.

11.2.2 Deployment

Whether you have selected the virtual machine or container deployment mode, you must automate the deployment of HCP Terraform agent instances. There are two major steps in agent deployment:

1. Create the agent pool and generate the registration token.
2. Deploy the agent instance(s) in an agent pool.

Create Agent pools

HCP Terraform organizes agents into pools. Each workspace can specify which agent pool should run its workloads. If you've defined the default execution mode for the organization to be "Agent", then you can assign a default agent pool for workspaces.

The creation of agent pools can be done using the WebUI, the API (see the documentation for HCP Terraform, and Terraform Enterprise), or the Terraform TFE provider (tfe_agent_pool, tfe_agent_pool_allowed_workspaces, tfe_agent_token). We recommend using the Terraform TFE provider to configure your HCP Terraform or Terraform Enterprise instance. Review the applicable security controls for the agent tokens as you will likely need to implement a token rotation schedule.

Design

Your agent pool strategy will depend on a number of factors, the most common ones being:

- The limits of the product (HCP Terraform or Terraform Enterprise)
- The reason why you're implementing HCP Terraform agents.
- The cloud service providers you need to deploy infrastructure to.
- Your organization's network segmentation strategy.

HCP Terraform and Terraform Enterprise have different constraints regarding the number of HCP Terraform agents running concurrently:

- HCP Terraform enforces run concurrency and self-hosted agent limits that very based on the chosen tier.
- Terraform Enterprise imposes no limit on the number of agents running concurrently.

When designing your agent pool setup, this aspect of the product must be considered, as well as the following:

- Cloud provider isolation: Dedicated agent pool(s) per cloud provider offer benefits such as a streamlined credential management process and, access to provider-specific resources that enhance security and efficiency.
- Environment isolation: Dedicated agent pools based on environment types prevent accidental changes across development, staging, and production, facilitating focused testing and validation efforts.
- Operations isolation: For highly privileged operations, separate pools enable granular permission management and efficient audit trail, mitigating security risks and ensuring compliance.

Tip

HashiCorp recommends dedicating an agent pool for every secured environment, such as on-premises data centers with strict controls, or cloud environments using GCP's VPC Service Controls (or the AWS or Azure equivalent). This strategy enhances security and scalability, reduces the blast radius in case of a security incident, and aligns with the security team's preferences for controls tailored for each environment.

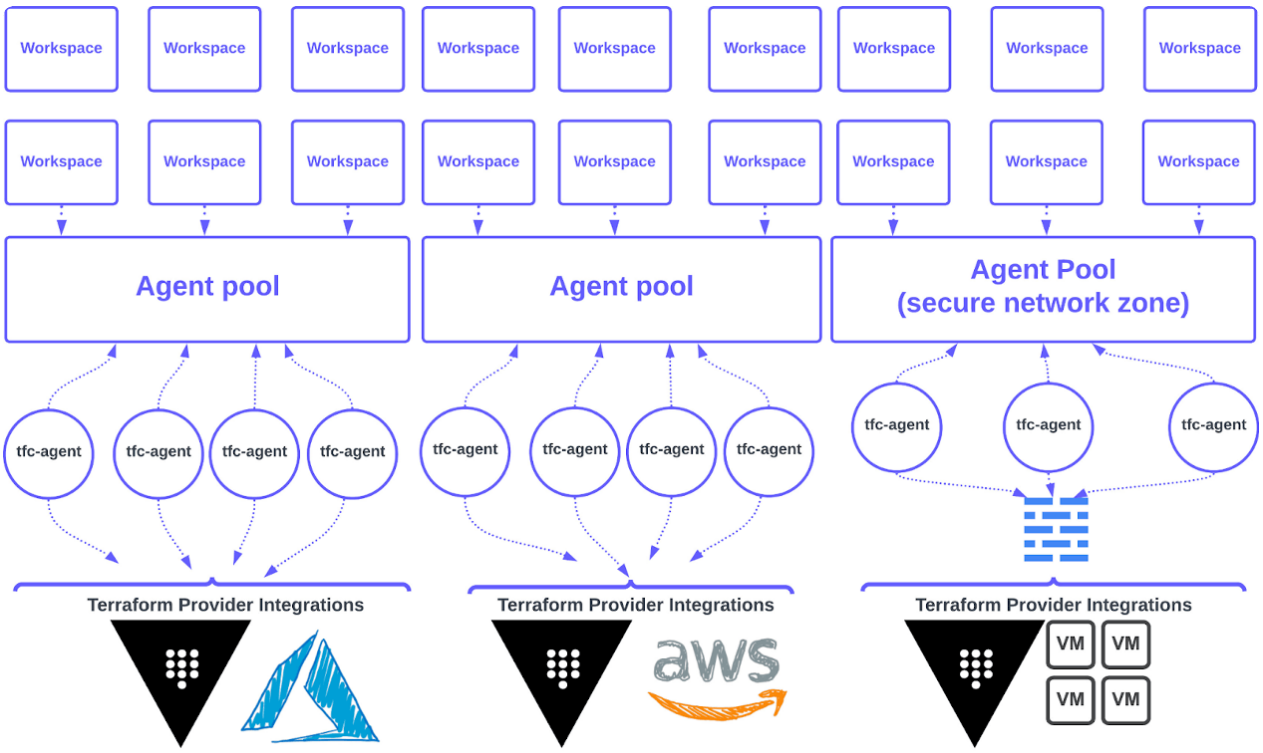


Figure 14: Naming Convention

Naming convention

Establishing a consistent and intuitive naming convention for your agent pools is essential for maintaining clarity, organization, and ease of management across your infrastructure. Agent pool names must be unique and will be used by workspace administrators when linking workspaces to a specific agent pool.

In the remainder of this section, we're covering a recommended naming convention pattern that incorporates various parameters such as team names, environment names, and cloud provider identifiers based on some of the factors discussed above.

The recommended agent pool naming convention is:

```
{environment}-{cloud service provider}-agentpool
```

Where:

- `{environment}` is the name or abbreviation of the environment associated with the agent pool (for example, `dev`, `staging`, `prod`). This can be a prefix for each associated network perimeter.
- `{cloud service provider}` is a short identifier for the cloud provider associated with the agent pool (for example `aws`, `azure`, `gcp`).

Some examples of agent pool names following the recommended naming convention:

- `dev-aws-agentpool`
- `staging-azure-agentpool`
- `prod-gcp-agentpool`

Additional considerations are:

- **Abbreviations**. Consider using standardized abbreviations for team names, environment names, and cloud provider names to keep the naming format concise and consistent.
- **Delimiters**. Use hyphens or underscores as delimiters between different components of the name for readability and consistency.
- **Lowercase**. Keep the entire name in lowercase for uniformity and ease of use.
- **Avoid special characters**. Avoid using special characters or spaces in the names to ensure compatibility with various systems and automation tools.

Documentation and communication:

- Document the naming convention standards in your organization's documentation or internal wiki to ensure consistency and provide guidance to team members.
- Communicate the naming convention to all relevant stakeholders, including teams responsible for provisioning, managing, and monitoring agent pools.

By following a clear and standardized naming convention like the one outlined above, you can streamline the identification, management, and communication of agent pools across your organization's multi-cloud environments while promoting consistency and clarity. Adjust the convention as needed to align with your organization's specific requirements, preferences, and existing naming conventions.

Deploying agents into agent pool

The options to deploy HCP Terraform agent instances depend on the deployment mode (virtual machine or container).

Virtual machines

If you are deploying HCP Terraform agents on virtual machines, we recommend using HCP Terraform or Terraform Enterprise to manage the virtual machines, leveraging the integration with HCP Packer (see Get started with HCP Packer). We also recommend using an automated scaling solution such as an autoscaling group (AWS), a managed instance group (GCP) or a scale set (Azure). The rolling upgrade feature of those automated scaling solutions helps simplify management, even when there is only a single HCP Terraform agent instance running.

When making scaling decisions for virtual machine deployments, you can leverage metrics produced by the agents as well as API endpoints.

The following HCP Terraform agent metrics should be used to make scaling decisions:

- `tfc-agent.core.status.busy` which is the number of agents in busy status at a particular point in time.
- `tfc-agent.core.status.idle` which is the number of agents in idle status at a particular point in time.

The following API endpoints are also a source of valuable information to automate scaling decisions:

- List all the agent pools configured,
- List all workspaces using an agent pool,
- List of all pending runs,
- List all runs in a workspace.

> **Note**
>
> Dynamically scaling the number of agents is not a trivial task and is often risky, especially when trying to reduce the number of agents that are currently running. If you have identified that you need to dynamically scale your agents, you should consider using the HCP Terraform Operator for Kubernetes mentioned in the next section.

Containers

If you are deploying HCP Terraform agents on containers, we recommend the following options:

- Deploy on a managed Kubernetes service using HCP Terraform Operator for Kubernetes.
- Deploy on a managed container orchestration tool such as ECS Fargate on AWS, Cloud Run on GCP, or ACI on Azure.
- Deploy on a managed OpenShift offering (for example ROSA on AWS or ARO on Azure).

A deployment on an orchestrator supported by the HCP Terraform Operator for Kubernetes is the option recommended by HashiCorp as it will solve both the deployment and dynamic scaling requirements.

11.2.3 Agent authentication

Agents and workspaces need to authenticate to the respective Cloud providers and tools to manage the lifecycle of the resources within those platforms. There are multiple ways to manage agent authentication to these platforms:

1. Dynamic provider credentials: With the OIDC protocol, a trust relationship between the Cloud Provider or external tool and HCP Terraform or Terraform Enterprise can be set up to generate temporary credentials used to authenticate to the respective platform. Dynamic credentials permissions can be scoped based on the project, workspace and Terraform operation (plan or apply), making it the most flexible option.
2. Cloud provider IAM roles attached to agents: Leverage the IAM roles attached to the underlying compute resources of the HCP Terraform agents to authenticate with the respective cloud provider.
3. Vault provider for Terraform: Retrieve credentials from Vault (static or dynamic credentials) and use those credentials for authentication.

4. Variables: Set static credentials as sensitive variables in workspaces. This approach can be extended and applied to multiple workspaces, projects or an organization as a whole by using variable sets. If necessary, use priority variable sets to prevent variables from more specific scopes from overwriting values set at less specific scope (for example, global or project scope).

Tip

HashiCorp recommends using the dynamic provider credentials (option #1) or cloud provider IAM roles (option #2) for authentication if those methods are supported by the managed platform. As an example, dynamic provider credentials are available for Hashicorp Vault, AWS, GCP, Azure, and Kubernetes. These two methods eliminate the need to manually manage and rotate credentials across your HCP Terraform or Terraform Enterprise organization, improving your security posture.

11.2.4 Observability

HCP Terraform agents produce execution logs and it is important that you include them in the logs you already collect and analyze. Doing so will help ensure that you have a complete picture of the operations of Terraform Enterprise (or HCP Terraform) and can detect any issue or error that may arise.

By analyzing all the logs related to your Infrastructure as Code service together, you'll be better equipped to make informed decisions about optimizing and improving your Terraform Enterprise or HCP Terraform deployment. Always remember to stay vigilant and monitor your logs to ensure the systems run smoothly and securely.

The HCP Terraform agent also exposes telemetry data using the OpenTelemetry protocol. This feature allows you to use a standard OpenTelemetry collector to push the metrics to a central monitoring solution that supports the protocol. OpenTelemetry is supported by a wide range of monitoring solutions, such as: Prometheus, DataDog, New Relic.

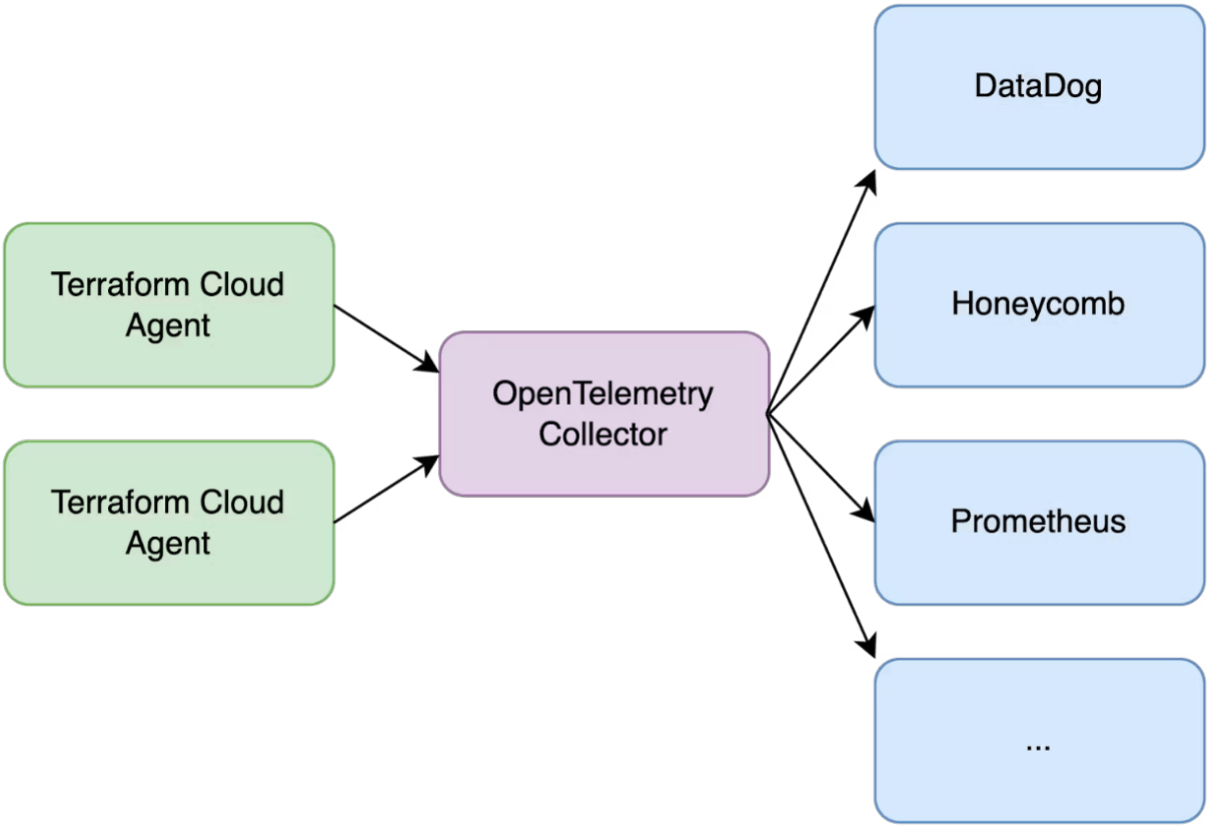


Figure 15: Self-hosted Agents Observability

To use telemetry data from your HCP Terraform agents, you need:

- To deploy and run OpenTelemetry collectors
- A central monitoring system that can integrate with OpenTelemetry collectors

You can find additional information by following those links:

- HCP Terraform agent telemetry
- HCP Terraform agent logs
- HCP Terraform agent metrics

Agent Configuration

To configure your agent to emit telemetry data, you must include the **-otlp-address** flag or **TFC_AGENT_OTLP_ADDRESS **environment variable. This should be set to the host:port address of an OpenTelemetry collector. This address should be a GRPC server running an OLTP collector.

Optionally, you can pass the **-otlp-cert-file** or **TFC_AGENT_OTLP_CERT_FILE**. The agent will use a certificate at the path supplied to encrypt the client connection to the OpenTelemetry collector. When omitted, client connections are not secured.

11.2.5 Considerations for Terraform Enterprise

External access to metadata endpoints

To verify signed JWTs, cloud platforms must have inbound network access (via the same FQDN configured within Terraform Enterprise) to the following static OIDC metadata endpoints within Terraform Enterprise:

Endpoint	Description
`/.well-known/openid-configuration`	Standard OIDC metadata
`/.well-known/jwks`	Public key(s) used by cloud platforms to verify the authenticity of tokens claiming to originate from the Terraform Enterprise instance.

External Vault policy

If you are using an external Vault instance you must ensure that your Vault instance has the correct policies set up as detailed in the External Vault Requirements for Terraform Enterprise documentation.

11.3 Request Forwarding (HCP Terraform Only - Premium SKU)

Details can be found here https://developer.hashicorp.com/terraform/cloud-docs/agents/request-forwarding

Request forwarding is an additional function that is used by HCP Terraform to access services in private networks, such as:

- A self-hosted version control system
- A service running a custom run task
- A service used when executing a Sentinel or OPA policy check

There are legitimate reasons for an organization to keep these services unreachable from the public internet, but this poses an issue if they are required by HCP Terraform. Request forwarding solves the problem of allowing HCP Terraform to access those systems without exposing them to the internet.

To take advantage of this feature you must:

- Deploy the agent into a private network where they have access to the target service, and are allowed to connect to HCP Terraform via an outbound-only connection.
- Enable the feature on a per-agent basis using a command line flag or an environment variable.

Once you have enabled the feature, the agent will register with HCP Terraform via outbound-only connections. HCP Terraform holds these connections open and uses them to transmit requests and responses between the target service and HCP Terraform.

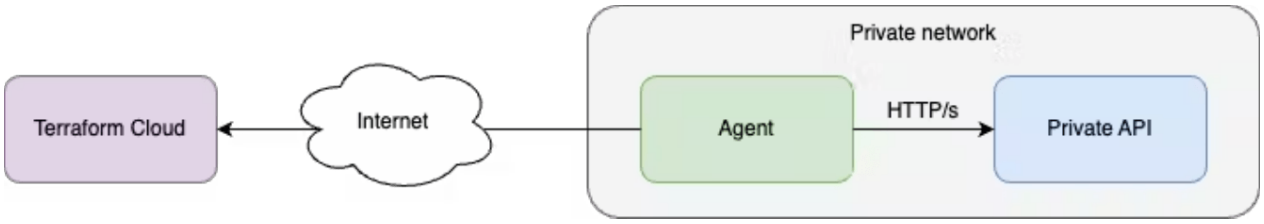


Figure 16: Agent Connections

Once the connection is established, HCP Terraform can leverage it to access a self-hosted VCS, as illustrated below:

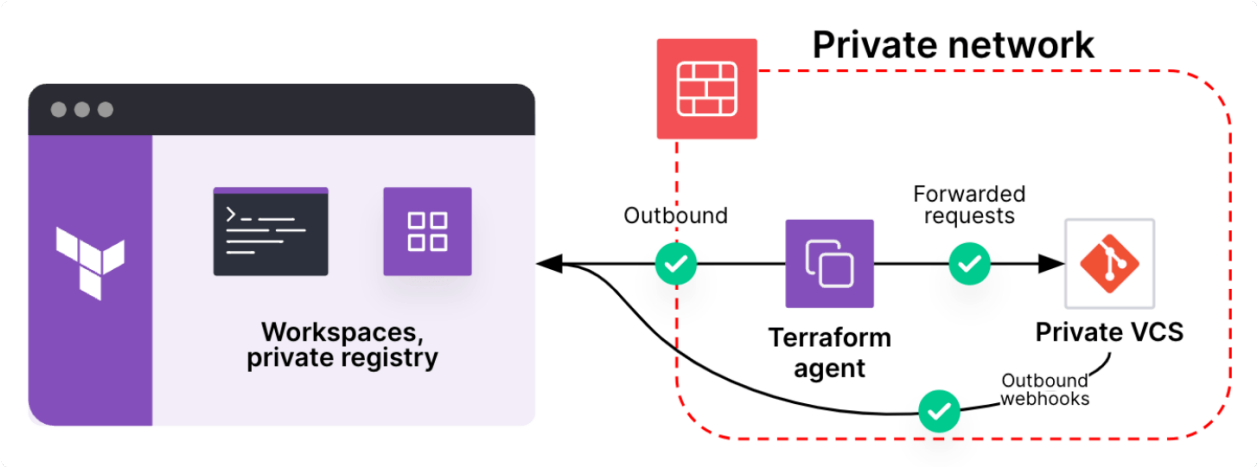


Figure 17: Private VCS

HCP Terraform an also use agent request forwarding to execute Sentinel or OPA policy checks on the agent (by default they run on the HCP Terraform infrastructure) as illustrated below:

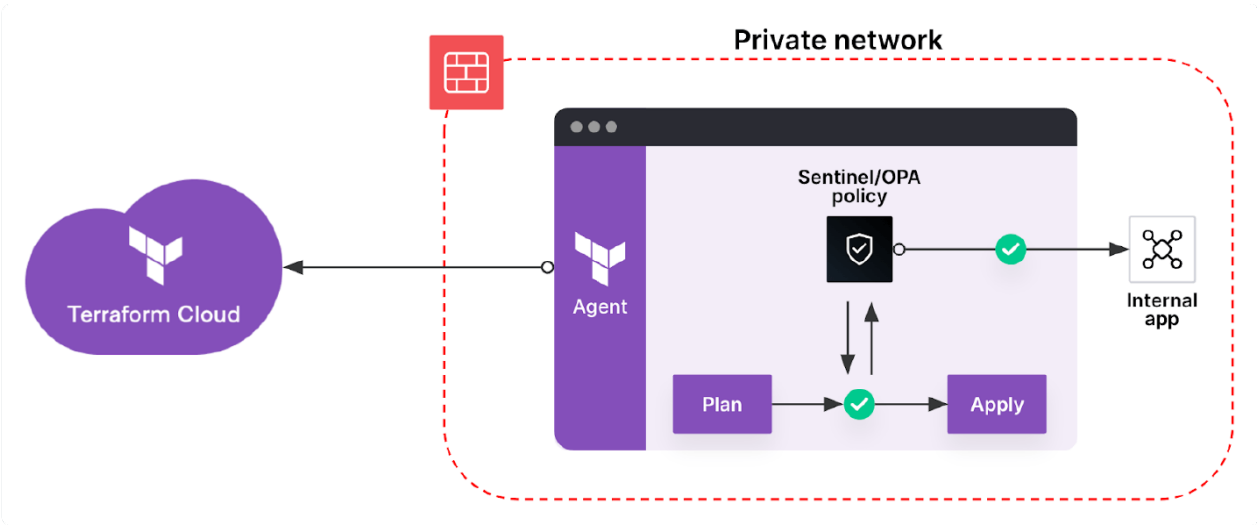


Figure 18: Sentinel Policies

If you choose to implement this feature, we recommend reviewing the documentation below:

- Requirements and limitations
- Security considerations

- Performance and scaling

11.4 Network requirements

Note

This section is covered to some extent in /validated-designs/terraform/administration-guide/identity-and-access-management#hcp-terraform-agents. Following contents are an extension to the same
It seems the previous link has been removed from the updated version of the HVD published in our website. The network requirements can now be found https://developer.hashicorp.com/terraform/cloud-docs/agents/requirements#networking-requirements

Note

Archivist whitelisting is required only for HCP Terraform

When Terraform Enterprise servers or HCP Terraform agents are deployed in a completely air-gapped environment where outbound access to **releases.hashicorp.com **and **registry.terraform.io **is restricted, the Terraform binary and the provider installations need to be managed in an alternative way. Additionally, your organization would like to have security scans in place for each Terraform binary and provider installation used in your workflows and also control what providers can be used with their version restrictions. These requirements can be achieved using the following methods:

1. **Network mirror: **The provider network mirror protocol is an optional protocol that you can implement to provide an alternative installation source for Terraform providers, regardless of their origin registries.
2. **Private registry: **Private providers can be published to an organization's private registry and can be referenced during Terraform runs. This does not manage Terraform binaries.

   Detailed implementation steps can be found in the guide https://developer.hashicorp.com/terraform/cloud-docs/registry/publish-providers

In an air-gapped environment with restricted outbound connectivity, it is recommended to follow the network mirror approach for managing Terraform binaries and provider installations. Network

mirrors allow you to manage the terraform binaries and provider installations separately in a private environment decoupled from each other.

Terraform uses network mirrors for **provider installations **only if you activate them explicitly in the CLI configuration's **provider_installation** block. When enabled, a network mirror can serve providers belonging to any registry hostname, allowing an organization to serve all of the Terraform providers they intend to use from an internal server, rather than from each provider's origin registry.

Terraform binary versions in Terraform Enterprise pointing to the Network Mirror source can be managed using the Admin API for Terraform Versions or by manually managing them through the Terraform Enterprise Admin UI.

Terraform binaries and providers referred via a network mirror must be stored in a remote File server discoverable by Terraform Enterprise. This **file server MUST be HTTPS! (Can be a cloud provider object storage eg: S3, GCS, Blob Storage URLs)**

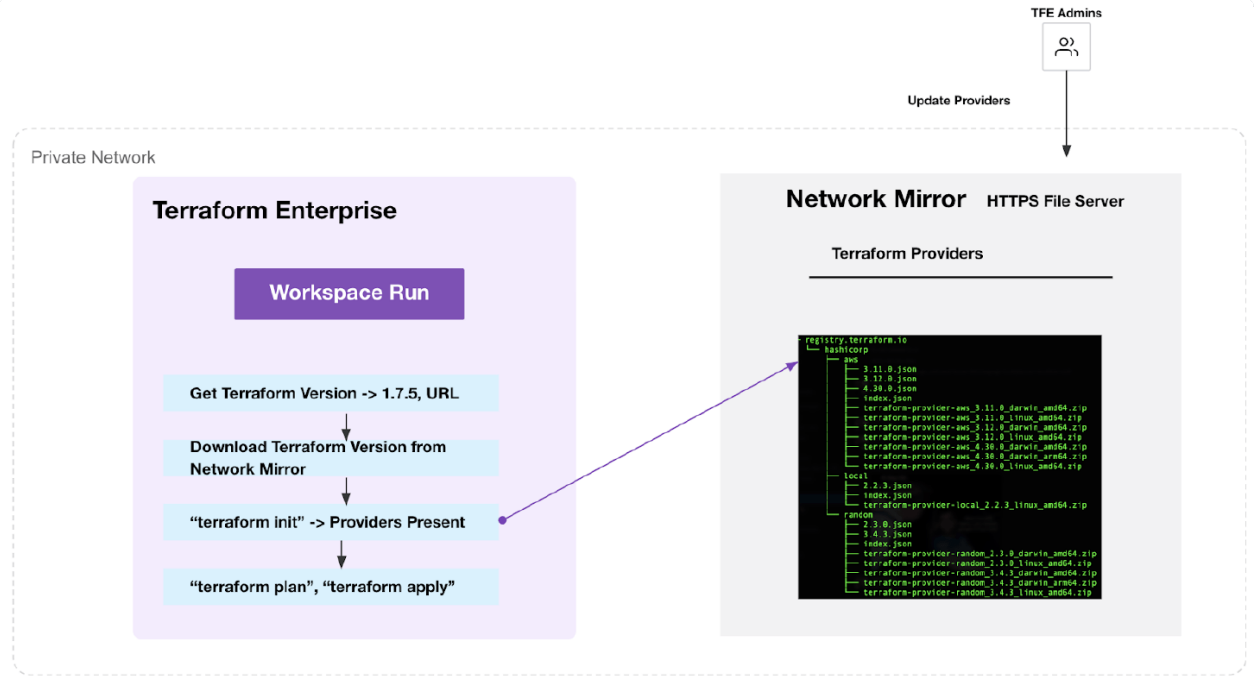


Figure 19: Selfhosted Agents

A summary of the steps are as follows:

1. Download/create a mirror repository for the Terraform binaries by downloading the required

versions of the Terraform binary from the official release repository
2. Download/create mirror repository by using "terraform providers mirror" command.
3. Sync/copy mirror directories of the binary and the providers to a file server.
4. Configure the Terraform run (CLI) config to use network mirror (Options)
    1. HCP Terraform agents with the network mirror config appended to the "$HOME/.terraformrc" file via a custom hook.
    2. Remote CLI users would modify their Terraform CLI config ("$HOME/.terraformrc") to include the provider_installation block
    3. Create a Custom HCP Terraform agent image with network mirror settings for every Terraform run
5. Add the Terraform version and its remote file server source via the Terraform Enterprise admin interface.

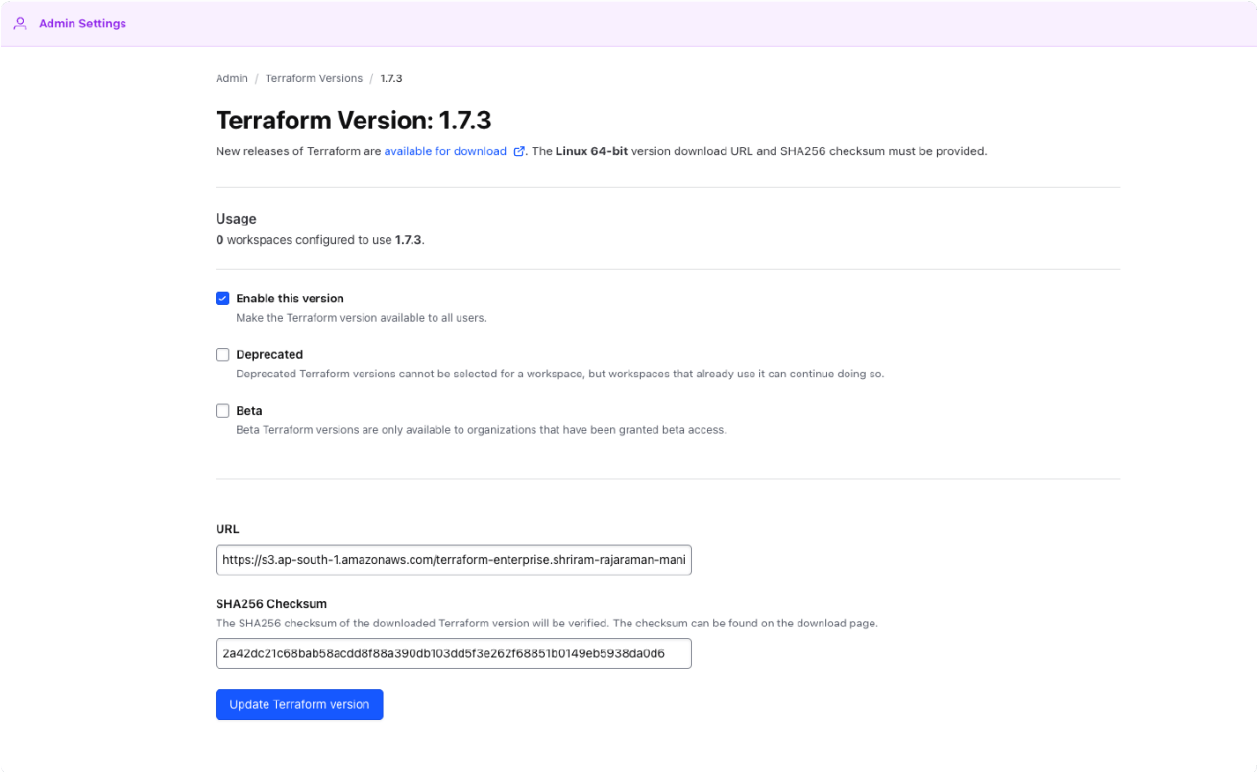


Figure 20: Terraform Admin

An automation process can be put in place to manage the various versions of the Terraform binaries

and providers referenced using the network mirror. This can be a CI pipeline that updates the latest versions of the binaries and providers and publishes them to the remote source of the network mirror periodically according to your organization's standards.

The Terraform CLI commands interact with the HashiCorp service Checkpoint to check for the availability of new versions and critical security bulletins about the current version. In an air-gapped environment where outbound access is restricted, this can be disabled using the environment variable **CHECKPOINT_DISABLE **and setting it to **true **or by adding the checkpoint setting **disable_checkpoint=true **in the **CLI Configuration file **used in the agent hooks.

12 Private registry management

> **Note**
>
> This page covers operator setup and management of the private registry. For how users discover and consume modules and providers, see the User Guide: Private Registry.

The Terraform private registry allows organizations to host and share their own Terraform modules and providers privately, limiting access to their own network or cloud environment. It can also serve as a gateway to whitelisted Terraform modules and providers sourced from the public registry. The Terraform private registry provides an alternative to the public Terraform Registry and enables organizations to manage and maintain their infrastructure as code more effectively. The two main methods for accessing the private registry are the web UI and the API

Feature	Public Registry	Private Registry
Access/Visibility	Anyone	Only authorized users in your organization
Content	All public modules and providers, TFC Run Tasks and Sentinel Policies.	Whitelisted public modules and providers, and your custom modules and providers.

The private registry has three main functions: sharing, discovery, and consumption of modules and providers. We'll cover those functions and more in this section.

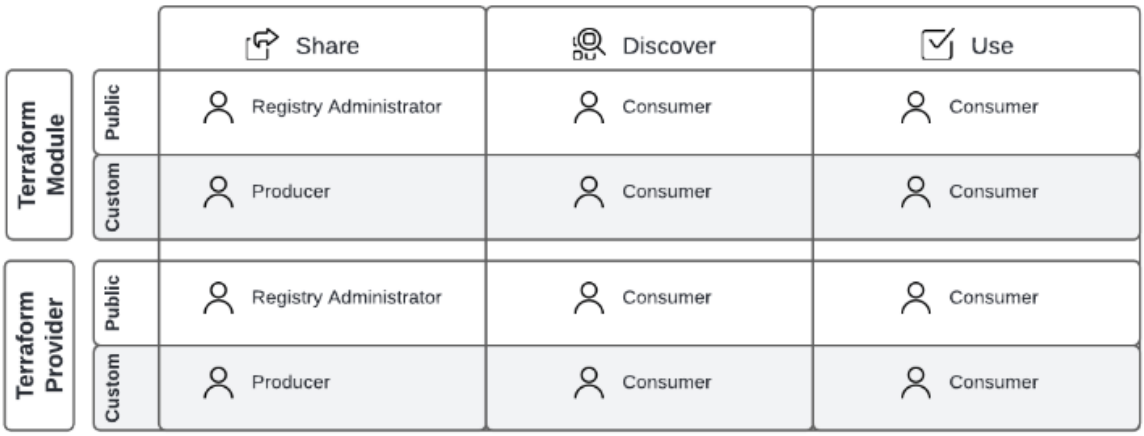


Figure 21: Private Registry main functions

12.1 Roles and responsibilities

When configuring and using the private registry, three roles must be considered:

1. Registry Administrator
2. Producer
3. Consumer

12.1.1 Registry Administrator

The Registry Administrator role is tasked with these responsibilities:

- Publish and delete modules in the organization's private registry, either from the public registry or from private sources.
- Publish and delete providers in the organization's private registry, either from the public registry or from private sources.

It is an organization-level permission in TFC/E and can be assigned to a specific team:

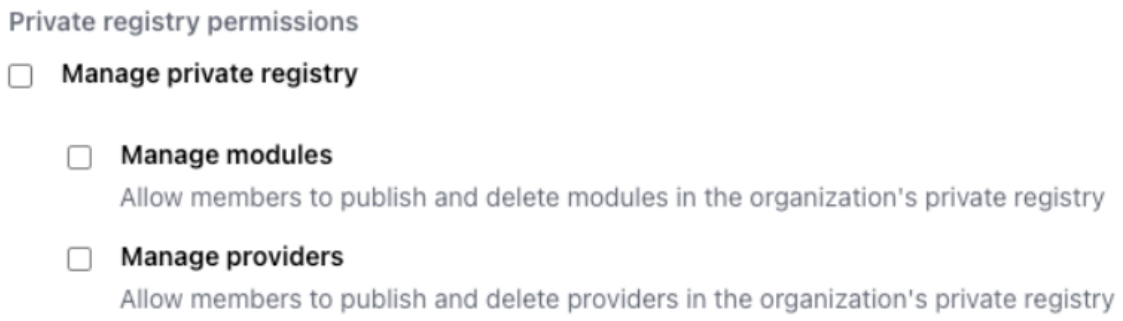


Figure 22: Private Registry Permissions

Examples of personas needing the Registry Administrator role in an organization:

- Platform team members.
- CI/CD pipelines automating the test and publishing process for custom modules or providers.

12.1.2 Producer

The Producer role is tasked with:

- Create and maintain the module or provider
- Publish a new release of the module or provider

The Producer needs "commit" access to the module or provider VCS repository, and the permissions of a Registry Administrator.

Examples of personas needing the Producer role in an organization:

- The Platform team members responsible for maintaining custom modules for your organization.
- The Service Owner team members responsible for maintaining custom modules for your organization.

HashiCorp recommends that the teams with the Producer role delegate the module release publishing activity to a CI/CD pipeline with the appropriate "Registry Administration" permission (either module or provider publishing). Those teams would not have "Registry Administration" permissions,

but the pipeline configuration would be configured with a Team API token with the "Registry Administration" permission. That API token has to be properly rotated or managed with a tool such as HashiCorp Vault and the Terraform Cloud secrets engine.

12.1.3 Consumer

The Consumer role is tasked with:

- Find providers and modules necessary to provision infrastructure.
- Use providers and modules in Terraform configuration.

To perform their role, the Consumer requires "commit" access to the VCS repository and write access to the HCP Terraform workspaces that use the Terraform configuration.

Example of personas needing the Consumer role in an organization:

- Members of application teams that are managing infrastructure using TFC/E using modules and providers.
- Members of the platform team using the TFE provider to manage the TFC/E configuration.

12.2 Sharing custom Terraform modules

Before modules and providers can be discovered and consumed from the private registry, they must first be shared.

Custom Terraform modules allow your organization to take advantage of the reusability aspect of Infrastructure as Code (IaC). They allow you to encapsulate common patterns into a parametrized Terraform unit of code and reuse those patterns across your IaC code base.

Early in your Terraform journey, your teams will discover infrastructure patterns that are optimal for your organization. Creating modules that implement those optimal infrastructure patterns and then sharing them with the rest of your IaC practitioners is a way to make those lessons learned easy to consume and address the skills gap.

We also recommend leveraging custom Terraform modules for additional benefits, including the implementation of the following:

- Your naming convention.
- Your security controls.

- FinOps standards.

Before creating custom Terraform modules, it is essential to understand specific requirements:

- Custom Terraform module code must be hosted in a VCS repository.
- Terraform modules must follow the semantic versioning scheme.
- Follow the official module naming convention.
- Follow the standard module structure.
- Module publishing permissions.
- Configured VCS provider.

In the next subsections, we'll cover those requirements in more detail.

Host custom Terraform module code in a VCS

HashiCorp recommends hosting Terraform module code in a supported Version Control System (VCS) repository. This allows for version control, which enables teams to track changes to code over time and revert to previous versions if necessary. It also facilitates collaboration, as team members can work on the same codebase and seamlessly merge changes. Finally, it provides a single source of truth for the codebase, allowing you to cut a release and publish it in the private registry.

Use Semantic versioning

HCL is designed to work with the semantic versioning scheme, as exemplified by the use of version constraints in the `terraform` block for the terraform binary version, or the provider versions, and the `module` block to set constraints on module versions.

HashiCorp recommends using the semantic versioning scheme so that your Terraform practitioners can take advantage of version constraints in the code and the available module publishing automation.

Follow the official module naming convention

HashiCorp has defined an official module naming convention, which is expected by the public and private registry. The naming convention is composed of three parts separated by a hyphen (-) character: `terraform-<PROVIDER>-<NAME>`. Note that the `<NAME>` segment may contain additional hyphens. Here are two examples of valid module names:

- `terraform-google-vault`
- `terraform-aws-ec2-instance`

Follow the standard module structure

The private registry expects the module structure to follow a certain pattern. This allows the registry to inspect the module and:

- Generate documentation
- Track resource usage
- Parse examples
- Parse submodules (if any)
- Execute the module test suite (if any and configured)

This information is then available to the practitioner and helps with module discovery and usage.

Module publishing permissions

Only members of the "Owners" team or members of a team that has been granted the organization-level "manage private registry" permission can publish (or delete) modules.

HashiCorp recommends setting up a team in TFC/E to delegate private registry management and not rely on the "Owners" team. If you need to integrate TFC/E with a CI/CD pipeline that will drive module publishing, we recommend using a team API token to allow the CI/CD pipeline to publish the module.

Configured VCS provider

You must have configured a VCS provider for the VCS system hosting your Terraform modules and provided administrative access to the repositories hosting the custom Terraform modules.

The procedure for configuring the VCS provider varies depending on the supported VCS system. Please see the online documentation for more information.

12.2.1 Publishing a module in the private registry

Once your module is written and all the requirements are met (i.e., host the code in a VCS, use semantic versioning, use the standard module naming convention and structure, and set up per-

missions in TFC/E), you are ready to publish the module in the private registry.

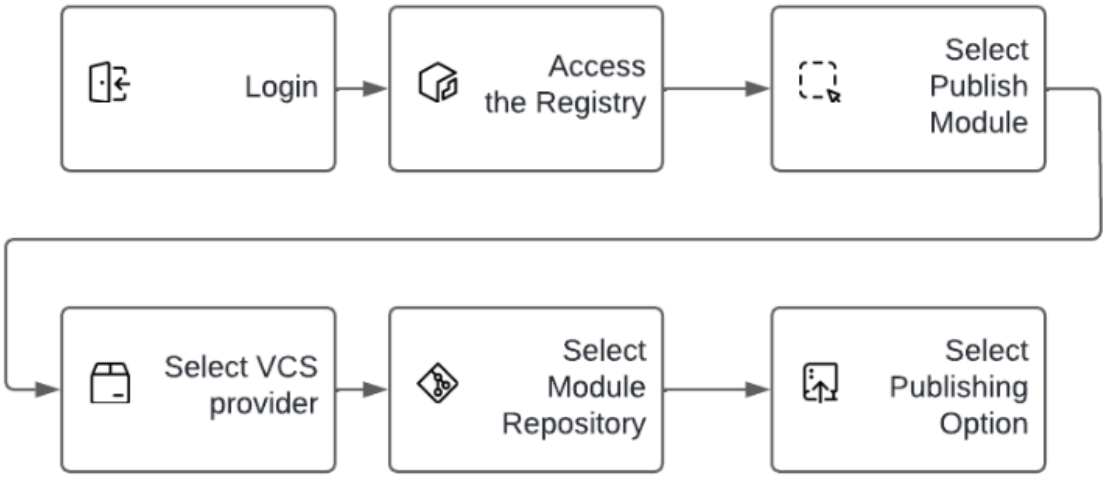


Figure 23: Publishing a module

You will be prompted to select either the Tag or Branch module publishing option. Here is what you need to know about each option:

Tag-based	Ideal for modules associated with release tags in your VCS repository. The registry will automatically detect and publish new versions based on these tags. If your VCS supports it, we recommend considering implementing tag protection rules to prevent a malicious user from changing the association of a tag with a commit.
Branch-based	Provides enhanced flexibility, permitting the selection of a particular branch in your VCS and the assignment of a distinct version number to the module. Currently, this mode is also required to make use of the integrated testing capabilities of the private registry.

Module publishing can also be done using the API and integrated into a CI/CD pipeline. See the documentation available online on our website.

Once a module has been created and published, you can switch to a module maintenance workflow, illustrated below.

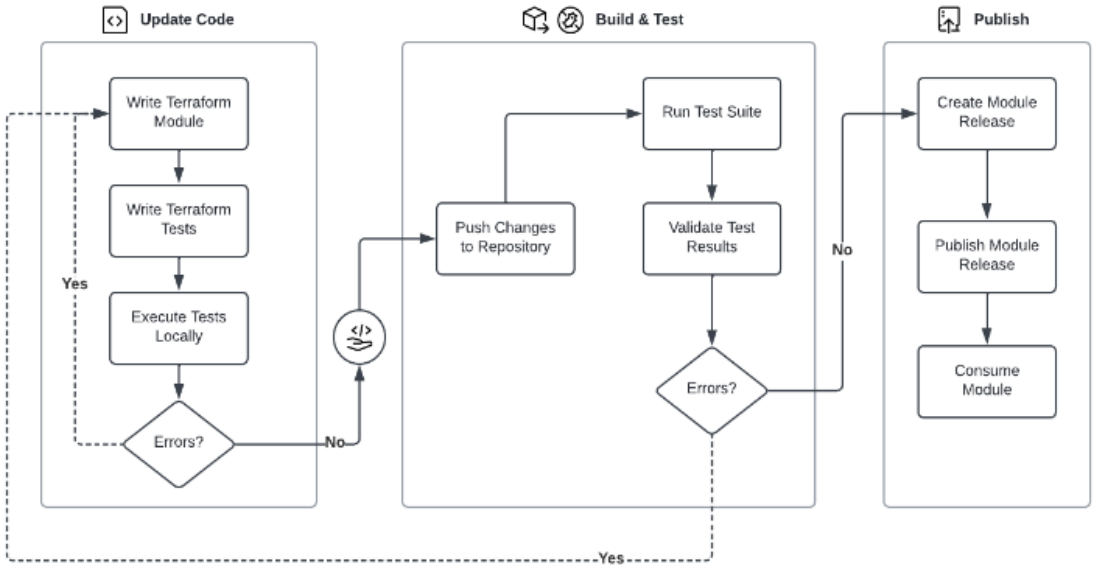


Figure 24: Module maintenance workflow

12.3 Sharing custom providers

There are many reasons why an organization may choose to develop and maintain custom providers that can be broken down into two categories:

1. Flexibility and customization
2. Efficiency and collaboration

Flexibility and customization

- **Extend Terraform capabilities:** With custom providers, organizations can create custom resources and data sources specific to their infrastructure or cloud platforms. This extends the reach of Terraform beyond standardized offerings, allowing for tailored automation and management.
- **Rapid prototyping and innovation:** Develop and test internal provider prototypes privately before exposing them to the broader community. This facilitates experimentation and innovation without the pressure of public scrutiny.

Efficiency and Collaboration

- **Reduced dependency fatigue:** Avoid relying on externally sourced providers with unknown provenance or compatibility issues. Custom providers enable the consolidation of internal modules, enhancing reliability and simplifying dependency management.
- **Centralized governance:** Facilitate centralized governance and approval processes for custom providers and modules. This ensures consistency and minimizes risks associated with uncontrolled infrastructure configuration.

Foster collaboration and knowledge sharing within your organization by making internal best practices and custom providers readily accessible to team members in order to standardize the approach.

Custom providers enable development and management of internal-only resources making them a valuable asset for managing complex and evolving on-premises infrastructure needs.

Remember that the decision to utilize custom providers should be based on your specific requirements and security considerations. While they offer numerous benefits, careful planning and management are crucial to ensure their effective implementation and secure operation.

12.3.1 Publishing a custom provider to your private registry

Prerequisites

Prior to creating custom Terraform modules, it's important to be aware of certain requirements:

- Custom Terraform provider code must be hosted in a VCS repository.
- Terraform providers must follow the semantic versioning scheme. Terraform supports specifying version constraints for providers as long as the provider uses semantic versioning.
- Terraform providers must be written using the Go programming language.
- You need a GPG key: Provider releases must be signed with a valid GPG key.
- Provider publishing permissions (see Registry Administrator).

Publishing steps

Publishing a custom provider to the private registry involves several key steps outlined in this section.

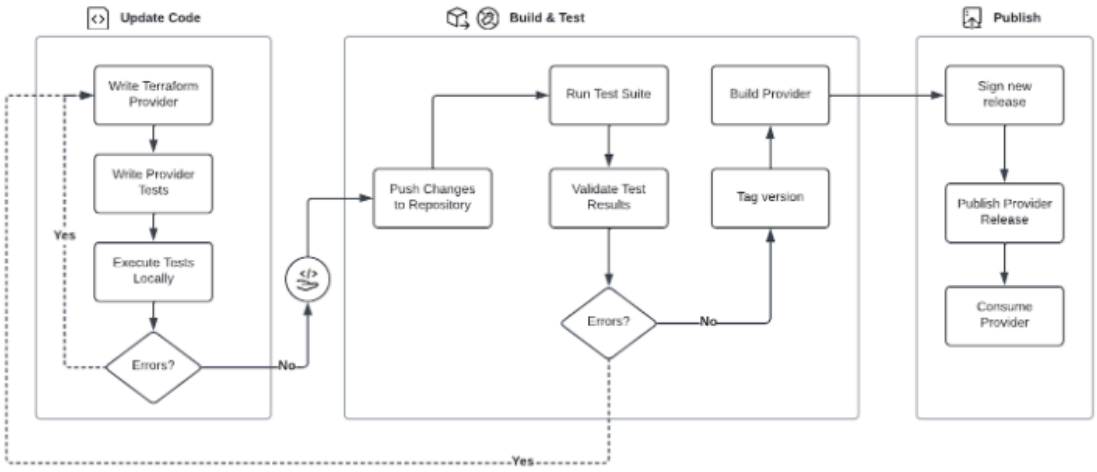


Figure 25: Publishing a custom provider

In addition to the steps illustrated above, you should also:

1. Add documentation
    - **Prepare documentation:** Write clear and comprehensive documentation for your provider, including how to use it, available resources, and configuration options.
    - **Host documentation:** You can host this documentation within the VCS repository, on an internal documentation site, or alongside the provider in the registry (if supported).
2. Announce and educate
    - **Announce the availability:** Inform all relevant stakeholders and potential users about the new provider.
    - **Provide training and resources:** Offer training sessions, documentation, and support channels to help users understand and effectively use the new provider.
3. Maintenance and updates
    - **Plan for regular updates:** Keep the provider up to date with regular updates and patches.
    - **Monitor usage and feedback:** Continuously monitor its usage, gather user feedback, and make improvements as necessary.
4. Additional tips

- **Change log:** Maintain a change log to document all changes, enhancements, and fixes in each version.
- **Security:** Regularly audit the provider for security vulnerabilities and address them promptly.

Please refer to the publishing provider section of the documentation for additional information and detailed examples.

12.4 Sharing whitelisted public Terraform modules and providers

Public Terraform modules are modules available on the Terraform public registry. The private registry allows the creation of a short list of modules from the public registry that are made available to the users via the private registry.

Using the modules available in the public registry can be a cost-effective way of achieving objectives similar to those of a custom module because a third party does maintenance. However, it does come with responsibilities, as we recommend vetting those modules first, which will require effort on your part. This step cannot be overlooked as supply-chain security is an important component of your security posture, as high-profile security incidents have proven in the last few years.

Similarly to Terraform modules, providers from the public registry can be whitelisted and made available via the private registry. The same vetting requirement applies to providers as they are also part of your "software supply chain".

12.4.1 Publishing public providers or modules to the private registry

Publishing public providers or modules in the private registry is a simple operation:

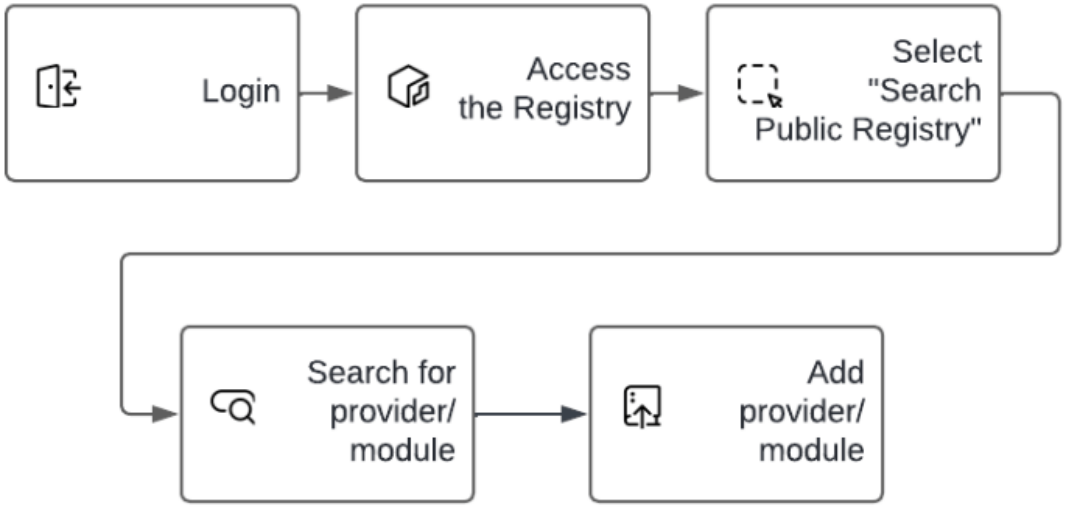


Figure 26: Publishing public providers or modules

12.4.2 Publishing public providers or modules to an air-gapped private registry

The above-mentioned publishing method can be used for TFC and TFE instances, but only if the TFE instance has access to the public Terraform registry. However, this may not always be possible due to organizational requirements, such as the need for increased security or if the TFE instance is located in a physical environment where internet access is limited or non-existent.

There is, however, a way to work around this obstacle for providers:

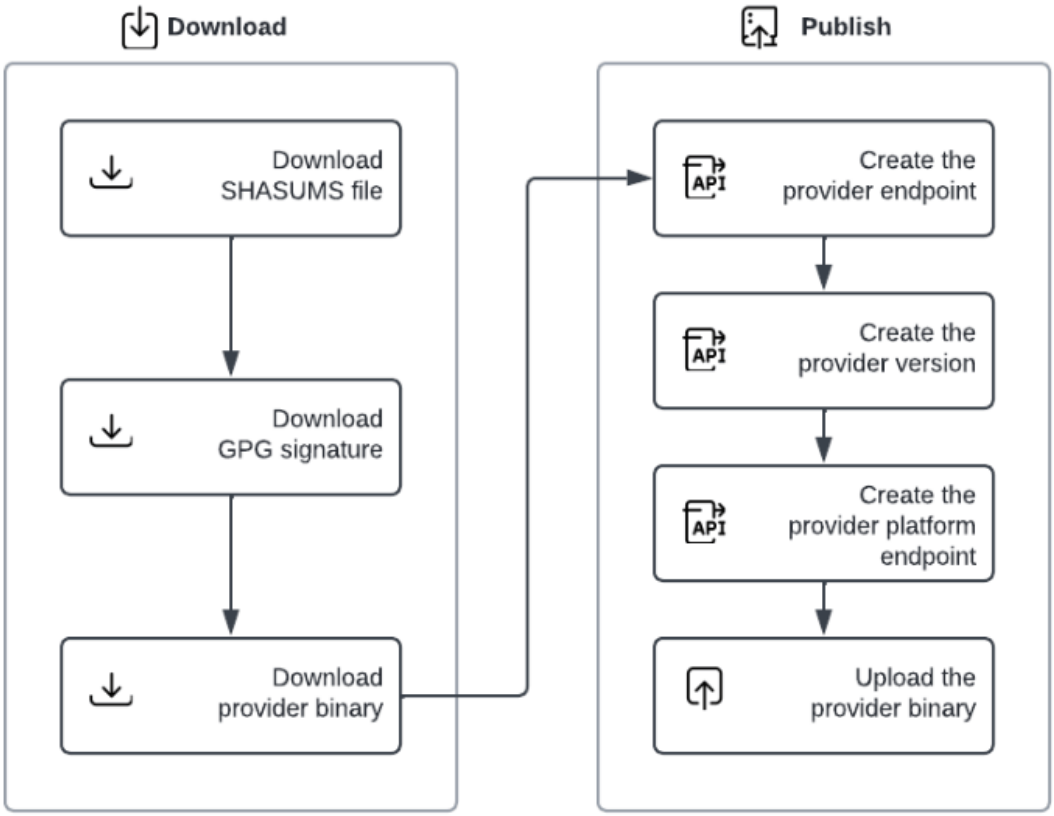


Figure 27: Publishing public providers or modules air-gapped

For additional information and detailed examples, please refer to the Publishing Public Providers in an Air-Gapped Terraform Enterprise section of the documentation.

An alternative approach is to use the provider network mirror protocol. This method relies on:

- The providers have been identified, downloaded and placed on an internal hosting site.
- The Terraform CLI configuration file includes a directive to point to the network mirror URL for provider installation.

Because this method requires the `provider_installation` directive in the Terraform configuration file, you will need to build and maintain a custom worker (or agent) image featuring this configuration.

For additional information and detailed examples about this alternative approach, please refer to the Provider Network Mirror Protocol and CLI Configuration File sections of the documentation.

Similarly, there is also a way around this obstacle for modules:

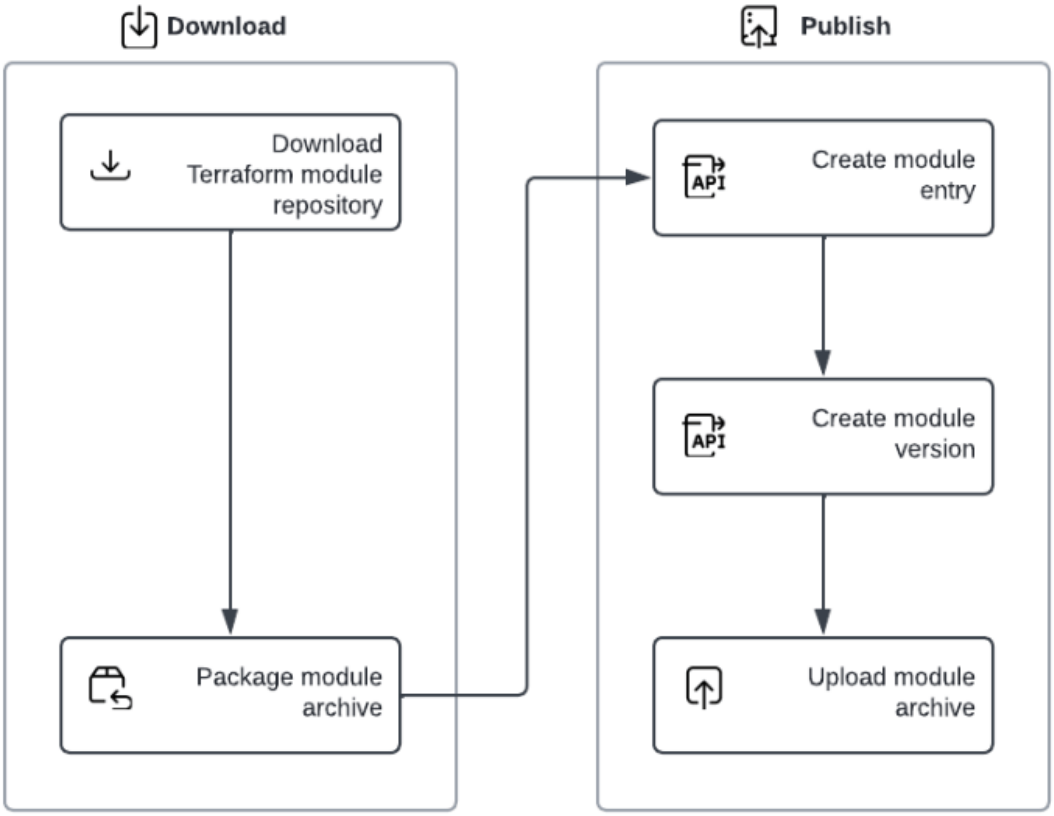


Figure 28: Publishing public providers or modules mirror

For additional information and detailed examples, please refer to:

- Create a Module (with no VCS connection)
- Create a Module Version
- Add a Module Version (upload)

12.4.3 Removing a public provider or module from the private registry

Deleting a public provider or module from your private registry does not remove it from the public Terraform Registry. Removing it from your private registry doesn't actually remove it from the public registry, so people in your organization could still find and use it. If you need to blacklist a public provider or module, you should implement a Sentinel policy that prevents Terraform runs that use a blacklisted provider or module.

Before proceeding with the deletion of a module or provider from your Terraform registry, make sure you understand how the deletion will affect your existing infrastructure. Here are the things you might consider to look for:

- Dependencies:
    - Identify Dependent Configurations: Deleting a resource with active dependencies can disrupt infrastructure management and potentially lead to errors or unexpected behavior.
    - Plan for Alternatives: If dependencies exist, consider strategies to address them before deletion. This might involve migrating to alternative providers or modules, adjusting configurations to eliminate reliance on the resource, or implementing a phased removal approach.
- Usage within workspaces:
    - Review Workspace Configurations: Carefully inspect workspace configurations to confirm whether any active workspaces are currently utilizing the module or provider slated for deletion.
    - Consider Impact: If in use, evaluate the potential implications of removal on those workspaces, including potential disruptions to infrastructure management or the need for configuration adjustments.
- Version control:
    - Preserve Version History: Ensure you maintain a comprehensive history of the module or provider's versions within your version control system, even after deletion from the registry. This safeguards the ability to revert to previous versions or access historical configuration details if necessary.
- Documentation:

    - Update Documentation: Meticulously update any organizational documentation or Terraform configuration examples that reference the removed module or provider. This maintains clarity and consistency, preventing confusion among team members who might reference outdated information.

- **Communication:**
    - Notify Relevant Teams: Inform relevant teams and individuals within your organization about the removal, particularly those who might have been actively using the resource. This fosters awareness and allows for necessary adjustments to their workflows.
- **Additional considerations:**
    - Alternative Access: Remember that deletion from the private registry doesn't preclude users from accessing the module or provider directly from the public Terraform Registry if needed.
    - Security Implications: If removing a provider due to security concerns, promptly address any affected infrastructure components to mitigate potential vulnerabilities.
    - Compliance Requirements: Adhere to organizational policies or regulatory compliance requirements that might mandate the retention of certain resources for auditing or record-keeping purposes.

12.5 Discovering Terraform Modules and Providers

Whether novice or expert, your Terraform practitioners benefit from an easy-to-use interface to the private registry instance. The discovery of Terraform modules and providers is done from the TFC interface by selecting the **Registry** entry in the left panel.

The web interface is designed to speed discovery. It features tabs to select either the "Modules" or the "Providers" view.

The "Modules" view features a flexible filter that allows users to select modules by provider, source (from the public or private registry), and publishing mode (tags-based or branch-based). The list can also be restricted to show only "no-code ready" modules.

The web interface allows quick access to a number of useful information about a module, such as:

- The module's README documentation

- Input variables
- Output variables
- Dependencies
- Resource types created
- Configuration snippet (which can be copied into the clipboard in a single click)
- Module download statistics
- The Terraform Configuration Designer

The "Providers" view is simpler and provides a paginated list of available providers.

The web interface allows quick access to a number of useful information about a module, such as:

- Module documentation
- Provider download statistics
- Source VCS repository
- Configuration snippet (which can be copied into the clipboard in a single click)

12.6 Consuming Terraform Modules or Providers

Consuming Terraform modules or providers is relatively simple:

- Find the relevant module or providers in the private registry.
- Use them in your Terraform configuration

As a user of Terraform modules and providers, you should use version constraints because they are an easy way to avoid breaking changes that could happen if you always use the latest version. But you should also:

- Monitor the modules and providers you are using for deprecation notices.
- Set aside time to upgrade to a newer version of the module or provider before its deprecation is effective.
- Volunteer applicable feedback and lessons learned to the module or provider author(s) so that they can be improved for your use cases.

12.7 Module development and maintenance

Now that we've covered how to share, discover, and consume modules using the private registry, we must cover a few key points about module development and maintenance:

- Module testing
- Module version lifecycle

12.7.1 Module testing

With Terraform v1.6.0, HashiCorp introduced the `terraform test` command and the official test framework for Terraform code. Since then, we've made continuous improvements to the framework, adding mocking capabilities in v1.7.0, for example.

We consider that Terraform modules should have a test suite and that the test suite should be maintained to match the module's features. Our recommended module publishing high-level workflow includes testing steps, as highlighted in the diagram below.

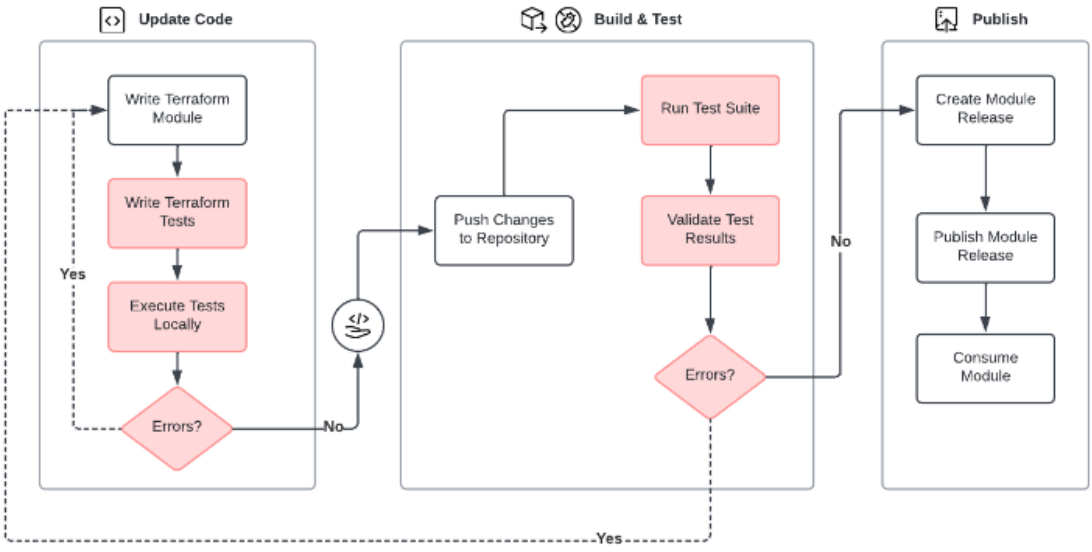


Figure 29: Module publishing high-level workflow

For example, if you advertise to your practitioners that the module will deploy a virtual machine instance in one of the authorized regions, then you will want to:

- Expose variables to take the target region as an input.
- Validate that the specified region is a valid one and return a useful error message if it's not.
- Have a test suite that validates that given a valid region as input, the module can deploy a virtual machine. The test suite will also ensure that given an invalid region as input, the module will not deploy the virtual machine.

The advertised features of the module are the "contract" between the module author and the Terraform practitioner. As a module author, you want to ensure that as you add more features to the module or fix bugs, you don't break the contract.

As the features supported by the module grow, and the contract grows with it, it becomes more and more costly for the module author to check manually that changes don't break the contract. While maintaining the test suite does represent more work, it's easy to realize that it will pay dividends and is still a more efficient alternative to sticking with manual checks.

Terraform's automated testing capabilities are a robust and efficient way to ensure that the contract will not be broken by mistake.

Module authors can implement two types of tests using the Terraform test framework:

1. Unit tests
2. Integration tests

We'll go into more detail in the rest of the section.

Unit testing

Unit tests verify individual resources and configurations for expected values. They focus on things that are known at plan time:

- Model different combinations of input values
- Verify that conditionals/logic are working as expected
- Validate string interpolations
- Catch unintended consequences of refactoring

They also focus on things that should fail when fed invalid input:

- Input validation
- Preconditions
- Checks

For more information, see the following documents:

- Terraform v1.7 adds test mocking and config-driven remove
- Input Variable Validation
- Preconditions and Postconditions
- Checks with Assertions
- Expecting failures
- Terraform test Assertions

Integration testing

Integration tests verify that a configuration using the Terraform module and valid inputs is able to deploy the configuration successfully. They focus on things that are only known after the Apply:

- Verify expected values for calculated attributes & outputs
- Apply-time errors such as:
    - Missing permissions
    - Features or APIs not enabled in your account
    - Invalid YAML/JSON in a policy or job spec

They also focus on interoperability testing:

- Validate the impact of a new
    - Terraform version
    - Provider version
    - Child module version
- Detect inconsistencies between cloud regions and partitions

For more information, see the following documents:

- Write Terraform Tests

Selected examples

We'll present examples of the following test techniques:

- Input validation

- Pre and postconditions
- Check

Input validation is about specifying a custom validation rule for a particular variable. The example below validates that the AMI ID has the correct format.

```
variable "image_id" {
  type        = string
  description = "The id of the machine image (AMI) to use for the server."

  validation {
    # regex(...) fails if it cannot find a match
    condition     = can(regex("^ami-", var.image_id))
    error_message = "The image_id value must be a valid AMI id, starting with \"ami
        -\"."
  }
}
```

Pre and postconditions are used with resources, data sources and outputs. Terraform checks a precondition before evaluating the object it is associated with and checks a postcondition after evaluating the object. Terraform evaluates custom conditions as early as possible but must defer conditions that depend on unknown values until the Apply phase. The example below shows a precondition. In this scenario, the code author wants to ensure that the AMI architecture used to deploy the EC2 instance is of type `arm64`.

```
resource "aws_instance" "this" {
  instance_type = "t3.micro"
  ami           = data.aws_ami.this.id

  lifecycle {
    # The AMI ID must refer to an AMI that contains an operating system
    # for the `arm64` architecture.
    precondition {
      condition     = data.aws_ami.this.architecture == "arm64"
      error_message = "The selected AMI must be for the arm64 architecture."
    }
  }
}
```

The condition will validate that the AMI is of the correct architecture type, and if it's not, Terraform will throw the `error_message`.

The example below shows a postcondition. The code retrieves an `aws_ami` using a data source.

The data source block features a `lifecycle` block and a `postcondition` block, which validates if the AMI ID has the required tag.

```
data "aws_ami" "this" {
  id = var.aws_ami_id

  lifecycle {
    # The AMI ID must refer to an existing AMI that has the tag "tfe-server".
    postcondition {
      condition     = self.tags["Component"] == "tfe-server"
      error_message = "tags[\"Component\"] must be \"tfe-server\"."
    }
  }
}
```

The `condition` will validate if the object referenced using the keyword `self` has a `Component` tag set to `tfe-server`. If that is not the case, Terraform will throw the `error_message`.

The check block validates your infrastructure outside of the usual resource lifecycle. It uses one or more assert blocks to perform validations. The example below performs a GET request on the Terraform website and throws the error_message if the returned status_code is anything but 200.

```
check "health_check" {
  data "http" "terraform_io" {
    url = "https://app.terraform.io/"
  }

  assert {
    condition     = data.http.terraform_io.status_code == 200
    error_message = "${data.http.terraform_io.url} returned an unhealthy status code"
  }

}
```

Terraform Validation Options Summary

We've presented a number of strategies to perform validations in Terraform. The table below lists the options and their characteristics so that you can select the best option for the job.

Capability	Version	Audience	Blocking?	Scope/Lifecycle
Input validations	>= 0.13	Consumer	Yes	Input variables
Pre/postconditions	>= 1.2	Consumer	Yes	Resource, data source, output
Checks	>= 1.5	Consumer	No	Terraform configuration, post-plan/apply
Tests	>= 1.6	Producer	No	Modules, separate from plan/apply
Policies	TFC/E	Consumer	Optional	All in-scope provisioning runs - all resources, plan, state, and run data

Test-integrated module publishing

With the introduction of the Terraform test framework, HashiCorp also rolled out the "test-integrated module publishing" feature in HCP Terraform. The objective of this feature of the private registry is to streamline the module testing and publishing process.

The test-integrated module publishing option is available for the branch-based module publishing option. Once enabled on the module, test runs will execute automatically based on version control events such as pull requests and merges. They can also be initiated from the command line or using the API.

In addition to the improved integration with pull requests and merges, this feature allows tests to execute remotely in a secure environment, eliminating the need for developers to handle sensitive cloud credentials on their workstations.

For more information, see the following document: Test-integrated modules in the HCP Terraform private registry.

12.7.2 Module version deprecation

As you release new module versions, you may need to deprecate versions for a few reasons:

- The module author(s) have discovered a critical issue with a version, and your Terraform practitioners should not use that version due to its critical nature.
- The module author(s) may EOL older versions that are no longer supported. To strike a balance between adequate support and available resources, you may have an internal policy to only support the latest n-2 versions of a module. All versions older than that would then be "unsupported" and therefore deprecated.

While there are legitimate reasons to deprecate a module version, module authors must consider the needs of their stakeholders, which in this case are the teams using those module versions. To minimize disruption and ensure a smooth transition, we recommend a number of practices to handle module version deprecation:

- Be transparent and communicate
- Make it easy to transition to a supported version
- Manage the post-deprecation lifecycle
- Support your stakeholders

Transparent deprecation announcement

- Early communication: Clearly announce the deprecation of a module version well in advance of its end-of-life (EOL) date. Utilize readily accessible channels such as documentation, release notes, and dedicated communication platforms to reach all users.
- Detailed timeline: Establish a clearly defined timeline for the deprecation process. Include milestones such as the announcement date, EOL date, and any interim points for limited support or resource availability.

Facilitate the transition

- Provide migration paths: Offer well-documented alternatives for users looking to migrate from the deprecated version. This can include newer versions of the same module, compatible modules from diverse sources, or even entirely different approaches to achieving the desired outcomes.

- Develop migration guides: Create comprehensive, step-by-step guides to accompany each migration path. These guides should address common scenarios, potential complications, and best practices for transitioning configurations.

Manage the post-deprecation lifecycle

- EOL enforcement: Upon reaching the EOL date, use Sentinel policies (advisory) to flag Terraform code that still uses deprecated modules and inform the user. Consider removing the module version from all public registries or repositories to prevent inadvertent access and deployment.
- Preserving knowledge: Maintain the deprecated version's documentation for reference purposes. This will aid existing users still within the migration window and facilitate support during the transition period.

Support your stakeholders

- Limited support window: Extend limited support for the deprecated version during the transition period. Focus on addressing critical bugs and issues that might impede ongoing operations.
- Open communication channels: Establish effective communication channels to answer user questions, address concerns, and gather feedback during the deprecation process. Consider hosting discussions, Q&A sessions, or dedicated support channels to facilitate information exchange. Open communication with your internal customer base cannot be understated.

13 Policy as code and Sentinel management

> **Note**
>
> This page covers operator setup and management of Sentinel policy as code. For the user-facing policy enforcement experience, see the User Guide: Policy as Code - Sentinel.

HashiCorp Sentinel is a robust policy as code framework. By integrating with HCP Terraform, Sentinel advances infrastructure lifecycle management by enabling the codification and automation of policies, governance, risk management and compliance directly into the process.

Please note that HashiCorp supports Sentinel and Open Policy Agent (OPA) as frameworks for implementing policy as languages. Sentinel or OPA can be used in workflows.

If trying to decide between Sentinel and OPA, we recommend that Sentinel is leveraged because of its established performance characteristics. OPA is recommended when an organization has standardized with OPA and has existing policies that can be brought into the platform. Hybrid situations where sentinel and OPA are jointly used because of the availability of policies in both languages are also encouraged.

13.1 Benefits of using Sentinel

- Mitigate risk
  - Reducing risks is a key driver for all organizations and especially those with regulatory requirements. Sentinel mitigates risk by automating policy provisions.
  - Sentinel actively works to lower the chances of errors and vulnerabilities. By enforcing rules during the planning and execution phases, it acts as a shield against unwanted surprises, ensuring the stability and security of your digital assets.
- Regulatory governance and policy enforcement
  - Sentinel ensures that every action aligns with the established organizational policies, regulatory guidelines, and security standards.
  - Large scale policy and governance can be managed in a cohesive and concise manner.
  - Auditing becomes straightforward, and reduces company time spent by Platform Team staff with auditors.
- Separation of concerns
  - Sentinel policies can be defined, managed and applied by platform, compliance, security, or regulatory functions within the business, separate from the application teams deploying infrastructure.
  - Unlike other forms of testing and validation, Sentinel policies are not defined in the same codebase as the infrastructure to which the policies apply.
  - Project or workspace owners cannot opt out of policies unless given explicit permissions to do so. This is an important enabler for self-service workflows, reducing time-to-value for application teams, while ensuring compliance through policy-as-code.
- Sandboxing

    - Policies act as core guardrails. With the increasing complexity of IT systems over time, the need arises to shield them from potentially harmful actions.
    - Manual verification is less efficient, prompting the adoption of policies represented as code.

- Codification
    - The representation of policy logic as code makes governance clearer, more efficient, consistent, and operationally reproducible.
    - Encoding policy logic eliminates reliance on oral traditions and removes risk of misinterpretation.
    - Fosters transparency and aids the dissemination of knowledge within the organization.
- Version control
    - Version control provides history tracking, diffs, pull requests and other modern collaboration practices.
    - It controls policy evolution tracking by implementing demonstrable and auditable change.
- Testing
    - Treating policies as code facilitates syntax and behavior validation.
    - Sentinel ensures that policies align with expected outcomes, establishing a baseline for policy correctness.
    - Automated CI testing is possible through the built-in Sentinel testing framework which reduces TCO for governance, particularly given how complex modern systems have become.
- Automation
    - Policy-as-code means automation of policy deployment which is far faster than current manual work and requires fewer humans.
    - Automation allows consistency of governance at scale, across the entire infrastructure.
    - Reduces manual intervention and mitigates the risk of human error.

13.2 Explanation of the policy enforcement workflow

The policy enforcement workflow with HashiCorp Sentinel is a structured process.

- Organizations begin by defining policies that articulate the rules governing their infrastructure.
  - Customers are advised to render their governance and compliance policies in such a way so that Sentinel policies are then authored to satisfy each.
  - Doing so will enable your business to operationalize governance and controls through automated means.
  - Typical policies customers start with include governance of maintenance windows protecting software components from adverse change at the wrong time, and enforcement of metadata tagging of cloud resources, and infrastructure-as-code style such as ensuring Terraform module versions are pinned and usable only from your organization's private registry.
- The next step is to code the policy using the Sentinel language. See below for more on how code repositories are structured.
  - Policies are arranged in sets.
  - Policy sets are scoped to an entire organization or to one or more projects and/or workspaces in your HCP Terraform organization.
  - Scoping policy sets at the project level allows policy enforcement to be integrated into self-service workflows, which typically create new workspaces on the fly then run them immediately. Any new self-service workspace will inherit the policy sets that are scoped to the parent project.
- Sentinel is configured to operate according to the policies in the policy sets that are scoped to the running workspace.
  - Depending on the enforcement level (advisory, soft- or hard-mandatory) of the policy, Sentinel will either allow or deny the apply phase of the workspace run.

13.3 Key people and their responsibilities

- Sentinel policies should be owned by the Security team, with input taken from other areas of the business who are responsible for regulatory compliance.
- The CCoE/Platform Team typically will understand both Sentinel code, and the means to employ it effectively in production pipelines.
- The CCoE/Platform Team should thus partner with the Security Team to manage ownership and RBAC over Sentinel policy code and its application to ensure sufficient protection of the code repositories and the automation used to employ it.

13.4 Sentinel policy authoring best practices

These guidelines will help you create and manage Sentinel policies that are effective, maintainable, and keep your Terraform environments secure. Whether you're new or an experienced Sentinel practitioner, these best practices enhance the effectiveness and maintainability of your policies.

> Note
>
> The guidance here does not apply to individual policy library contributions to the public registry which has different requirements and is covered here.

13.4.1 Repository organization

- Organizing your HashiCorp Terraform Sentinel Policies in a clear and structured manner is crucial for collaboration and maintainability. Your repository structure should resemble the structure shown below.

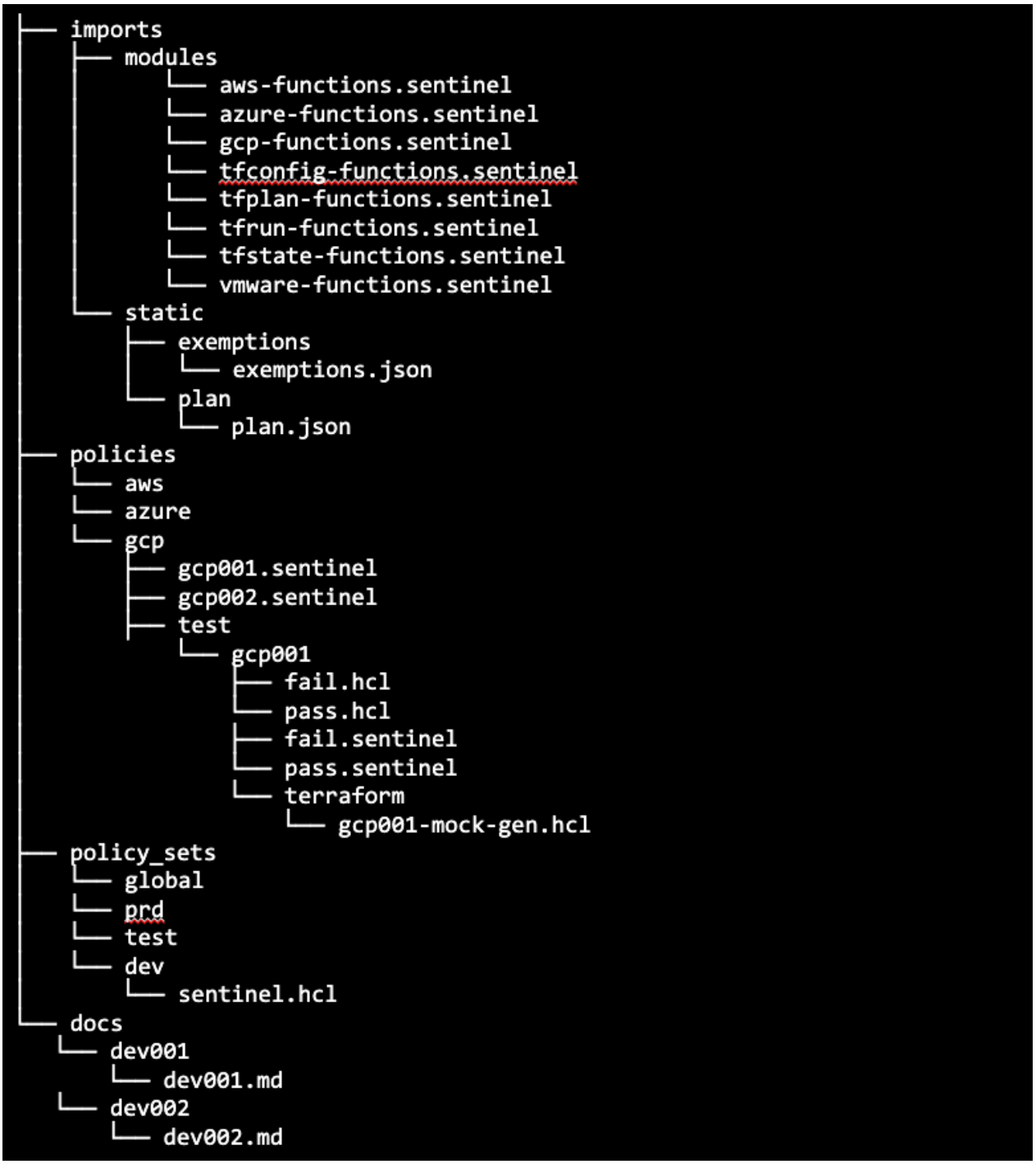


Figure 30: Repository Organization

- **policies/**: This is where you store main Sentinel policies, organized by environment — the same as for the policy sets themselves. Under each environment subdirectory, the policies are contained with a further test subdirectory under which the test files, generated mock data and Terraform configuration used to generate the mock data are stored. This keeps the test data all together and is a pattern we have found very common in our customer base.
- **modules/**: Create, organize and store your reusable policy modules here. Each module contains its own policies. Per the illustration above, the common tf import functions, which have been available free from HashiCorp, are stored together with site-specific ones for your business, contained in files with illustrative names so a simple, flat structure can be used.
- **docs/**: Always fully document your policies, and their tests so that you can come back to them after they have been working for months unchanged. This is the best practice for all code.

13.5 Writing policies

- In order to author Sentinel policies for your organization, acquire the IT security policies relevant to the deployment of IT infrastructure, if it exists. The set of policies for your organization should be translated into policy-as-code so that company policy can be asserted automatically with HCP Terraform. A set of policies required for implementation should be derived from company policy.
- If such a policy list does not exist, a general set of controls should be agreed and documented internally to the Platform Team, in terms of public and/or private cloud infrastructure deployment best practices. This set of controls should be designed to be extended over time.
- If a CCoE exists, the policy list should be communicated to the CCoE team for publication on your internal documentation platform as a reference for what developers should expect to be bound by when they develop IaC.
- Next, it is imperative that staff responsible for policy development understand the Sentinel language. The language documentation should be read so that developers are familiar with the available language constructs.
- We strongly recommend that developers attend the HashiCorp Sentinel Academy training, which teaches how to write and implement policies, integrate Sentinel with Terraform, and more. It also includes hands-on labs and real-world examples. Reach out to your HashiCorp Solution Engineer or Customer Success Manager to get access to this resource.
- Sentinel 0.15.0 introduced the concept of modules, which allow you to specify reusable func-

tions that significantly reduce the length of your codebase. More information is in the *Extending Sentinel* section in the language site. We recommend that developers clone the terraform-sentinel-policies GitHub repository because this provides both a set of prewritten policies that work for a set of public and private cloud providers and also a set of common reusable modules (there called *functions*), which will allow you to work more efficiently. These policies work with v2 Terraform imports. The content provides an excellent example from which to derive your own policies. Many customers start with these policies, generating a global policy set from a subset of the policies contained in the repository, and extend from there.

- From the policies stored in the repository, read all that apply to your work. We recommend extensive experimentation so policies are well-understood.
- Policy authoring software development life cycle is covered in the section below.
- Document the policies in use.
- Peruse the HashiCorp public registry as it has a set of policies already available. Note that this is a free resource, and we recommend developers should read and understand the policies present on the public registry prior to using them, and only in development to start with, as they may need modifying prior to use in production. The policies in the terraform-sentinel-policies repository is being migrated to the public registry over time.

13.5.1 Enforcement level

The assignment of enforcement levels to policies is recorded in a configuration file named `sentinel.hcl`, residing in the same repository as the policies. The example below illustrates how this is applied.

```
policy "restrict-expensive-workspaces" {
  source            = "./restrict-expensive-workspaces/restrict-expensive-workspaces.
      sentinel"
  enforcement_level = "hard-mandatory"
}
```

There are three enforcement levels which may be applied to Sentinel policies. Typically, we advise using advisory in development environments in order to help advise developers where policies may restrict configurations in high environments, but this depends on your organizational requirements.

- Advisory: This enforcement level offers warnings of non-conformance but does not obstruct the Terraform workflow. As such, this level is useful for suggesting improvements or pointing

out potential issues without imposing strict requirements.

- Soft Mandatory: Failure of a configuration to conform to a soft mandatory policy causes the Terraform workflow to stop and provides a meaningful output. It lets an organization owner or a user with override privileges proceed with the run in the event of failure.They are suitable for providing guidance and best practices to users without preventing them from proceeding with their Terraform actions.
- Hard Mandatory: The hard mandatory enforcement level imposes strict requirements and prevents non-compliant actions during the Terraform workflow. This enforcement level is critical for enforcing security, compliance, or other essential constraints that must be adhered to without exceptions.

13.5.2 Testing policies

Unit test

- Read the public document on testing Sentinel policies. This section focuses the reader on standardizing your organization's approach to the unit testing of Sentinel policies.
- Use the Sentinel command line utility for testing policy authoring, because it is quicker than iteratively committing change to a policy repository and testing through a Terraform workspace.
- Standardize on HCL instead of JSON for the syntax of Sentinel Configuration files.
- Use this standard structure for testing Sentinel policies, bearing in mind the opinion referred in the linked document above:
- Using a repository which contains a set of policies (for example called policy1, policy2 and policy3), individual policy tests should be run with `name=policy1.sentinel make test`
- To test the whole set with this method, running `make tests` will iteratively test all the policies in the repository.
- Note that the individual test command is verbose. Standardize your approach by testing your policies individually to start with and then for convenience, run unit tests on the whole set.
- The key differences here are that you:
    - Use a local simulator which is faster
    - Use a Makefile for convenience and speed
    - Use the command line, so integration with CI/CD pipelines is also straightforward.
- We recommend a CI tool to automate policy testing with the following considerations:

    - The time it takes for an orchestrator to deploy a runner which clones the policy repository and run unit tests is about the same as it would to commit the changes to a feature branch and get HCP Terraform to run a workspace to test it.
    - HashiCorp wrote the Sentinel CLI tool to reduce the time the developer waits for this process to occur, so we recommend keeping as tight a loop as possible for unit testing, and use the CI tool for more complex integration testing as below.
- Mock data used for testing Sentinel policies should be handled in accordance with the public documentation on the subject.
- HashiCorp's upcoming "HVD Terraform Administration and User Guides — Scale" will cover how to set up scaled policy authoring, including automated mock generation for unit testing.

Integration test

- When you have many policy sets, testing becomes more complex. We recommend use of the strategic CI tool your organization already uses for continuous integration testing.
- Before setting up your Sentinel integrations, you'll need to plan your testing. Come up with test scenarios that combine different policy sets and reflect your unique policies. Especially in regulated industries, make sure you can show how each scenario maps to the specific regulations you must follow. This upfront planning will ensure your Sentinel setup keeps you in compliance.
- Sentinel policy sets are also applicable at the project level. As we recommend the application of projects to app teams, the use of the `tfe_project_policy_set` Terraform Enterprise provider resource should be used to apply policy sets designed at the project/app team level should also be tested in this context.
- Use a Platform Team-specific instance of Terraform Enterprise, such as the one the team uses for upgrade testing, or a dedicated Terraform Enterprise instance depending on cost. This keeps the registration of test policy sets away from the production instance(s). Alternatively, use a dedicated HCP Terraform organization.
- Within the test organization, workspaces should be used to run the Terraform code which deploys the test workspaces and projects used for testing, and also the policy set assignments. Typically, these runs will not be subject to Sentinel policies due to the foundational nature of the work.
- Write pipeline automation to run the tests against workspaces in HCP Terraform.
- Use the Terraform Cloud/Enterprise provider to automate the creation and configuration of test workspaces and projects. This should also involve deployment of representative variable

sets at both workspace and project levels accordingly, containing organization-specific values and authentication data for workflow identity, so the workspaces run correctly to properly scope success criteria. We also assume that the root module code and child modules called by it are error-free via the use of `terraform test` referred elsewhere in this HVD.

- Use the Terraform Cloud/Enterprise provider to automate the assignment of policy sets to the test workspaces and projects. Bear in mind the `tfe_policy_set_parameter` resource which should be used for KV as applicable.
- At this point, your test Terraform Enterprise or HCP Terraform instance will have one or more workspaces for running deployment of test objects, and once run, commensurate test workspaces and projects with test policy sets registered.
- The Platform Team should also have prepared Terraform configurations to which the policies you need to test will run against. These configurations will run in the provisioned project/-workspace contexts allowing you to test the success criteria of potentially multiple policy sets, again combinations of workspaces and projects accordingly.
- Once the tests complete, the pipeline should remove the workspace(s) and policy set(s). Always aim for a nominal set of zero infrastructural objects and configuration irrespective of success or failure of the test, in order to remove cloud waste. The tests should be documented in the CCoE for developers whose configurations will be subject to the policies applied.
- The aim of this section is to ensure that the deployment of test workspaces and projects works, that the application of policy set changes works as expected, and that the changes in the contained policies operate as expected against relevant Terraform configurations.

13.6 Policy Sets

13.6.1 Boundaries

- While general architectural guidance for IaC is to avoid monorepos, for policy-as-code, we see customers most successfully versioning their policy data in a single repository. This also allows for the maintenance of one set of common modules all policies can refer to. The exception to this would be in rare cases where there are multiple, completely separate governance/security teams that are required to manage/isolate their own policy set(s).
- Organize your policy sets by provider, what they target (e.g., security, networking), and which environments they apply to. This flexibility will make it easier to apply them correctly. For example, you might have one AWS networking policy set for all workspaces, a stricter one

just for development, and yet another set specifically for Vault infrastructure, regardless of environment:

- AWS - networking - Global
- AWS - networking - Development-only
- AWS - networking - Vault
- AWS - compute
- GCP - networking
- GCP - databases
- Cloud agnostic
- ...

13.6.2 Policy Set structure

- Policy sets should map to the repository structure which stores the policies and provides the opinionated directory layout required by the Sentinel CLI during unit testing.
- The best practices for policy set structure should be followed irrespective of whether you intend to publish your policy set to the public registry.

13.7 Sentinel Policy Set management

- The ideal state for your policy-as-code is to have reached a production-ready state where your organization's list of policies has been coded and tested. However, as with all modern software development life cycles, this process should be ongoing. An agile approach will help you manage your policy backlog effectively.
- Recognizing that there will be a backlog with perennial, outstanding tasks as you move constantly to ensure your business remains agile in its marketplace means assigning a product owner, scrum master and development team to the task of managing policy-as-code, or equivalents respectively.
- Policy sets are scoped globally, or to sets of projects and/or workspaces. For segregation of duty, we recommend that you deploy development, test and production infrastructure from separate workspaces, labeled as such. As such, the application of policy sets should be based on:
    - Global guardrails. There will always be a significant number of policies which need to be applied to everyone. At a minimum, the following guardrails should be applied globally:

    - Restrict allowed providers to those signed-off for use.
    - Restrict provisioners from being used; these should be avoided if possible and only considered a last resort.
    - All child module calls are pinned to a version; this prevents workspace runs from using unexpected versions of child modules.
    - All providers are pinned to a version to protect against changes to the underlying provider.
    - All child modules are used from the private registry; this removes risk of developers using git source strings in their child module calls in workspace configurations, and thus possibly unstable branches.
    - Minimum allowed Terraform binary versions to reduce the risk of unexpected behavior..
    - All variables should have descriptions - this enforces good Terraform code hygiene.
- Projects. Some teams will have different needs from others. Many project teams have specific requirements which can be catered for by policies explicitly scoped at the project level, such as those with exceptions to use specific providers which all other teams are prohibited from using.
- Environments. Your business may allow development workspaces to access certain resource types for experimental purposes which are not allowed in production. As such, you will need to author specific policies that are only applied in development, test and production.
- Exclusions. It may be necessary to maintain a small number of workspaces that are excluded from what would otherwise be global or project-scoped policies. This is possible using Policy set exclusions and the "exclude from" feature.

13.8 Sentinel SDLC

13.8.1 General points

- We anticipate that your organization's policies, from which your Sentinel code will be derived, must not be made public. However, we recommend that you still follow best practices.
- Over time, you will accumulate a series of repositories named policy-library-X, following the creation, test and application detailed above. Continue to ensure write permissions on these repositories are closely managed and monitored as these repositories provide your business with compliance, and will be audited.

- Ensure to use git best practices including feature branches pull requests, even if only one team has write access. Always ensure another human reads your changes before they are applied to the default branch.
- Ensure that your repositories are documented with a README.md file, and also each policy is correctly documented in the repositories under docs/policies/<policy-name>/md so that your future self can refer back to the code many months after you originally wrote it and quickly get back to adding modifications required. Not doing this will waste more time than it takes to write it.
- Be aware that applying a new policy set, or updating an existing policy set, may block workspace runs that were previously successful, thus affecting application provisioning. Ideally workspace owners should be given the opportunity to review the effect of any proposed policy changes, and adapt their Terraform code if necessary to make it compliant, before the policy change is implemented. There are various strategies to manage this:
    - Make use of on-demand policy evaluation to check the effect of any policy changes before applying the new or updated policy set
    - Apply an advisory-only version of each proposed new or updated policy set for a trial period before making the change, allowing workspace owners to observe the effect without risking any runs being blocked

Dev-Test-Prod

- Development should proceed based on the internal policy list likely published to the Platform Team by the Security Team. This should be the initial focus in order to move from where you are today to a place of compliance through policy-as-code.
- Your work will result in a series of repositories with version tags.
- Start your work with a version of the Sentinel binary which is used by your Terraform Enterprise or Cloud instance (generally the latest version of Sentinel will be available to HCP Terraform users).
    - Continue your development with the same version of Sentinel, even if a new version is released.
    - Structure your testing time so that you can unit test all policy repositories with the next version of Sentinel separately, and sign-off that there are no errors.
    - Perform a full set of integration tests using the new version of Sentinel prior to raising a change request to apply this new version of Sentinel to the policy sets you have regis-

tered in your Terraform Enterprise or HCP Terraform organization.
    - Continue processing your development backlog on the new version of Sentinel.
- Unit and integration testing should follow the provisions laid out above, and automation will be needed to allow easy regression testing for the policy estate at scale.

13.9 Summary

- Sentinel allows you to automate policy-as-code and thus your governance and compliance work.
- As Sentinel is a policy-as-code framework, it means you can apply standard software management, company SDLC policy and git best practices to the provisioning of guardrails for your development community.
- We recommend that you implement automation as standard where possible, with the exception of mock data generation at this standardization stage.
- Turn your organization's policies and requirements into Sentinel code.

Additional resources for Sentinel

- Writing and Testing Sentinel Policies for Terraform
- Defining Policies - Sentinel - Terraform | Terraform | HashiCorp Developer
- Sentinel for Terraform Part 1
- GitHub - hashicorp/terraform-sentinel-policies: Example Sentinel Policies for use with Terraform
- Enforce policy as code

14 Run tasks management

> **Note**
>
> This page covers operator setup and management of run tasks. For how users encounter run-task results during Terraform runs, see the User Guide: Run Tasks.

HCP Terraform run tasks let you directly integrate third-party tools and services at certain stages in the HCP Terraform run lifecycle. You can use run tasks to validate Terraform configuration files,

analyze execution plans before applying them, scan for security vulnerabilities or compliance requirements, perform detailed cost analysis, or perform other custom actions.

Run tasks send data about a run to an external service at specific run stages. The external service processes the data as appropriate, can evaluate whether the run passes or fails, and sends a response back to HCP Terraform. HCP Terraform uses this response and the run task enforcement level to determine if a run can proceed. The external service can store and monitor the run-related data for continuous review and monitoring and for historical purposes.

The run tasks functionality sends an API payload to the external service endpoint. This payload contains the information Run tasks send an API payload to an external service. This payload contains the pertinent information related to the workspace and run, and includes a callback URL which the external service uses to return a pass or fail status and messaging to HCP Terraform.

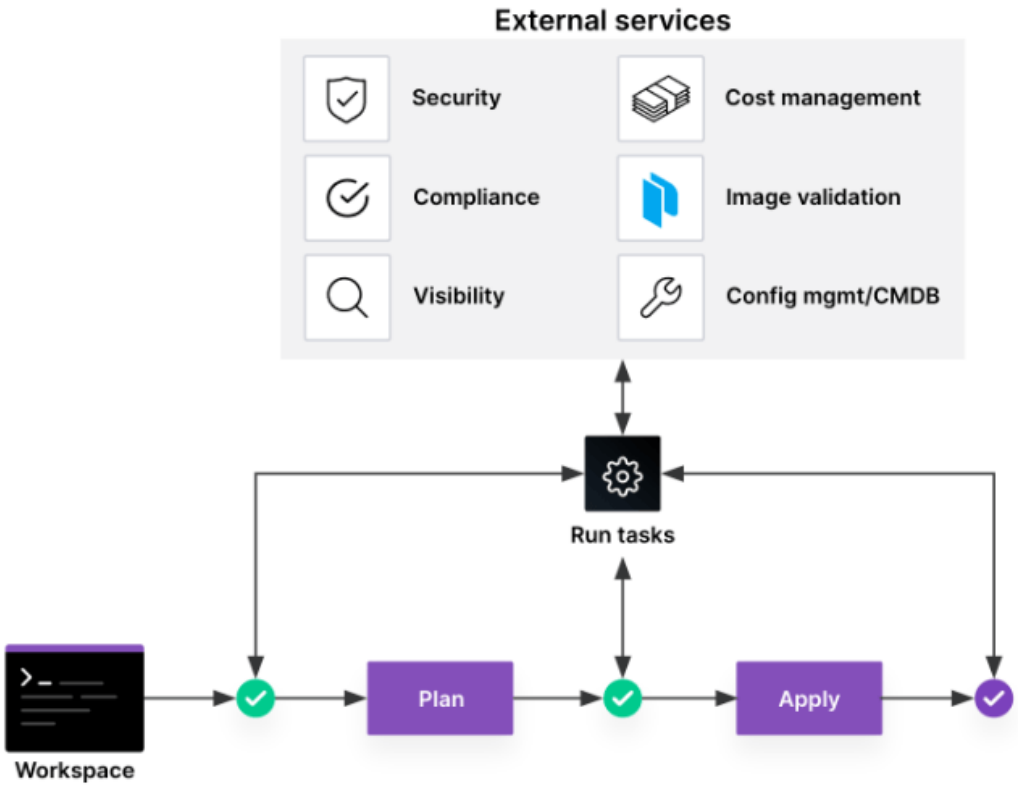


Figure 31: Run Task Embedded in the Run Workflow

14.1 Benefits of Run Tasks

Your teams can integrate third-party tools into the HCP Terraform workflow during any of the run stages: pre-plan, post-plan/pre-apply, and post-apply. By allowing for this real-time integration during the run workflow, conditions can be set for runs to pass and continue, or fail and terminate, with only a brief interruption in the run flow.

Run tasks can take the place of some Sentinel policies that you would otherwise have to develop and manage. The replacement would be an integration with a third party product well-suited to testing the specific intricacies of the infrastructure provisioning concerns they support as experts. This can reduce development and testing time while providing more stringent and accurate security and compliance testing.

Run tasks now support a rapidly growing number of integrations with many well-known tools that you may already be taking advantage of. And if you find that an internal or external product or requirement is not currently supported in the way you require, you can develop a custom run-task to meet the need, or possibly request the product owner to explore that as a valuable add-on feature to their offering. Some examples of the specific use cases for post-plan run task integrations available and currently in use are:

- **Security and regulatory compliance:** Ensure that you're not provisioning misconfigurations that cause security or compliance issues using tools like Palo Alto Networks Prisma Cloud, Zscaler, Snyk, Tenable,, Sophos, Aqua Security, Firefly, and
- **Cost control:** Provide visibility into the costs of infrastructure prior to any change using Infra-cost, Vantage, or Kion
- **Visibility:** Enhanced resource visualization with Pluralith
- **Image compliance:** Verify that only approved and tracked golden images are used with the HCP Packer run task

For a complete list of available run tasks please refer to public Terraform Registry.

14.2 Overall flow for implementing Run Tasks

The overall flow of setting up to execute a run task against a workspace, or set of workspaces is as follows:

- Select the desired run task from the registry, review the requirements and documentation, and ensure connectivity can be made in a two-way fashion with the HCP Terraform Platform.

- Connectivity between HCP Terraform platform and the run task endpoint must be established and verified. This will likely require network and security modifications to open the path.
- The desired run task must be "created" in the Terraform Organization in order to be available for assigning to workspaces.
- This is an action of configuring the run task and connecting the endpoints on both sides and testing the communication path has been established.
- The run task must be associated with a workspace and configured as to what stage to execute in and whether the outcome will be handled as Advisory or Mandatory.
- The run task will execute as part of the normal run cycles for that workspace and the results can be examined in the same way run completions are validated.

14.3 Overall flow for Run Tasks within a run

Run tasks are invoked before and after the plan and apply stages of a Terraform run. The information that is available at each stage is presented to the run task when invoked. At the pre-plan stage, the information available will mainly be the code and other attributes; at post-plan and later stages, more information associated with the results of the plan will be available.

Selecting the stage to associate a run task to will vary for a number of factors. The pre-plan stage would be used when the run task is going to examine the code and make a determination if entering the plan stage should even be allowed. The other stages primarily depend on how far you want a plan to go before being stopped by a failure result of the run task and what states of provisioning you are able to accept.

For example, a post-plan or pre-apply run task will examine the plan and determine whether an apply should be allowed. A post-apply would be used mainly to test and possibly gather/store information related to the infrastructure that has been provisioned. Check the documentation for the run task being configured if there are particular stages required or suggested to satisfy its requirements.

Multiple run tasks can be defined at a single stage. They will execute in sequence and in no particular order that could be depended on (do not assume any dependencies between run tasks). Once all run tasks complete for a stage, the continuation of the run is determined by the most restrictive enforcement level in each associated run task.

For example, if a mandatory task fails and an advisory task succeeds, the run fails. If an advisory task fails, but a mandatory task succeeds, the run succeeds and proceeds to the next stage. Regardless

of the exit status of the run tasks, HCP Terraform displays the status and any related messages are recorded and appear in the run results.

14.4 Available Run Tasks

The common implementation of run tasks will be to implement an existing run task that integrates with a specific complementary product, usually provided and supported by a HashiCorp technology partner that provides that product. These are published in the public Terraform Registry where you can then view the most up-to-date list of available run tasks with links to the partner's usage documentation. Other vendors or community members may offer run tasks for specific integrations, but they will not be published to the public Terraform Registry, and should be handled as any other open source software.

In addition to using existing technology partner integrations, HashiCorp Terraform customers can build their own custom run task integrations. The process of developing custom run tasks is discussed in a separate topic in this documentation set.

14.5 Run Task specific requirements

Each run task provides specific configuration requirements for integration with HCP Terraform. The documentation supplied by the run task's owner should include all necessary details for setup. You should carefully review these requirements and prepare accordingly to complete the configuration and ensure proper connectivity. This process typically involves adjustments to network and security settings and generating tokens to establish secure connections between the services.

14.5.1 Terraform version

You can only assign run tasks to workspaces that use a Terraform Core version of 1.1.9 and later. You can later downgrade a workspace with existing runs to use a prior Terraform version without receiving an error related to the run task association. However, HCP Terraform will no longer trigger associated run tasks during the run workflow.

14.5.2 Permissions required for managing Run Tasks

To create a run task in an Organization, you must have a user account with the Manage Run Tasks permission. To associate run tasks with a workspace, you need the Manage Workspace Run Tasks permission on that particular workspace.

Manage Run Tasks permission

Manage Workspace Run Tasks permission

14.6 Managing Run Tasks in an organization

Run tasks can be configured and managed in any of the standard ways you can configure HCP Terraform in general. The methods for configuration are:

- HCP Terraform UI
- HCP Terraform APIs
- TFE Provider

We strongly encourage you to use a repeatable automation method over using the manual method of managing them through the HCP Terraform UI. The UI can be useful for initial experimentation and testing, however, a reusable automation will provide an easy, repeatable, and auditable way to manage the configurations. The TFE Provider provides the cleanest DevOps implementation, by not only being repeatable and portable, but will also track the state and history and only apply new changes on each invocation.

The workflow for creating and maintaining run tasks and managing the association of run tasks to workspaces is the same regardless of the configuration method used. The documentation for creating run tasks and associating to workspaces using the UI, describe the flow and the details of the actions and their attributes that must be supplied and can be used for navigating the other methods:

- Creating a Run Task
- Associating a Run Tasks with a Workspace

Please note that removing a run task from a workspace does not delete it from the organization. To fully delete the run task from the organization, it must be detached from all workspaces before it can be removed. Additionally, it's important to remember that a run task cannot be deleted from the organization while it is still linked to any workspace.

14.6.1 Using HCP Terraform UI

These tutorials provide an example of setting up run tasks in general, and use the HCP Terraform UI method:

- Configure Snyk run task in HCP Terraform
- Set up HCP Terraform run task for HCP Packer

14.6.2 Using Run Tasks API

Run tasks can be managed with automation using the Run Task API, built specifically for this purpose. HashiCorp recommends automating the management of these tasks, preferably by including the automatic association of run tasks in your onboarding and Landing Zone type automations that are already managing the creation and other attributes of workspaces.

Since some run tasks will be applicable to sets of workspaces and possibly all workspaces in an organization, associating them manually will be time-consuming. For consistency and security, we recommend automating run tasks whenever possible.

The complete documentation for the Run Task API is located here Run Tasks - API Docs - HCP Terraform.

The API includes all of the standard CRUD requests that allow you to manage the run tasks and the associations of run tasks to workspaces. You can develop the processes that invoke the APIs using the standard tools or languages that you use for such tasks.

Managing run tasks with APIs requires API tokens generated from the same permissions as managing them through the UI. Information on working with HCP Terraform tokens is located here: API Tokens - HCP Terraform. .

The Run Task API has two distinct sets of endpoints: one for managing the run tasks within an organization and the other for managing the association of run tasks to workspaces.

The API endpoints for creating and managing run tasks in an organization are straightforward. The initial creation and list endpoints start are based at the organization level.

The remaining endpoints are based at the task level as they rely on the run task ID that was assigned as part of the initial creation.

The API endpoints for associating and managing run tasks for a workspace are also straightforward. All of the endpoints start from the workspaces endpoint, as the run task associations are by

workspace level and require a workspace ID, and then the tasks endpoint within the workspace. The endpoints addressing a specific and existing run task association require a run task id, while Associate and List do not.

14.6.3 Using the TFE Provider

The TFE Provider is a very powerful and logical way of managing the overall HCP Terraform configuration and provisioning platform. Standard Terraform code and workspaces are used to describe and provision the necessary resources. The TFE Provider includes the resources to manage run tasks at the same level as using the UI or API, but with all the added benefits of infrastructure as code. As a HashiCorp written and supported provider, the documentation containing the specifics of how to use the resources is located in the public registry:

tfe provider

tfe_organization_run_task

tfe_workspace_run_task

If this mechanism of managing the platform is already in general use, you may be able to just add these resources into the current mix by adding to the existing code/modules. If not, or if the preference is to keep such components isolated, a new workspace or set or workspaces could be provisioned with their own code repositories where the resources could be maintained.

14.7 Best practices for Run Tasks

The primary use case for run tasks is to integrate third-party tools and services (including internally developed services). As such, they should be handled in the same ways that you select and implement any software into your environments. They are similar to Terraform Providers, in that the sources of run tasks, and the tools they integrate with, should be carefully scrutinized and evaluated.

Run tasks listed in the registry are provided by HashiCorp and HashiCorp Partners, and can generally be trusted. You still need to do proper due diligence. Run tasks written by others and shared with you should be considered and treated like any OSS software, with more stringent evaluation and handling. For example, you should possibly fork and own the code as opposed to simply downloading and replacing as new changes are introduced.

14.7.1 Security considerations when using Run Tasks

Run tasks, by definition, are invoked through https calls to API endpoints. All normal precautions should be taken to first ensure the two-way handshake between the HCP Terraform service and the run task provider is possible, and then that the opening of the path has been done in a correct manner that does not open the door to any unwanted access. This will involve having the necessary firewall and security groups configured specifically for this exchange.

The run task implementation readily provides for the use of a security token in the form of a signature generated using an HMAC key that can be exchanged between the target run task configuration and the HCP Terraform run task configuration. This is a common security practice that greatly enhances ensuring that only approved and registered access between the two endpoints can take place. This capability should be taken advantage of whenever possible. If a vendor or other run task supplier has not implemented this capability, we recommend having this security token exchange added as an enhancement.

14.7.2 Tips for effective use of Run Tasks in HCP Terraform

Run tasks greatly expand the functionality and capabilities of a Terraform run workflow. The use of run tasks is highly encouraged as the mechanism to meet additional requirements or add additional desired tooling in order to further customize your workflow for your needs. Here are some recommendations on how to use them effectively, both from viewpoints of the benefits and the possible downsides.

14.7.3 Select Run Tasks to integrate based on need and benefit

Take advantage of run tasks, but do not unnecessarily complicate and burden your Terraform workflows by adding several trivial run tasks that do not add additional value. Items like compliance, code quality, standards adherence should certainly be considered. Adding a lot of internal minor tasks may be better handled via other mechanisms.

14.7.4 Manage the number and impact of Run Tasks

Every run task associated with the workflow for a workspace will add some overhead to the run time and possibly resource requirements of every run. This continues the consideration of numbers and

need, but also applies to individual run tasks. Run tasks should be quick in-and-out events that capture data and make decisions on it. Extended processing at the time of a run task invocation will cost in time a run will be active while occupying a slot that another run could be using.

14.7.5 Consider the availability of Run Tasks when selecting tools

Similar to the pattern that has developed for Terraform Providers, many of the vendors in the infrastructure related space are supplying run tasks to allow for the integration. It is a benefit to their product offerings and their customers who are already using or considering HashiCorp products. It is in your best interest to look for this integration or intent in products you are using or considering.

15 Custom Run Task Integrations

15.1 Building custom Run Task integrations

In addition to using existing technology partner's integrations, HCP Terraform customers and other interested parties can build their own custom run task integrations. Custom integrations have access to plan and apply details in the same way as pre-existing run tasks such as from the public registry. They can perform distinct tasks with that data and display custom messages within the run pipeline as well as prevent a run from continuing to the next stage.

There are many useful and valid applications of run tasks that will enhance the Terraform workflow for the entire organization or specialized to meet needs of sections such as application teams. The first place to look for run tasks that may be beneficial to your organization is in the public registry where HashiCorp and HashiCorp technology partners publish run tasks that integrate with commercial and other product offerings. Similar to using Terraform providers, your first choice of selecting and using a run task integration would be to take advantage of the one written for and by the product supplier.

When there is not an available run task to meet the specific needs or support the products and services in use, particularly internal to the organization, developing a run task is the next option. The process and activities to develop a custom run task are fairly straightforward as there is a well defined API that is the core of the integration, and several examples of the run tasks already in existence.

The primary consideration for developing a custom run task is that it should execute short-lived tasks that fit well as an event in the Terraform workflow. Run tasks that are associated with many workspaces will add some run time and overhead to each run, so you want to be careful of adding any considerable load to the platform. The run task should also be accomplishing a task that needs to be and/or is best achieved by being executed inline within the Terraform run workflow.

A primary example of custom run tasks that may be developed would be to integrate with internally built tools and services. Such as when there is a custom code scanning process, a run task could be developed that would invoke the scan and prevent the run from continuing if any corporate compliance controls were discovered that should not be followed. Or if there is a form of an inventory or change tracking process in use, a custom run task would be notified of a run event and use the details to record the activity.

15.1.1 Requirements for building a custom integration

To build a custom integration, you must have an online process capable of receiving API requests from HCP Terraform and responding with a properly formed response containing the status and other pertinent information to a supplied callback URL. When creating/configuring a run task in an organization, the requester supplies an endpoint url that will be available to receive requests from HCP Terraform. A test POST request will be sent to the supplied URL and the run task must respond with a success code 200 for the run task to be created. That is how you will ensure that a run task and all the plumbing to communicate with it has been properly deployed.

The payload supplied to the endpoint will contain a url pointing to the plan or apply data in a JSON output. The integrated service must be able to interpret and process the information provided in order to perform its specific actions.

More details on developing the integration required for a custom run task can be found here: Run Tasks Integration

Since the run task interface is through standard API exchange, any programming language can be used that supports this. Hosting the API endpoint can also be done with any mechanism supporting such services, and will be your responsibility. This template can be used to get started on developing a custom run task using golang: terraform-run-task-scaffolding-go

15.2 The Run Tasks integration API

The exchange between HCP Terraform and the run task is defined by the run tasks integration API, documented here: Run Tasks Integration - API Docs - HCP Terraform. Your custom run tasks must fully support and be able to interact with this API as prescribed. It is the contract between HCP Terraform and any run task.

15.2.1 Run Task flow of events

- HCP Terraform will build and populate the initial request to start the run task that has been configured and issue it to the integration endpoint. The run task integration that is listening for and responding to such requests is responsible to acknowledge the request and return a series of responses until a final response that reflects the final status.
- In the response body, there is the ability to supply the current status, that could be "running" for an interim progressive response, or must be "passed" or "failed" for the final response. Basic status description/messages can and should be included.
- At any time, an "outcomes" block can be included in the payload. However, it is a good practice to only send outcomes as part of the progressive status when a run task is running. Run task outcomes can contain tags named "severity" or "status" which will then enrich the outcomes displayed in HCP Terraform, enabling a quicker response to issues containing severity and status.

The suggested sequence of the exchange is as follows:

HCP Terraform	Run Task Callback from Integration
Run Task Request with payload	200 OK - No Content
	200 OK - status running with progress outcomes (optional/multiple)
	200 OK - status passed or failed with final results
Finish run task reacting to status by continuing or failing the run	

The initial request will contain an access-token value that is used as the authentication mechanism between HCP Terraform and this instance of the run task invocation. HCP Terraform expects you to callback to the supplied task_result_callback_url using the access_token as an Authentication Header. HCP Terraform expects to receive the callback within 10 minutes, or the task will be considered to have errored.

Sample run task flow transactions and sequences are located in the API documentation.

15.2.2 Best practices for building and testing Custom Run Task integrations

Avoid long-running Run Tasks

As mentioned previously, run tasks should not take a lot of time as that will increase the overall time a Terraform run will take and will be multiplied by how many workspaces are associated and how often runs occur. If a lot of processing is needed for an activity, an alternative would be to use the run task to store enough information for the event to allow another off-cycle process to come in later and do the heavy lifting.

Develop Run Tasks that behave well

Run tasks will potentially execute numerous times a day, often impacting critical infrastructure. The process should be designed to be robust and as straightforward as possible, minimizing the chance of failure. Well-executed run tasks can provide significant benefits, but incorporating a poorly performing process into your workflow can create more issues than it solves.

Develop test suites

Similar to automated infrastructure building, the primary way to test run tasks is with real terraform runs. In line with the best practices for software delivery, it's essential to have repeatable and comprehensive testing for custom run tasks.

To effectively simulate the infrastructure scenarios for run tasks, it may be necessary to create and populate one or more workspaces. These should be designed to yield both successful and failing outcomes from the run tasks, encompassing as wide a range of test cases as possible. Specifically, if the run tasks involve scanning code, a test harness could be developed to generate various code scenarios. This harness would allow the same workspace(s) to be used repeatedly for testing purposes.

16 No-code provisioning management

> Note
>
> This page covers operator setup and management of no-code provisioning. For how users consume published no-code modules, see the User Guide: No-code provisioning.

No-code provisioning is a simplified option allowing non-practitioners to provision and manage infrastructure leveraging your existing Terraform codebase. Developers can now deploy infrastructure using Terraform without having to learn the language, in a self-service fashion. We recommend that No-Code modules are made available for common patterns/Use Cases (e.g RDS database, Java Web Application that can be used by end users to deploy infrastructure rapidly without having to write terraform code.

No-code provisioning allows you to take the infrastructure patterns you've developed and codified as Terraform modules and easily repackage them so that a wider population of users in your organization can benefit from those building blocks.

Updates to the pattern can easily be published and applied to existing no-code workspaces as HCP Terraform will notify every workspace that uses the module that a newer version is available. The user can then view the update change request, and apply the changes to the deployed infrastructure.

Please review the official documentation for more details on this capability.

16.1 Implementation guidance

The no-code module feature relies on the private module registry. You must have adopted it in your workflow and have established practices in place to test and release modules in the private registry.

When publishing a no-code module to the private registry or convert an existing private module as no-code, you must explicitly add the it to the no-code provision allow-list. This can be done using the WebUI, the API, the TFE provider (tfe_no_code_module).

16.1.1 Roles and responsibilities

The platform team's responsibilities are to:

- Create and maintain no-code modules (including publishing module version updates as necessary).
- Update the HCP Terraform configuration to allow users to use no-code modules to deploy infrastructure in a self-service manner.

The no-code consumers' responsibilities are to:

- Create infrastructure in a self-service model with no-code modules
- Review the update request when notified that a no-code workspace they created uses a module that was updated.

16.1.2 Permissions requirements

Consumers of no-code modules must have the Maintain permission set on the project(s) where the no-code modules are deployed.

16.1.3 Credentials for provisioning

Because no-code modules are used to provision infrastructure, you will need credentials with an appropriately scoped role (and permissions).

> Tip
>
> HashiCorp recommends the use of dynamic credentials, although you may also use IAM roles or static credentials (which should be your least favored option).

16.1.4 Configuring at scale

Project

The no-code feature depends on a properly configured project where the user can create no-code workspaces. The platform team must set up this project with the appropriate permissions for the users (Maintain permission set).

Project-scoped variable sets

The project must be assigned a variable set with the properly scoped credentials necessary to configure the infrastructure.

Workspace Settings

When used, no-code modules will result in a workspace which will inherit settings that are established for the org/project including execution mode, default policies and run-tasks. These can be updated after the workspace is created by the admin (platform or project admin) team.

Day 2 operations

Managing and upgrading no-code provisioned infrastructure involves several key processes to ensure smooth operations and minimal disruption. When a new module version becomes available, admins and module publishers must change the designated no-code version for the module in its no-code configuration settings. When a new version is available, no-code users will receive an upgrade notification in their workspace. We recommend that no-code module users upgrade to the latest version of the module as soon as it is feasible after thorough testing in their lower environments.

During the upgrade process, users may encounter new variables that require inputs or current variables that require different inputs. They will be guided through a provisional plan, and if the plan doesn't have an error it will auto-apply, which updates the module version used in the workspace. Currently, Terraform does not support rolling back to a previous module version if an upgrade does not go as planned.

Tip

HashiCorp recommends that admins and module authors thoroughly test new module versions before designating them as the no-code version. Additionally, many organizations practice applying infrastructure changes in separate lower environment workspaces before applying them in the Prod workspaces.

17 Changelog

17.1 2026-08

- Restructured legacy Terraform HVD content into the HVD 2.0 Terraform Administration Guide as part of the Installation / Administration / User guide migration.

17.2 2026-01

- Update meta descriptions
- Updated credits and contributors

17.3 2025-11

- Updated Terraform Stacks guidance
- Fixed typos and documentation errors

17.4 2025-10

- Add new Terraform Stacks content and adapt existing content

17.5 2025-09

- Reorganized Terraform Stacks documentation structure

17.6 2025-07

- Updated Terraform import guidance

17.7 2025-05

- Removed document structure section

17.8 2025-04

- Fixed code block formatting

17.9 2025-03

- Updated documentation links

17.10 2024-12

- Expanded section on VCS connection in post-installation tasks

17.11 2024-04

- Updated Terraform Cloud to HCP Terraform

17.12 2024-01

- Migrate to new repository with minor changes and corrections

17.13 2023-11

- Update credits

17.14 2023-10

- GA Release

18 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated

this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adam Cavaliere
- Adeel Ahmad
- Alex Basista
- Amy Brown
- Anna Chen
- Brandon Ferguson
- Chris Steinmeyer
- Chris Stella
- CJ Obermaier
- Emmanuel Rousselle
- Fraser Pollock
- James Purcell
- Jenna Degaust
- Judith Malnick
- Kalen Arndt
- Kranthi Katepalli
- Kyle Seneker
- Mark Lewis
- Michelle Roberts
- Mike Augustine
- Monica Miller
- Nada Elkelani
- Prakash Manglanathan
- Purna Mehta
- Randy Keener
- Richard Russell
- Salil Subbakrishna
- Sean Doyle
- Sean Walker
- Shriram Rajaraman
- Shyam Chaganti
- Simon Lynch

- Yogeshprabhu Jagadeesan
- Zachary Isom
- Zeeshan Ratani
- Zeshan Ratani